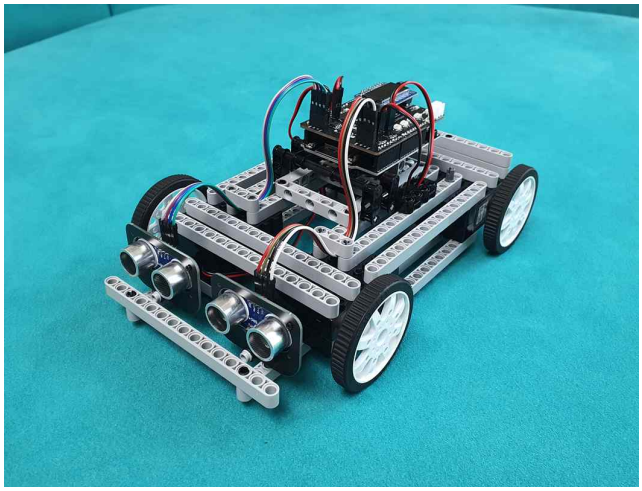
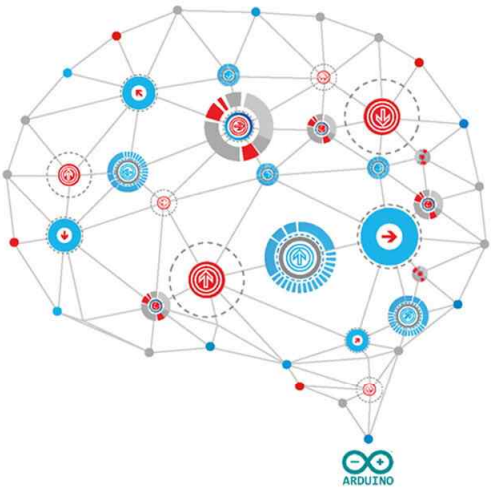
부록



아두이노로 구현해 보면서 인공지능의 딥러닝 원리를 알기 쉽게 이해하는 방법!



아두이노로 코딩하며 배우는 딥러닝

머신러닝과 **딥러닝** 원리와 모델을 78개 **아두이노 예제**로 직접 구현

서민우 지

앤써북

부록. 초음파 센서 자율주행 RC카 프로젝트

Chapter 01 아두이노 인공지능 자동차 준비하기

01 아두이노 인공지능 자동차 조립하기

01 부품 살펴보기
02 인공지능 자동차 조립하기

02 아두이노 인공지능 자동차 살펴보기

01 ESP32 아두이노 살펴보기
02 아두이노 인공지능 쉴드 핀 살펴보기

03 아두이노 개발 환경 구성하기

01 아두이노 소프트웨어 설치하기
02. USB 드라이버 설치하기
03 ESP32 아두이노 패키지 설치하기
04 아두이노 스케치 작성해 보기
05 아두이노 오류 발생시 대처방법
06 아두이노 스케치 구조 이해하기

Chapter 02 아두이노 인공지능 자동차 살펴보기

01 문자열 내보내기

01 여러 형식의 자료 내보내기
02 여러 형식의 숫자 내보내기

02 LED 켜고 끄기

01 LED 켜고 끄기 확인하기
02 LED 켜고 끄기 간격 줄여보기
03 LED 켜고 끄기를 밝기로 느껴보기
04 LED 어둡게 하기
05 LED 밝게 하기

03 LED 밝기 조절하기

01 주파수 조절로 LED 켜고 끄기 반복해 보기
02 상하비 조절로 LED 밝기 조절해 보기

04 DC 모터 제어하기
01 DC 모터 제어해 보기

05 사용자 입력 받기
01 사용자 입력 받기

06 버튼 상태 읽기
01 0, 1 읽어보기

07 빛 센서로 밝기 감지하기
01 빛 센서 읽기

08 초음파 센서로 거리 측정하기
01 초음파 센서 읽고 거리 계산하기

09 RGB 네오픽셀 LED 제어하기
01 NeoPixel 라이브러리 설치하기
02 RGB 네오픽셀 LED 제어해 보기

10 RGB LCD 제어하기
01 ST7735 라이브러리 설치하기
02 RGB LCD 제어해 보기

Chapter 03 초음파 센서 인공지능 자동차 프로젝트

01 데이터 수집하기
01 초음파 센서 자동차 테스트하기
02 데이터 수집하기

02 Torch로 학습시키기
01 수집된 데이터 확인하기
02 학습 시키고 모델 내보내기
03 NumPy로 예측해 보기
04 학습된 가중치 내보내기

03 아두이노에서 예측하기

01 아두이노에서 예측해 보기
02 초음파 센서 자동차에 인공지능 적용하기

Chapter 04 라인트레이서 인공지능 자동차 프로젝트

01 적외선 송수신 센서로 검정 선 감지하기

01 적외선 송수신 센서 읽고 검정색 감지하기

02 데이터 수집하기

01 라인트레이서 자동차 테스트하기
02 데이터 수집하기

03 Torch로 학습시키기

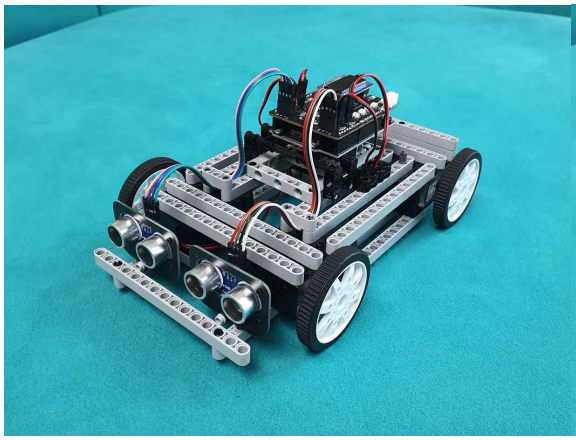
01 수집된 데이터 확인하기
02 학습 시키고 모델 내보내기
03 NumPy로 예측해 보기
04 학습된 가중치 내보내기

04 아두이노에서 예측하기

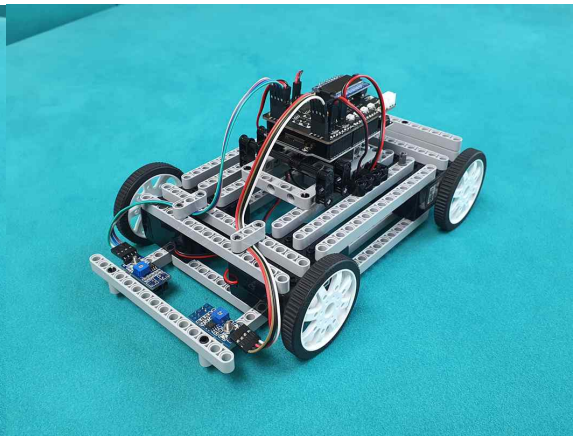
01 아두이노에서 예측해 보기
02 라인트레이서 자동차에 인공지능 적용하기

Chapter 01
아두이노 인공지능 자동차 준비하기

이번 장에서는 아두이노 인공지능 자동차를 조립하고, 아두이노 개발 환경을 구성합니다. 다음과 같이 2가지 형태의 인공지능 자동차로 구성할 수 있습니다.



<아두이노 초음파 센서 인공지능 자동차>



<아두이노 인공지능 라인트레이서>

01 아두이노 인공지능 자동차 조립하기

여기서는 아두이노 인공지능 자동차 부품을 살펴보고 조립합니다.

01 부품 살펴보기

이 책에서 사용될 아두이노 인공지능 자동차 부품 구성은 다음과 같습니다.

전자부품















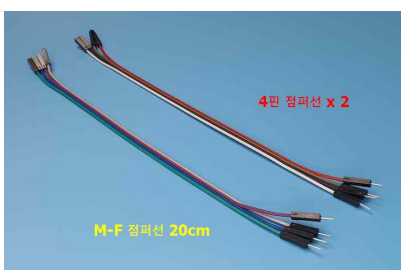




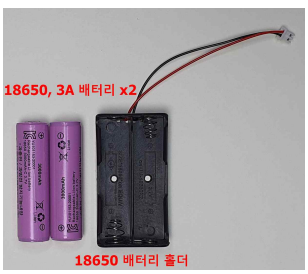




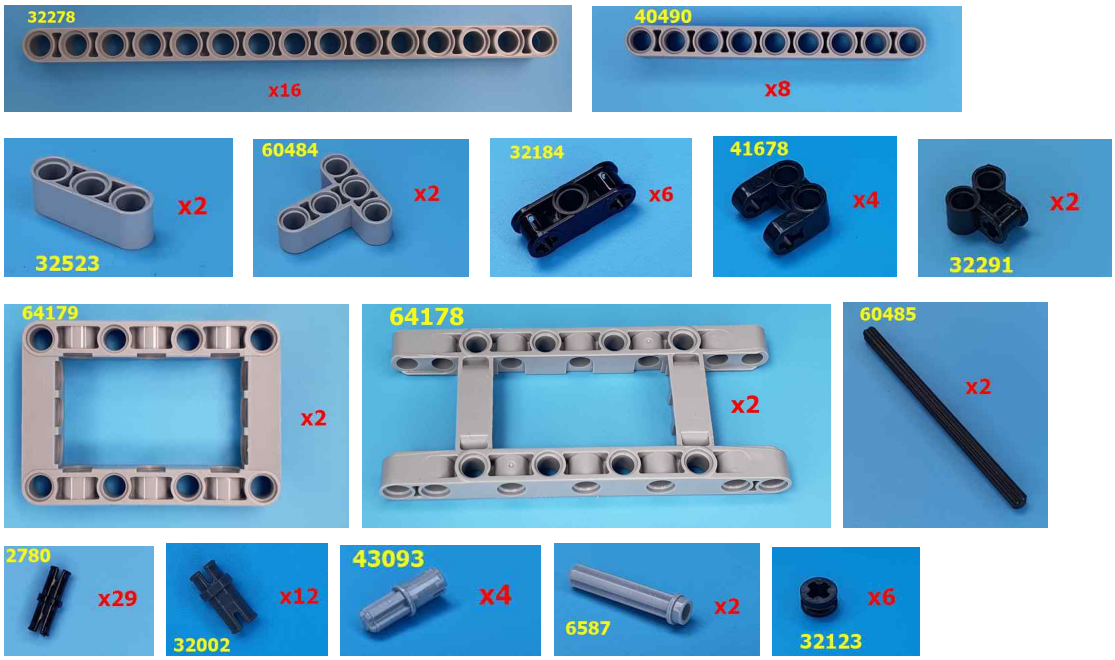








레고부품

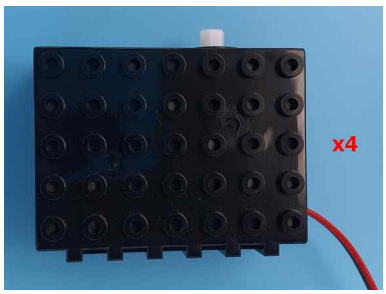


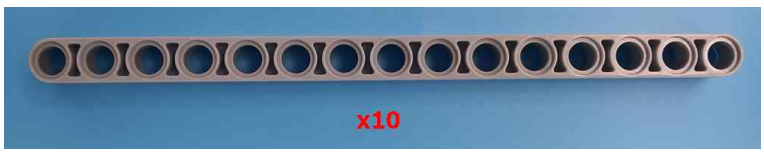
02 인공지능 자동차 조립하기

여기서는 차량 몸체 조립, 아두이노 부 조립, 초음파 센서 부 조립, 적외선 송수신 센서 부 조립, 차량 지지대 조립의 순서로 RC카를 조립합니다.

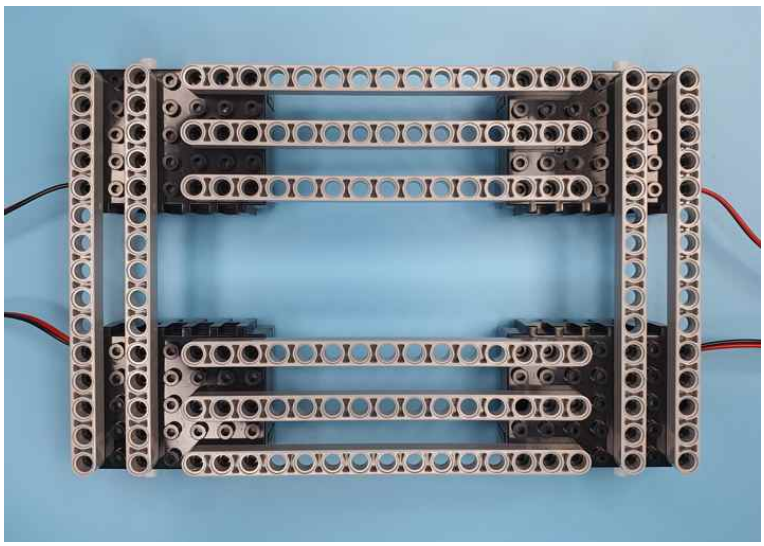
차량 몸체 조립하기

1. 다음과 같이 DC 모터 4개, 1x15구 레고 10개를 준비합니다.

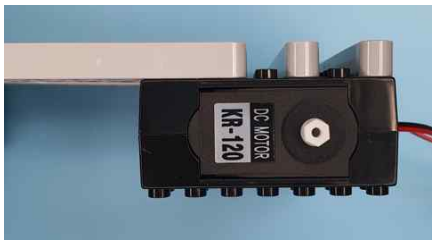




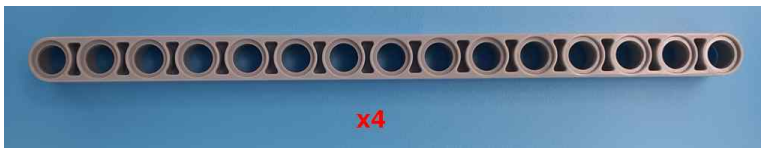
2. 다음과 같이 1x15구 레고 10개를 DC 모터 4개에 장착합니다.

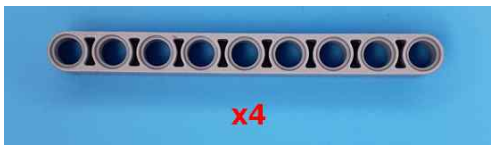


다음은 옆에서 본 모양입니다. 레고부품을 DC 모터에 단단히 밀착시킵니다.

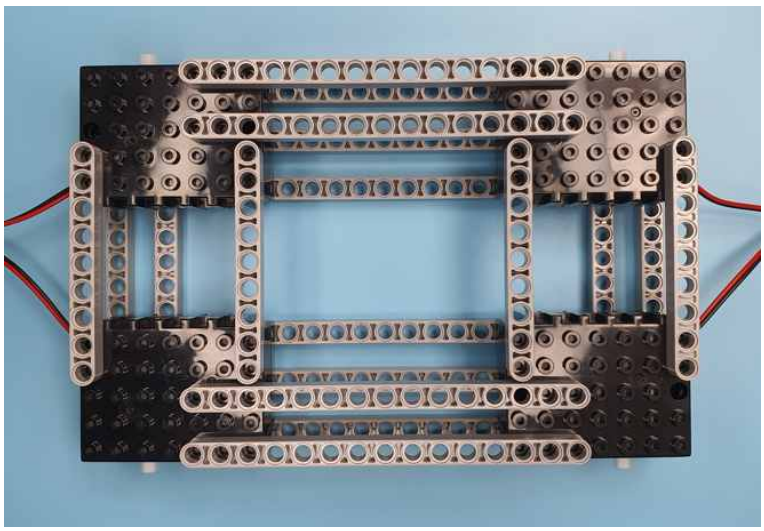


3. 다음과 같이 1x15구 레고 4개, 1x9구 레고 4개를 준비합니다.

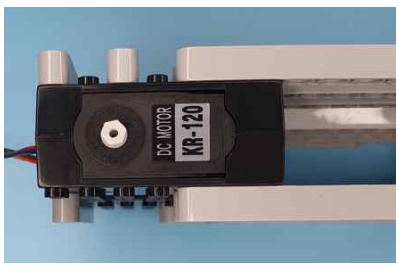




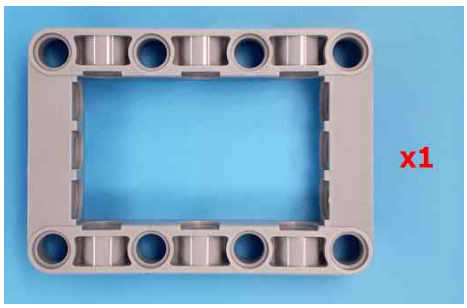
4. 다음과 같이 자동차 몸체의 하단부에 장착합니다.



다음은 옆에서 본 모양입니다. 레고부품을 DC 모터에 단단히 밀착시킵니다.

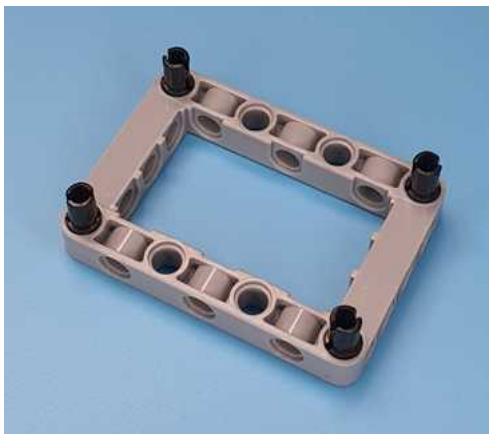


5. 다음과 같이 2가지 레고부품을 준비합니다.

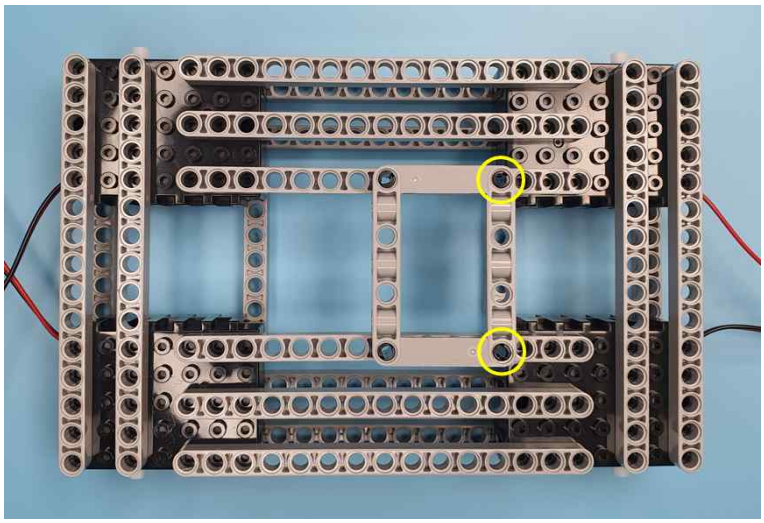




6. 다음과 같이 조립합니다.

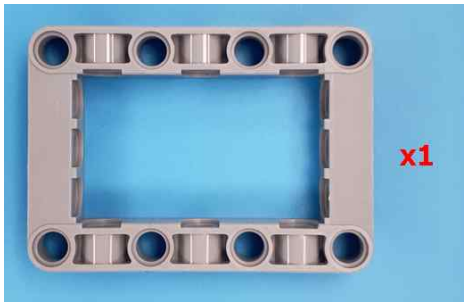


7. 자동차의 상단부에 다음과 같이 장착합니다. 이 부분은 앞 부분이 됩니다.



*** 원 표시 부분이 하단부 레고의 4번째 홈에 오도록 장착합니다.

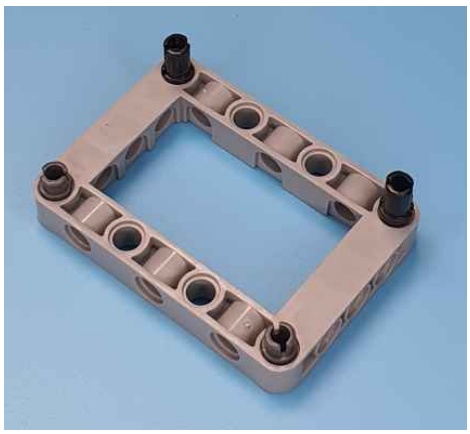
8. 다음과 같이 3가지 레고부품을 준비합니다.



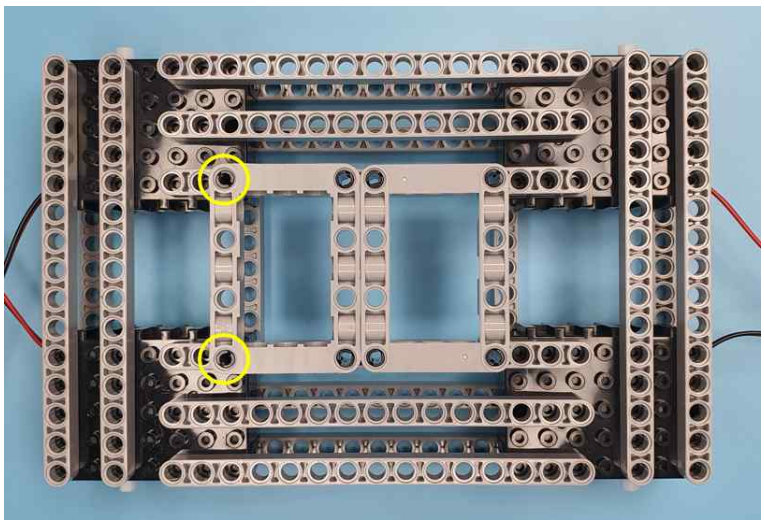




9. 다음과 같이 조립합니다.

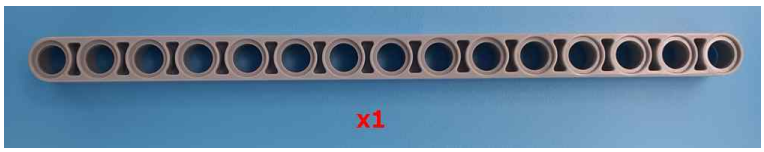


10. 자동차의 상단부에 다음과 같이 장착합니다. 이 부분은 뒷 부분이 됩니다.



*** 원 표시 부분은 짧은 형태의 레고 부품입니다. 이 부분이 하단부 레고의 3번째 홈에 오도록 장착합니다.

11. 다음과 같이 2가지 레고부품을 준비합니다.

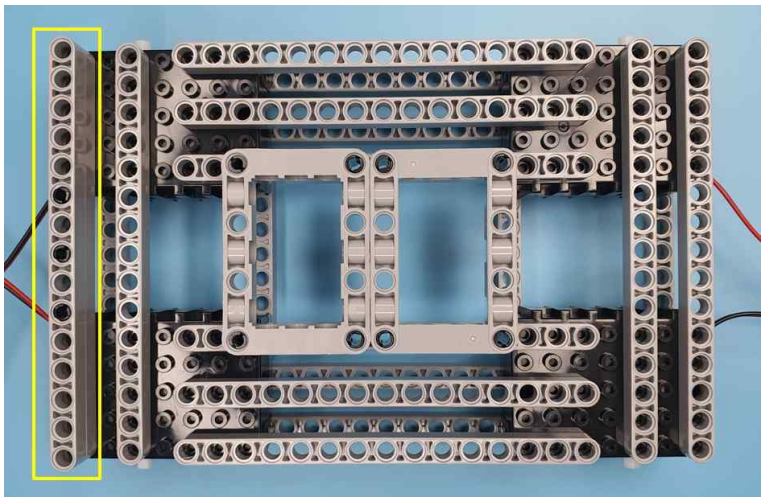




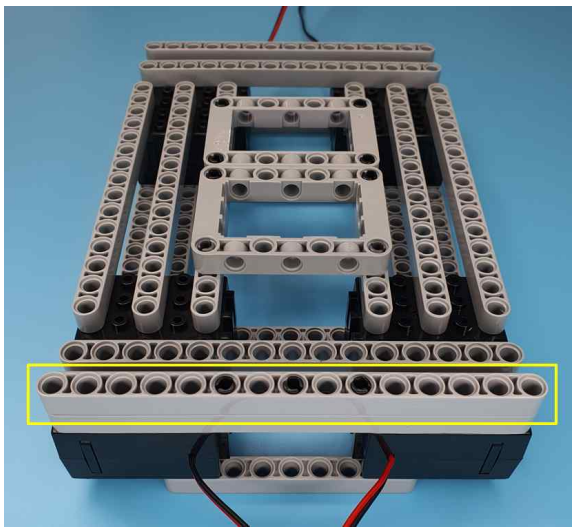
12. 다음과 같이 조립합니다.



13. 자동차의 상단부 뒷부분에 다음과 같이 장착합니다. 이 부분은 배터리 홀더 가드가 됩니다.



다음은 뒤에서 본 모양입니다.

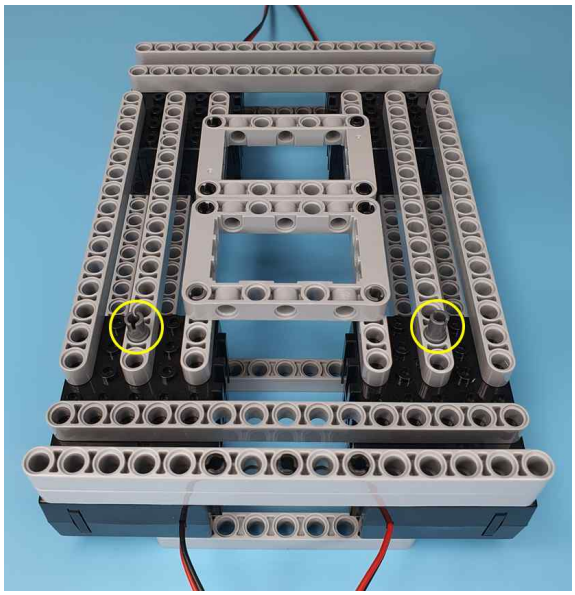


Kocolabs
www.kocolabs.co.kr

14. 다음과 같이 2가지 레고부품을 준비합니다.



15. 자동차의 상단부 뒷부분에 다음과 같이 장착합니다. 이 부분은 배터리 홀더 가드가 됩니다.



다음은 확대한 사진입니다. 레고 부품의 짧은 부분을 홈에 장착합니다.



15. 다음과 같이 2가지 레고부품을 준비합니다.

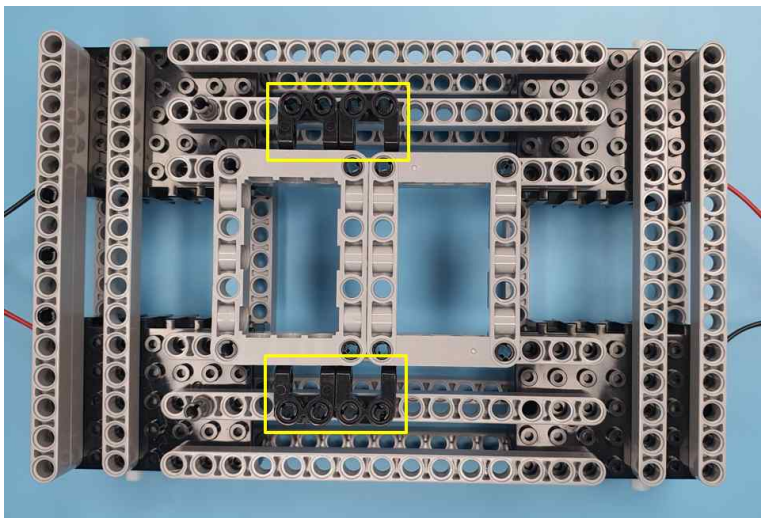




16. 다음과 같이 조립합니다.

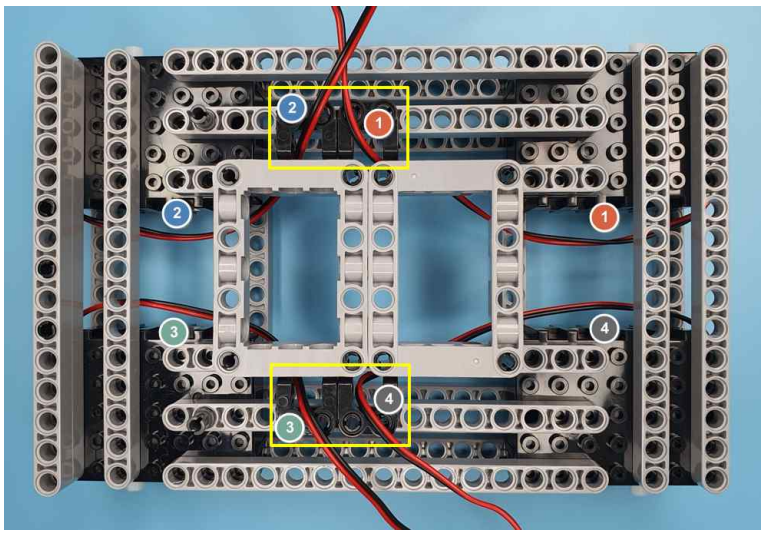


17. 자동차의 상단부 중간 뒷부분에 다음과 같이 장착합니다.

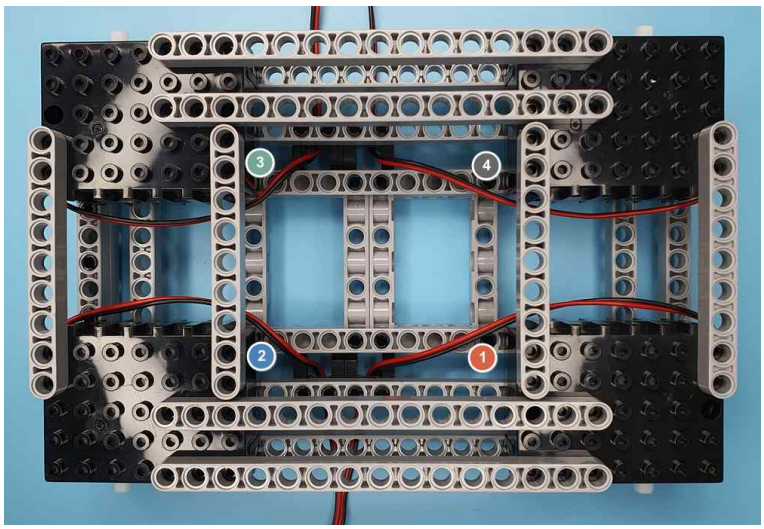


Kocolabs
www.kocolabs.co.kr

18. 다음과 같이 모터 전원선을 빼냅니다.



다음은 자동차 아래에서 본 모터 전원선을 정리한 모양입니다.



Kocolabs
www.kocolabs.co.kr

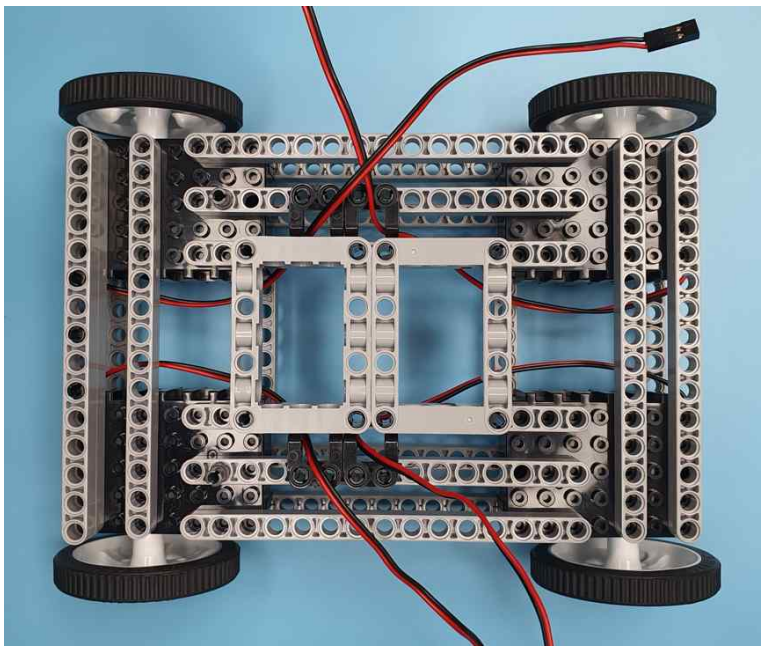
19. 다음과 같이 2가지 부품과 스크류 드라이버를 준비합니다.





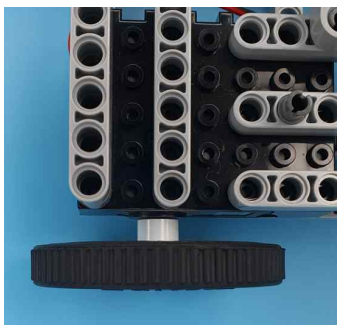


20. 다음과 같이 4개의 바퀴를 장착합니다.



다음은 옆과 위에서 바퀴가 장착된 모양입니다.





*** 바퀴가 헐겁지 않도록 나사를 적당히 조여줍니다.

Kocolabs
www.kocolabs.co.kr

21. 다음과 같이 2가지 레고부품을 준비합니다.

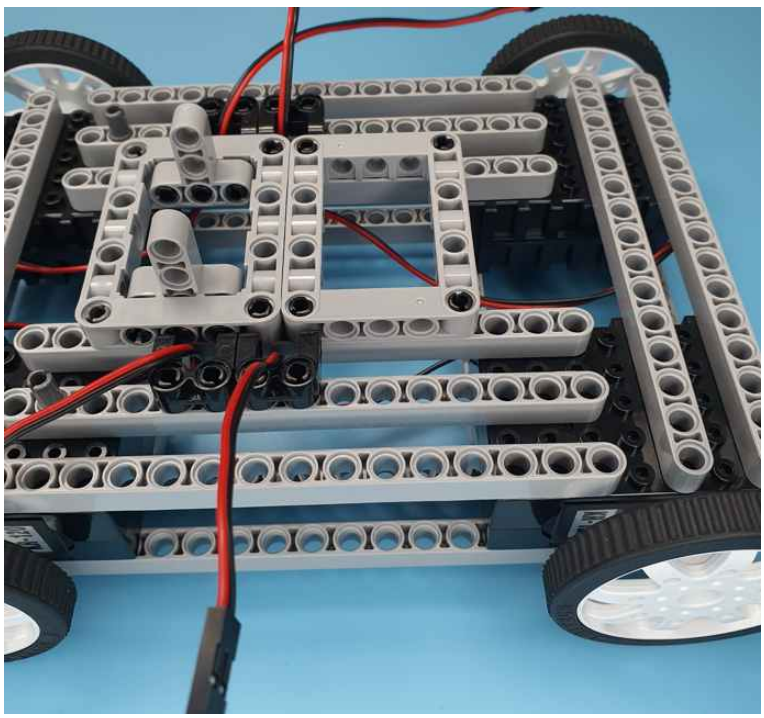




22. 다음과 같이 조립합니다.



23. 자동차의 상단부 뒷부분에 다음과 같이 장착합니다.

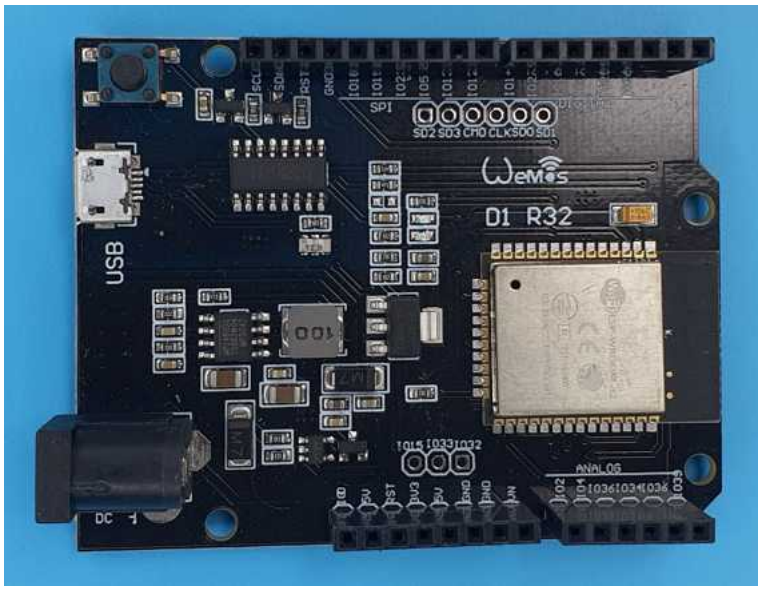


*** 이 부분은 아두이노의 지지대 역할을 합니다.

아두이노부 조립하기

다음은 아두이노부를 조립합니다.

1. 다음과 같이 아두이노 보드, 레고부품, 볼트, 너트를 준비합니다.



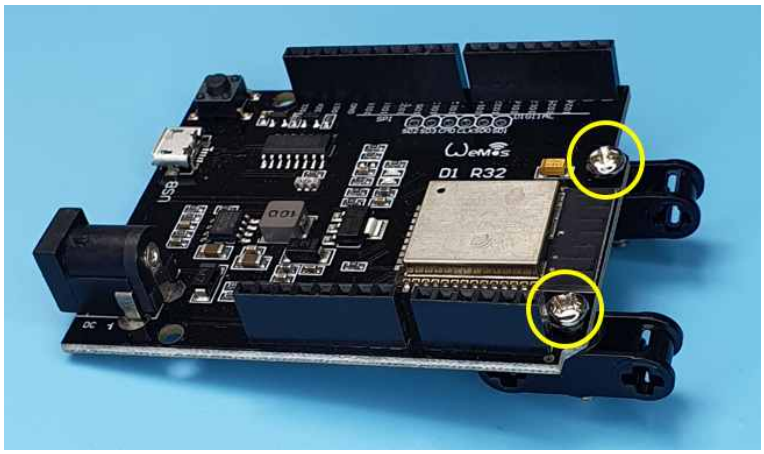




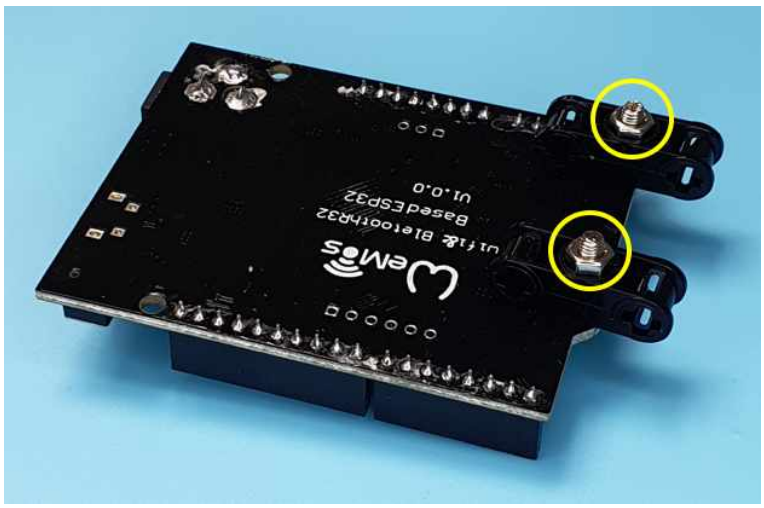


*** 부록에서는 아두이노 ESP32(Wemos D1 R32)를 사용합니다. 아두이노 ESP32는 높은 CPU 성능과 많은 메모리가 제공되어 인공지능 실습에 적합한 환경을 제공합니다.

2. 다음과 같이 볼트를 아두이노 보드와 레고 부품에 끼워 넣습니다.



3. 볼트에 너트를 느슨하게 끼워넣습니다.

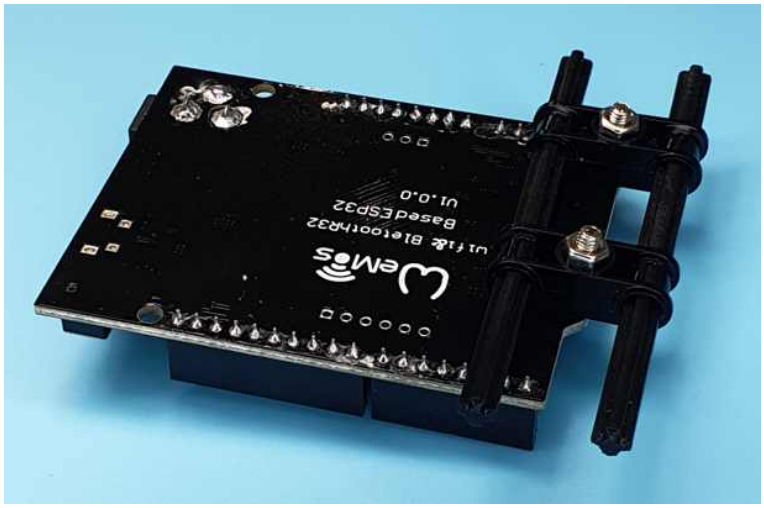


*** 여기서는 일단 적당히 느슨하게 끼워 넣도록 합니다. 뒤에서 추가 부품 장착후 조여주게 됩니다.

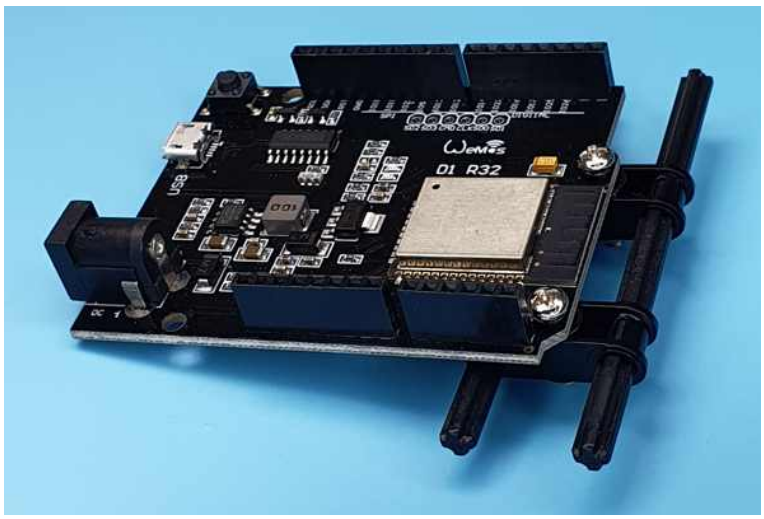
4. 다음과 같이 레고부품을 준비합니다.



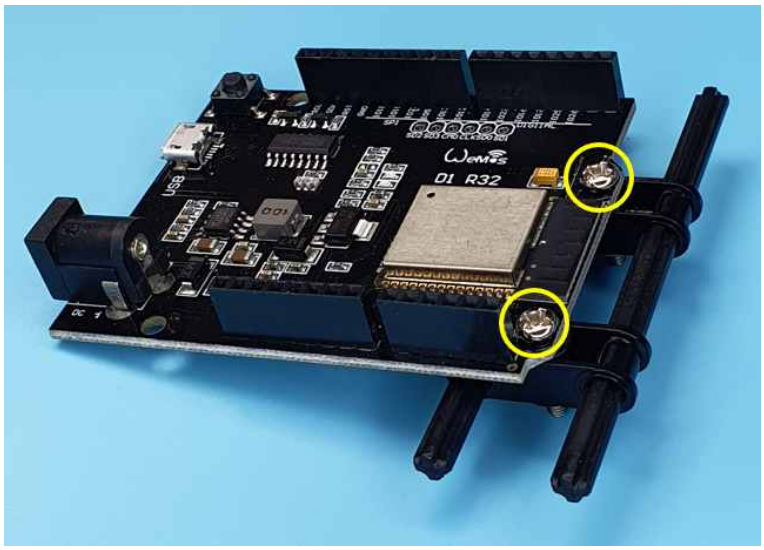
5. 레고 부품을 다음과 같이 끼워 넣습니다.



다음은 위에서 본 모양입니다.



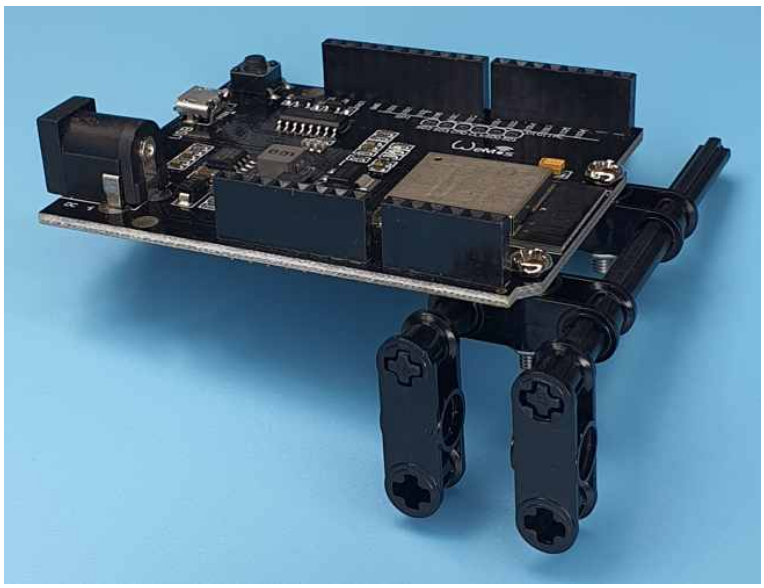
6. 스크류 드라이버를 이용하여 나사를 적당히 단단히 조여줍니다.



7. 다음과 같이 레고부품을 준비합니다.



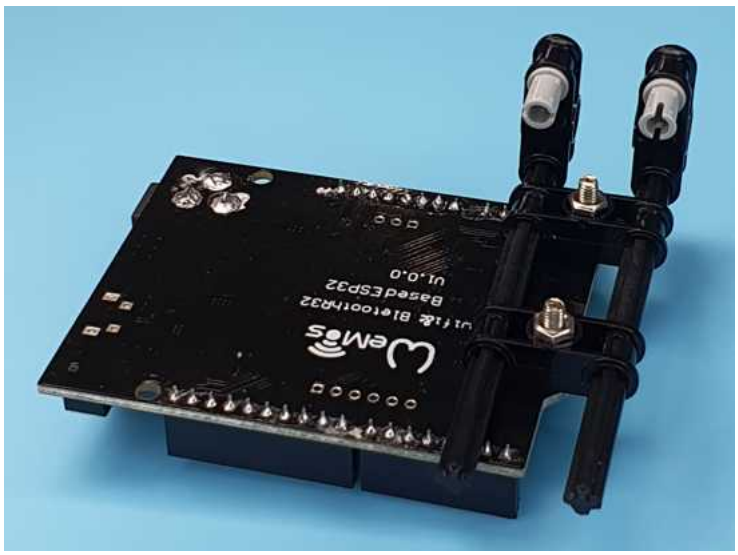
8. 다음과 같이 아두이노부에 장착합니다.



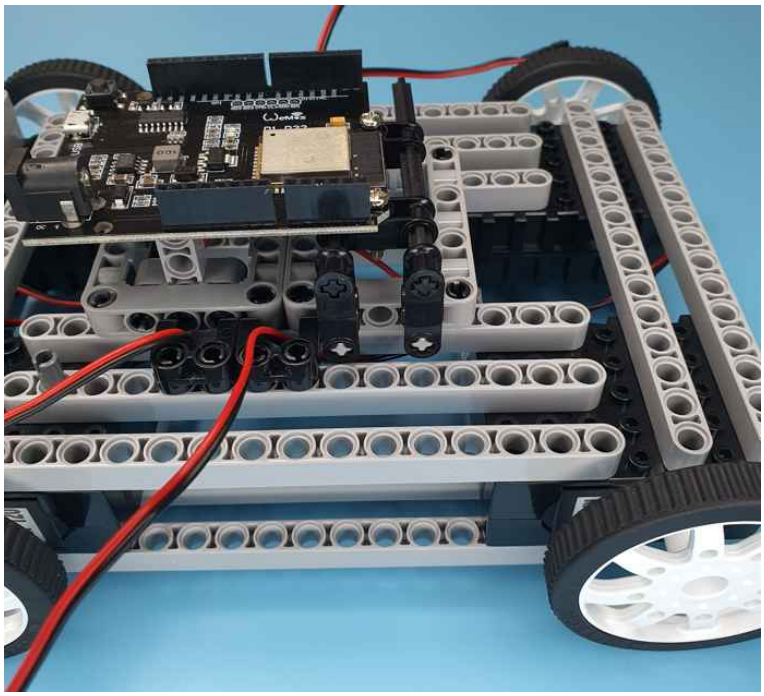
9. 다음과 같이 레고부품을 준비합니다.



10. 다음과 같이 아두이노부에 장착합니다.

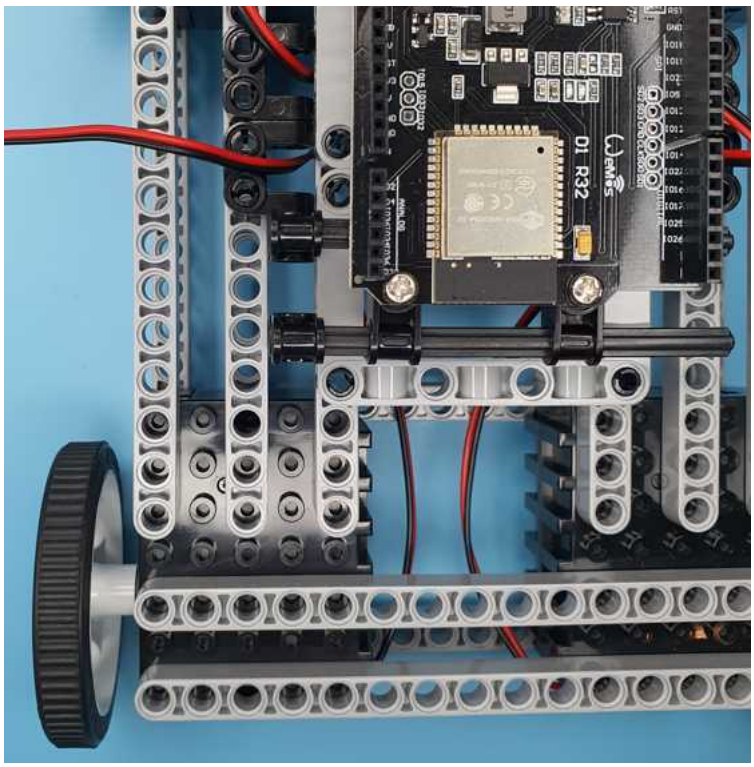


11. 아두이노부를 다음과 같이 자동차 몸체 앞 부분에 장착합니다.



Kocolabs
www.kocolabs.co.kr

다음은 위에서 본 모양입니다.



12. 다음과 같이 2가지 레고부품을 준비합니다.

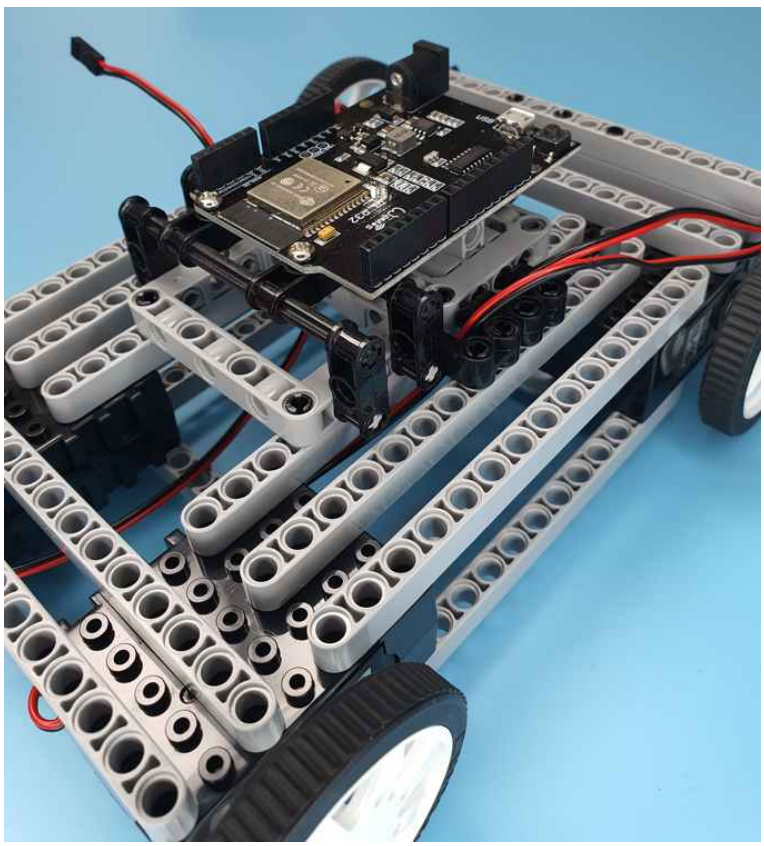




13. 다음과 같이 조립합니다.

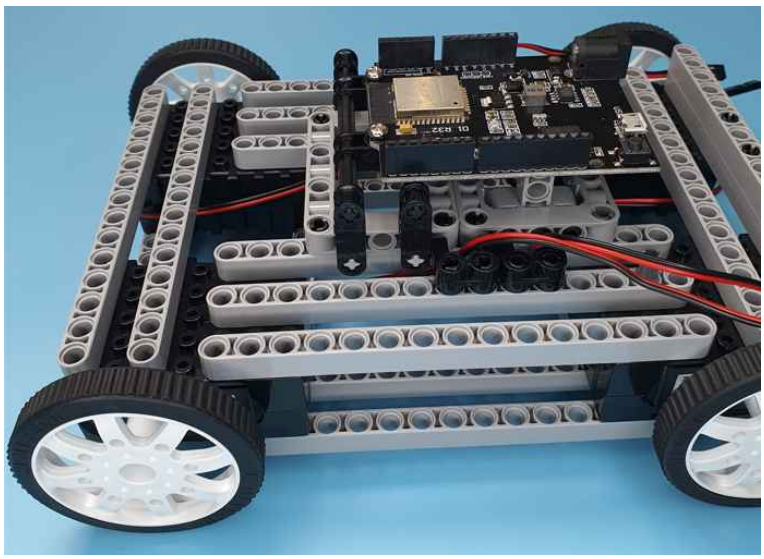


14. 다음과 같이 아두이노부와 자동차 몸체를 연결 장착합니다.

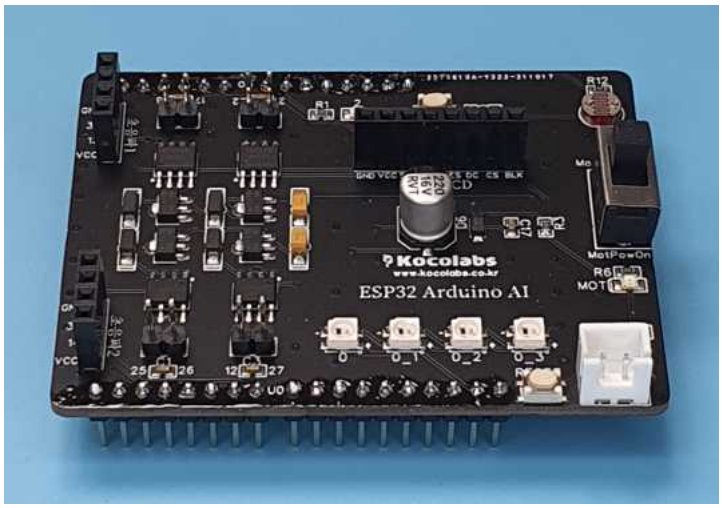


Kocolabs
www.kocolabs.co.kr

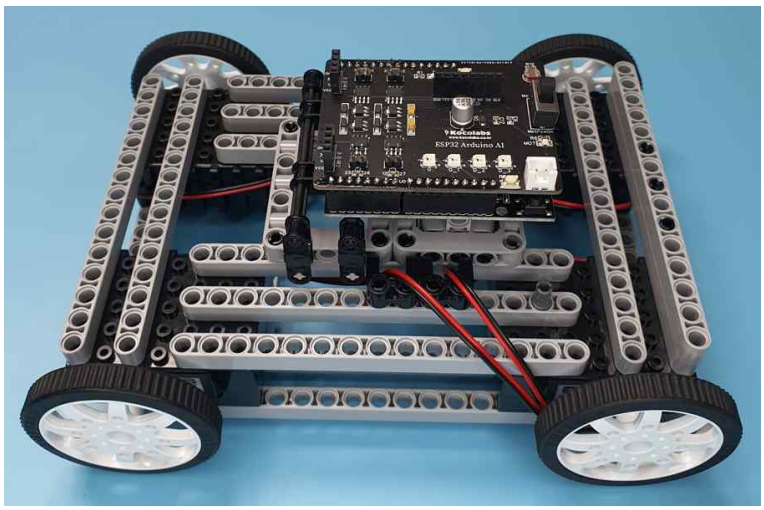
다음은 옆에서 본 모양입니다.



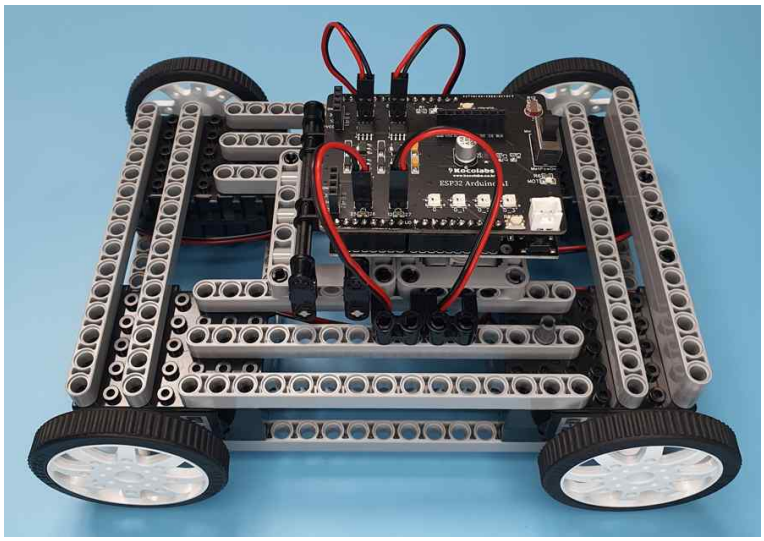
15. 다음과 같이 아두이노 쉴드를 준비합니다.



16. 다음과 같이 아두이노에 쉴드를 장착합니다.



17. 다음과 같이 모터 전원선을 연결합니다.

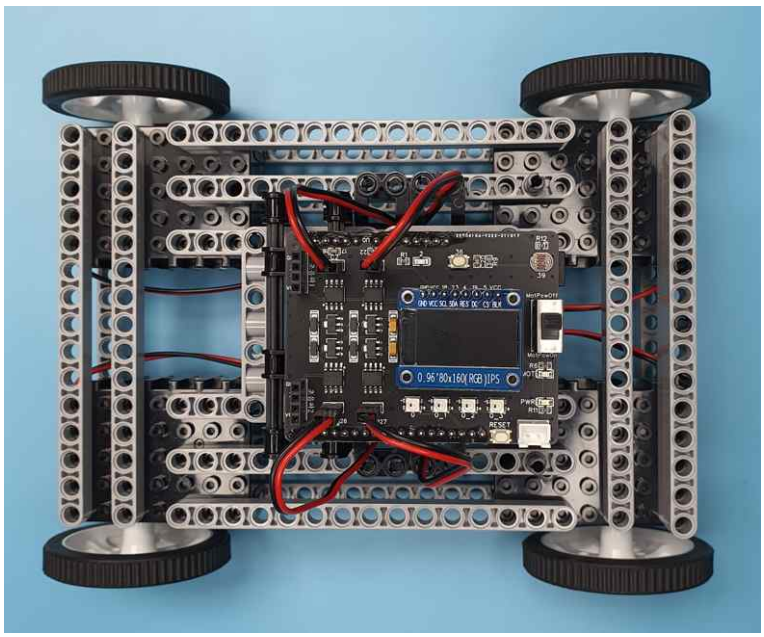


*** 전원선 연결시 선의 방향을 맞춰줍니다. 이 책에서는 모터 연결선 금속 면이 마주보도록 장착하고 있습니다. 전원선 연결 방향에 따라 모터의 방향이 바뀔수 있습니다.

18. 다음과 같이 LCD 부품을 준비합니다.



19. 다음과 같이 아두이노부에 장착합니다.



*** LCD 장착시 핀이 어긋나지 않도록 장착합니다.

초음파 센서부 조립하기

이제 거리 측정을 위한 초음파 센서부를 조립하고 장착합니다.

1. 다음과 같이 초음파 센서 관련 부품과 스크류 드라이버를 준비합니다.







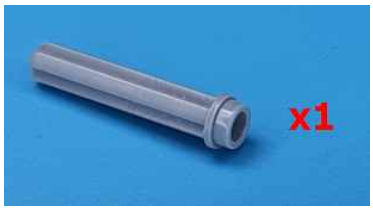


2. 다음과 같이 조립합니다.



*** 대각선 방향으로 2군데만 나사를 사용하면 됩니다.

3. 다음과 같이 2가지 레고부품을 준비합니다.

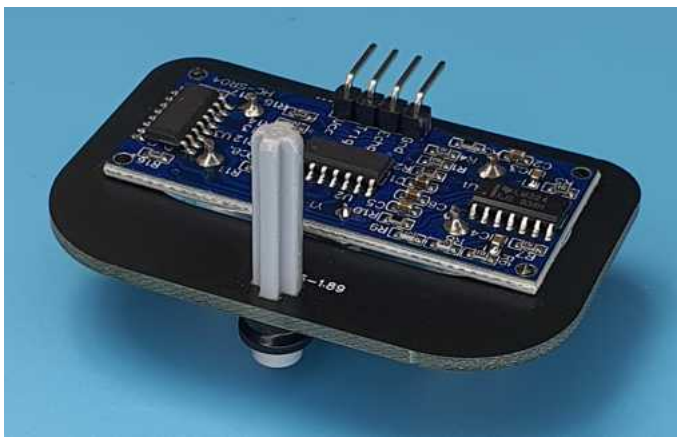




4. 다음과 같이 조립합니다.



5. 다음과 같이 초음파 센서에 장착합니다.



*** 레고 부품을 끼워 넣을 때 조금은 빡빡할 수 있으며 이때는 딱딱한 책상 면을 이용하여 힘있게 밀어넣어줍니다.

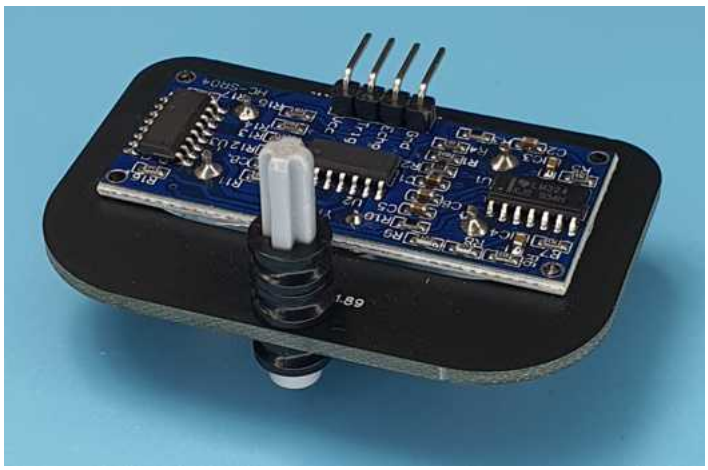
다음은 앞에서 본 모양입니다.



6. 다음과 같이 레고부품을 준비합니다.

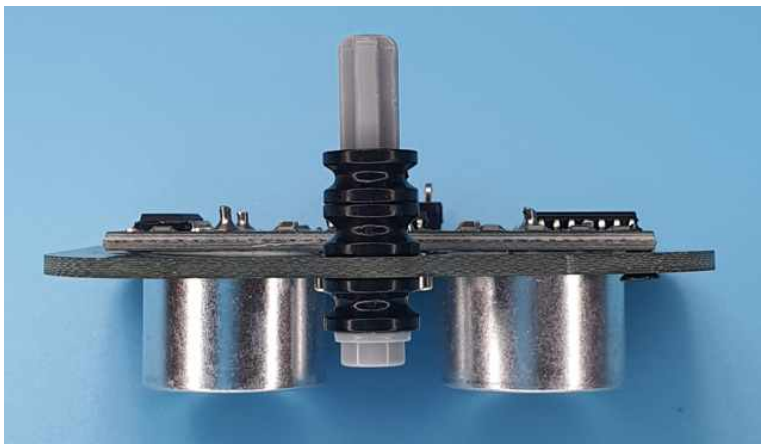


7. 다음과 같이 초음파 센서부에 장착합니다.



Kocolabs
www.kocolabs.co.kr

다음은 아래에서 본 모양입니다.



다음과 같이 2개를 조립합니다.



8. 다음과 같이 2가지 레고부품을 준비합니다.



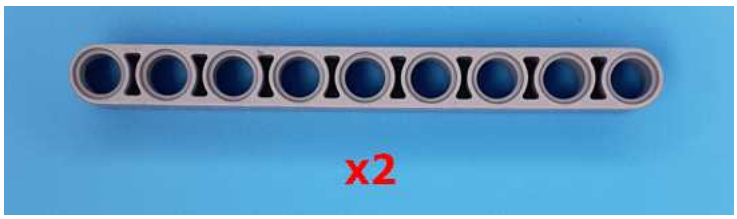


9. 다음과 같이 조립합니다.

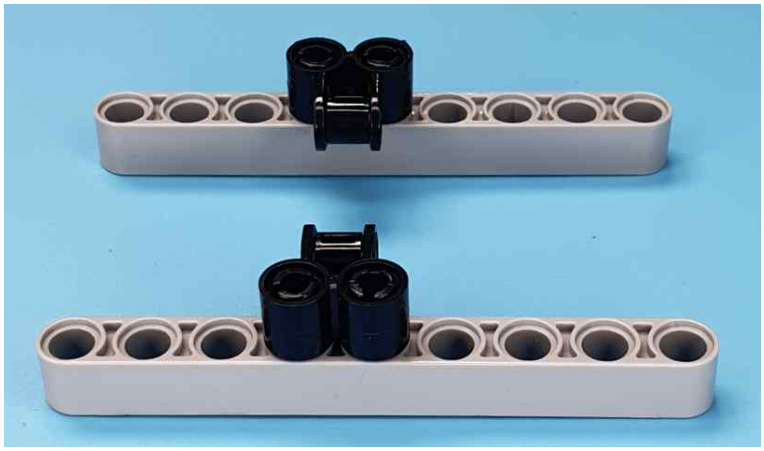


Kocolabs
www.kocolabs.co.kr

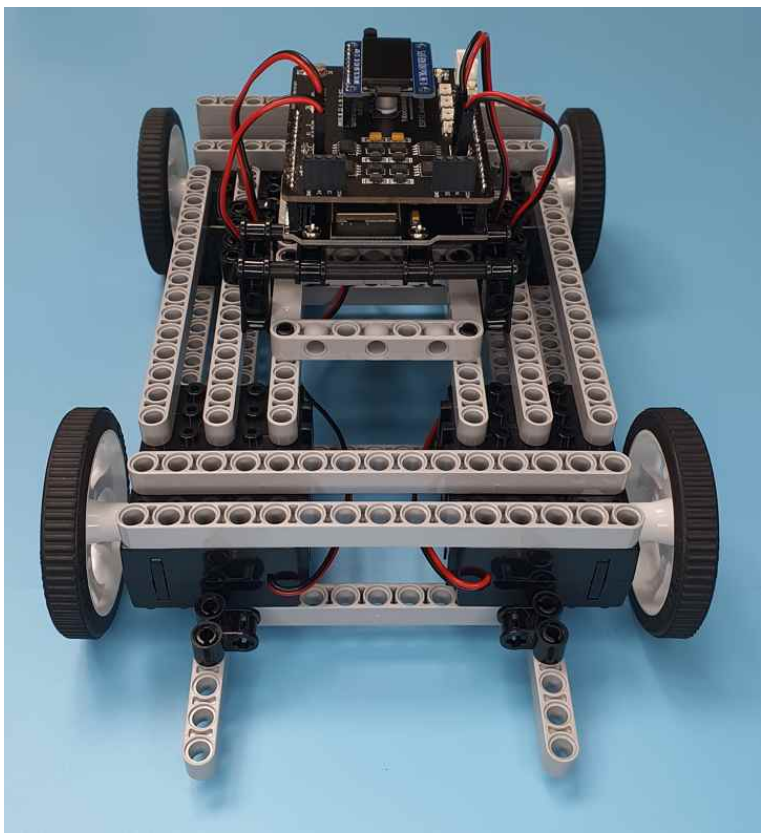
10. 다음과 같이 레고부품을 준비합니다.



11. 다음과 같이 조립합니다.

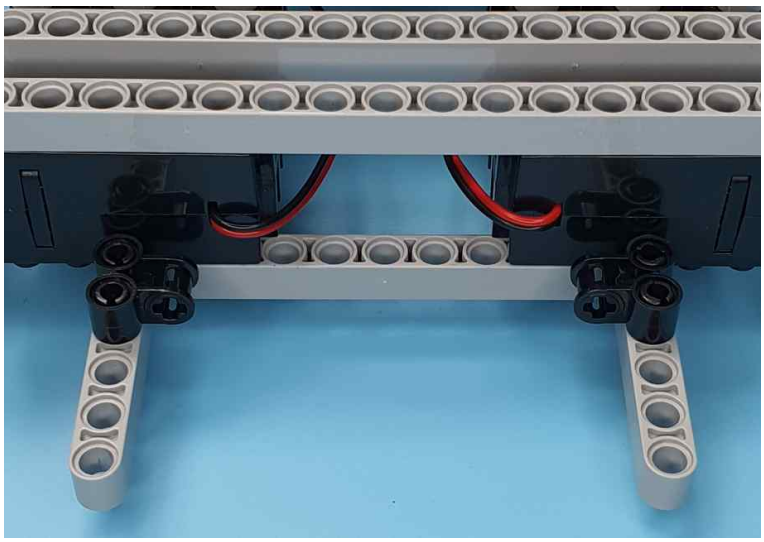


12. 다음과 같이 자동차 하단 앞 부분에 장착합니다.

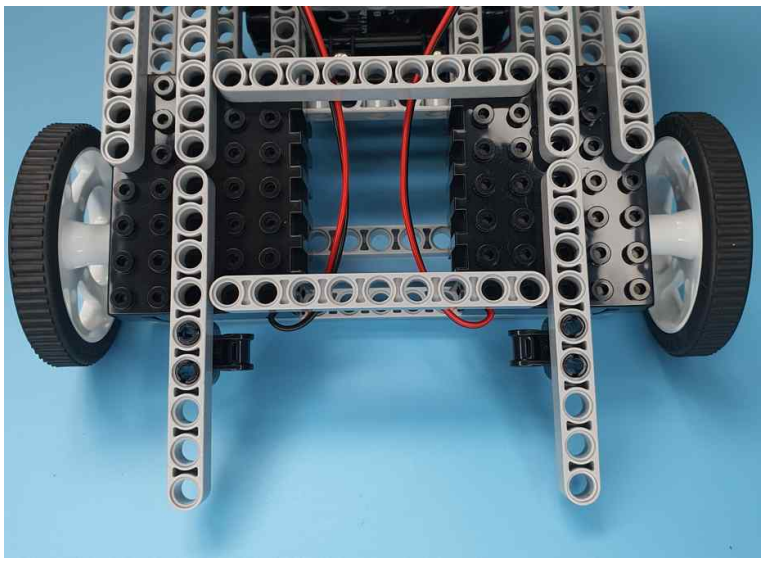


Kocolabs
www.kocolabs.co.kr

다음은 확대한 모양입니다.

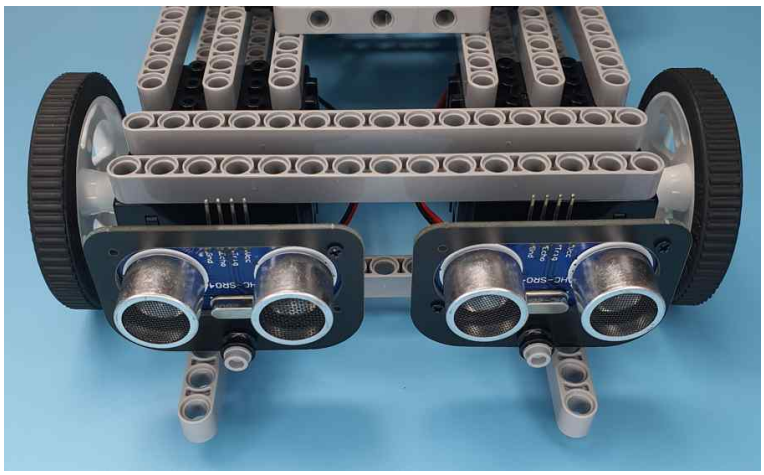


다음은 아래에서 본 모양입니다.

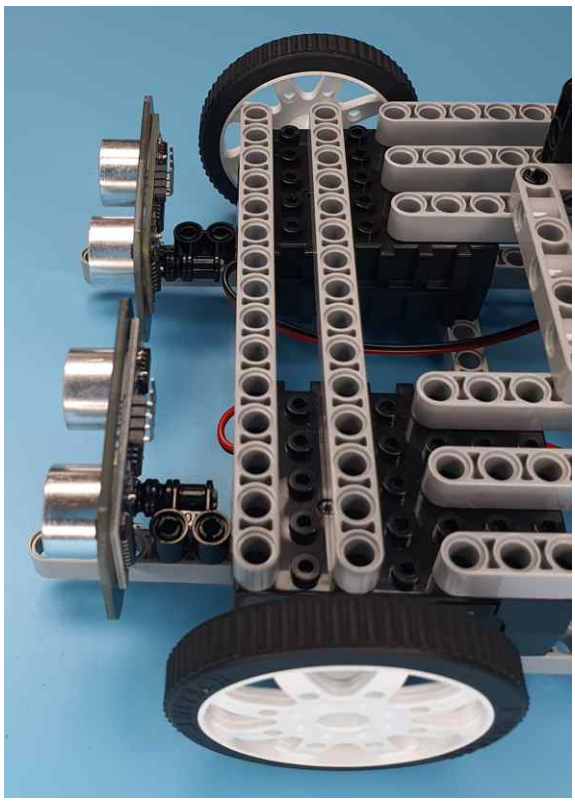


Kocolabs
www.kocolabs.co.kr

13. 다음과 같이 초음파 센서부를 장착합니다.

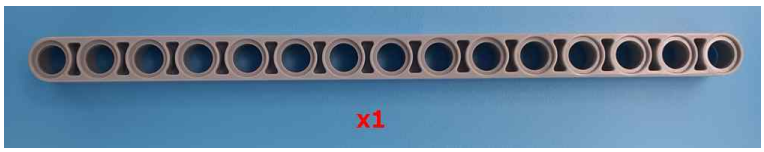


다음은 옆에소 본 모양입니다.



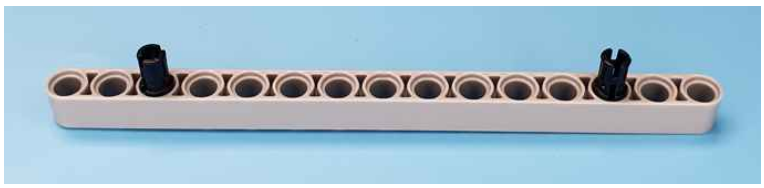
Kocolabs
www.kocolabs.co.kr

14. 다음과 같이 2가지 레고부품을 준비합니다.

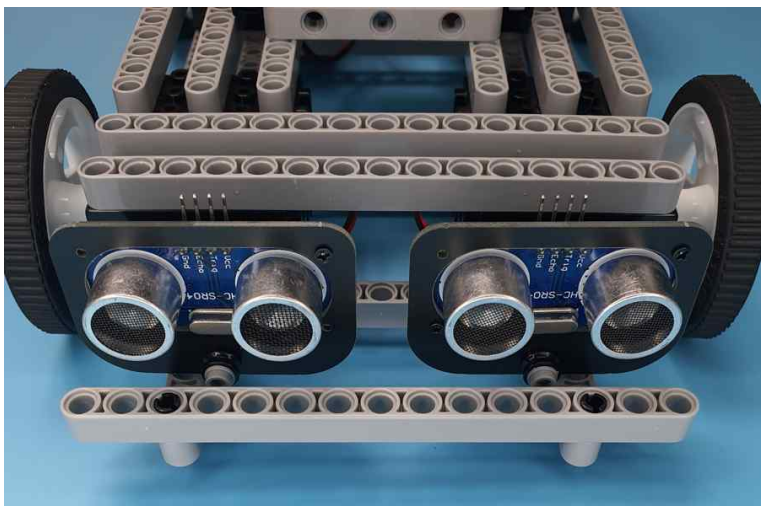




15. 다음과 같이 조립합니다.



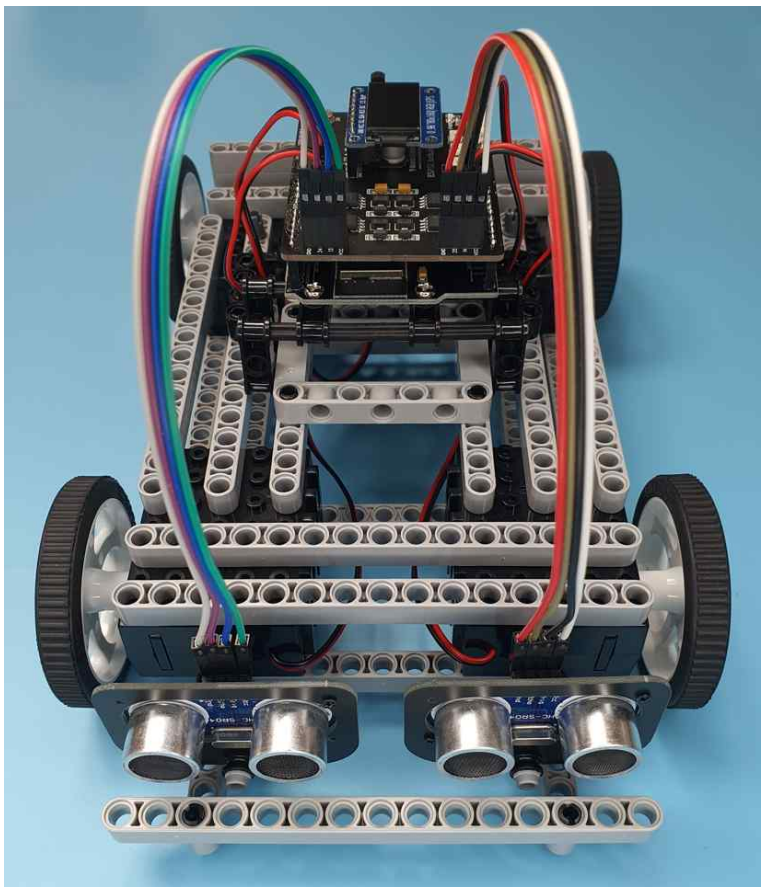
16. 다음과 같이 초음파 센서 앞 부분에 장착합니다.



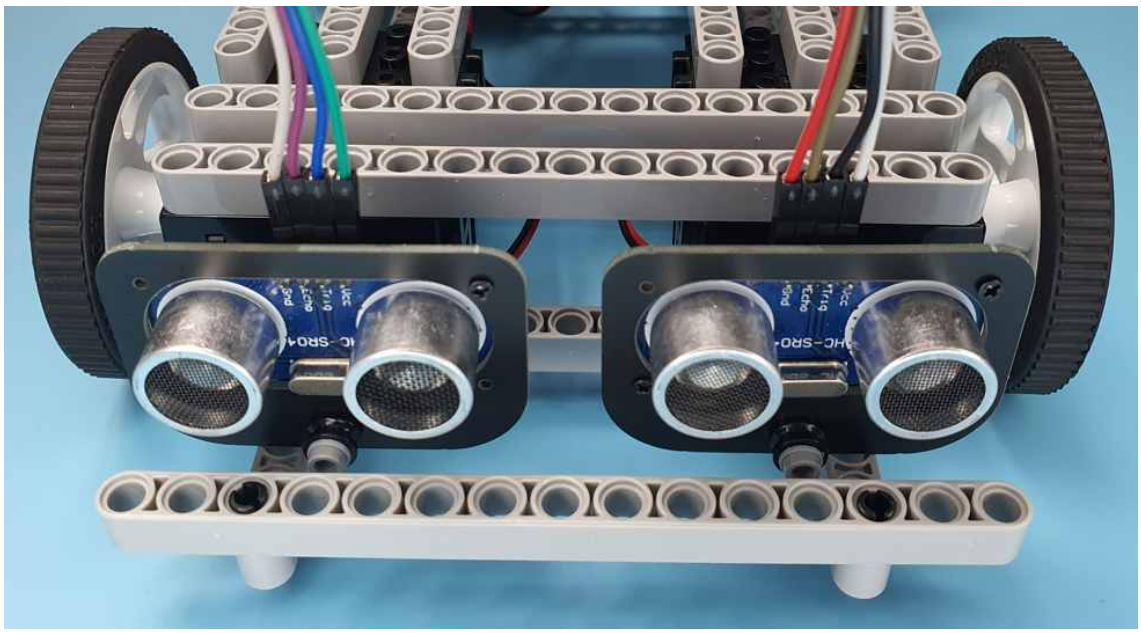
17. 다음과 같이 전선을 준비합니다.



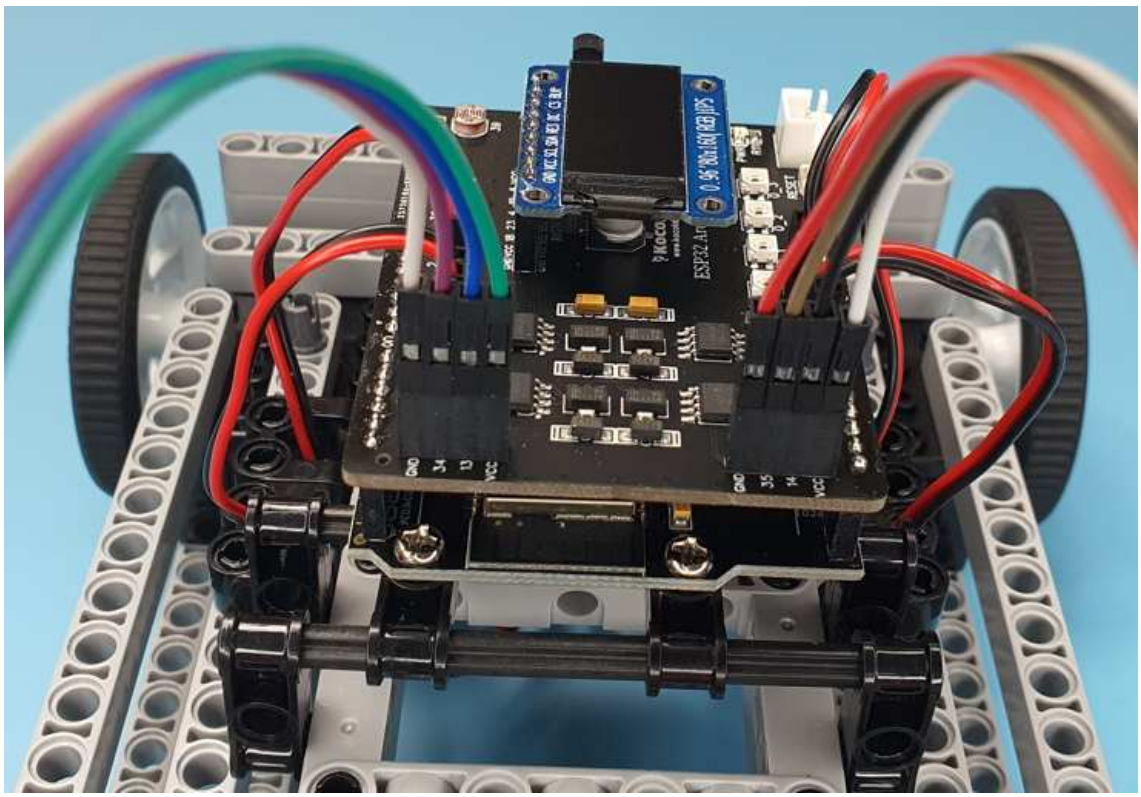
18. 다음과 같이 아두이노와 초음파 센서를 연결합니다.



다음은 초음파 센서부를 확대한 모양입니다.



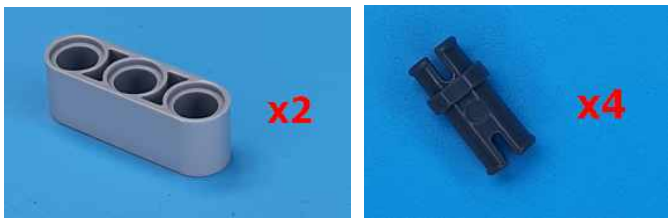
다음은 아두이노 부를 확대한 모양입니다.



*** 아두이노부의 핀 이름과 초음파 센서의 핀 이름에 맞춰 선을 연결해 줍니다.

아두이노부	초음파 센서부
VCC	Vcc
GND	Gnd
13, 14	Trig
34, 35	Echo

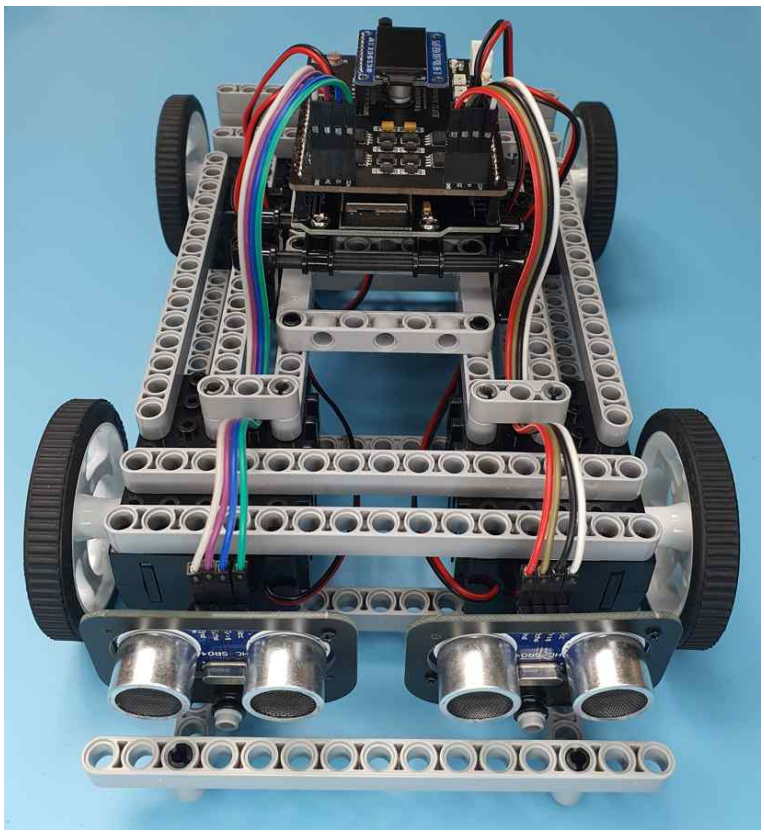
19. 다음과 같이 2가지 레고부품을 준비합니다.



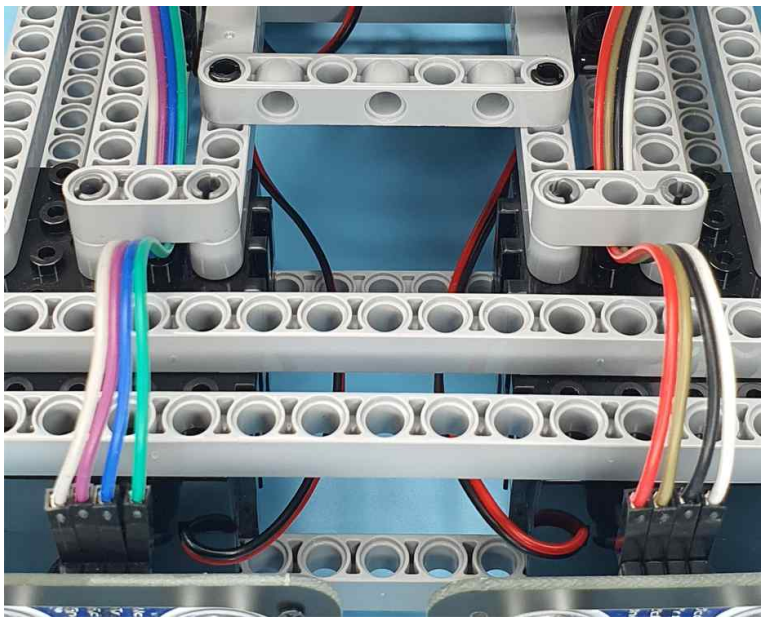
20. 다음과 같이 조립합니다.



21. 다음과 같이 선을 고정해 줍니다.

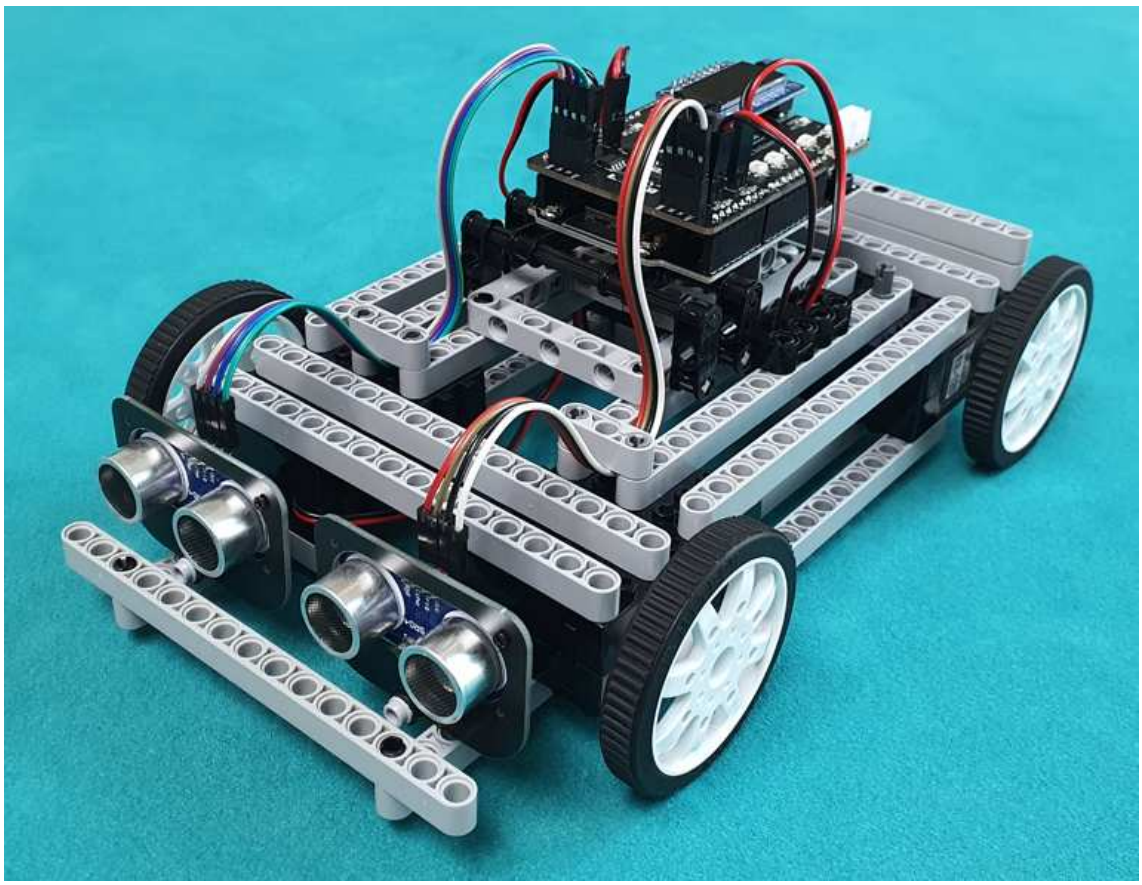


다음은 선 고정부분을 확대한 모양입니다.



Kocolabs
www.kocolabs.co.kr

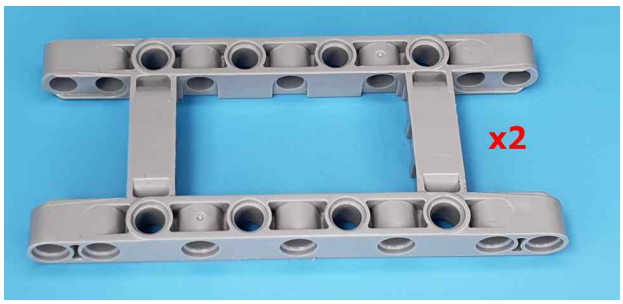
다음은 조립이 완료된 모양입니다.



차량 지지대 조립하기

다음은 차량 지지대를 조립하여 차량에 장착합니다. 차량 지지대가 있으면 DC 모터 실습하기가 편합니다.

1. 다음과 같이 2가지 레고부품을 준비합니다.





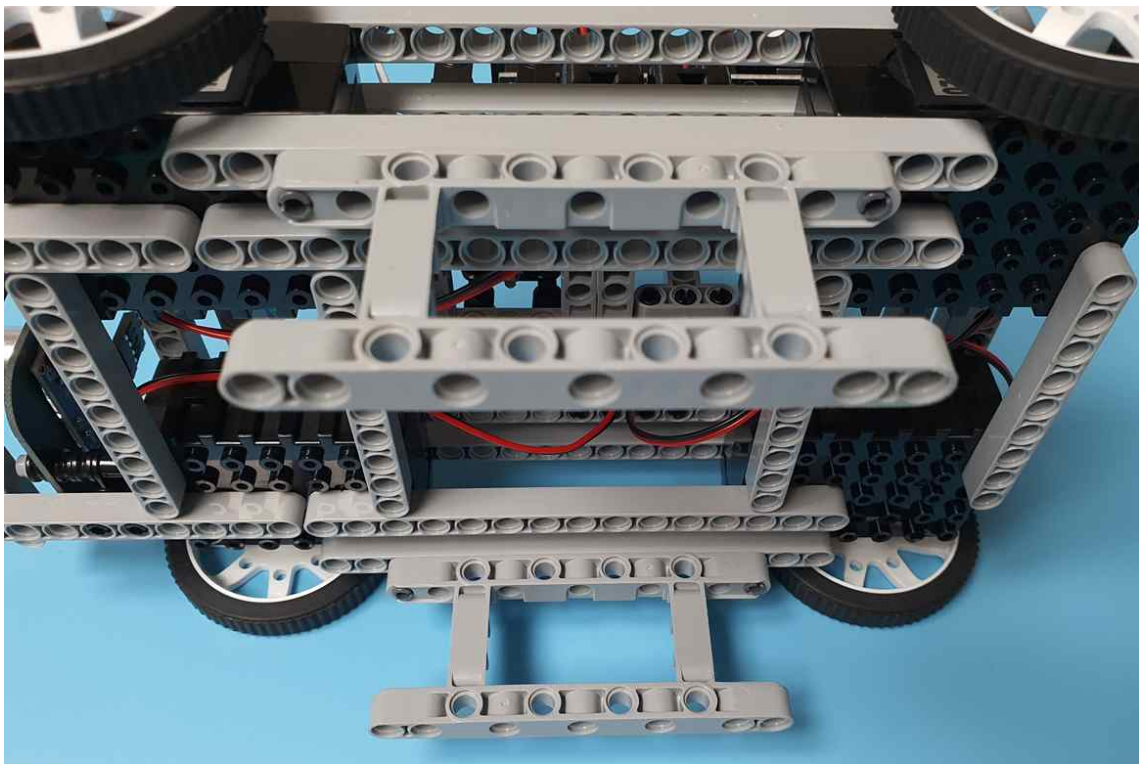
2. 다음과 같이 조립합니다.



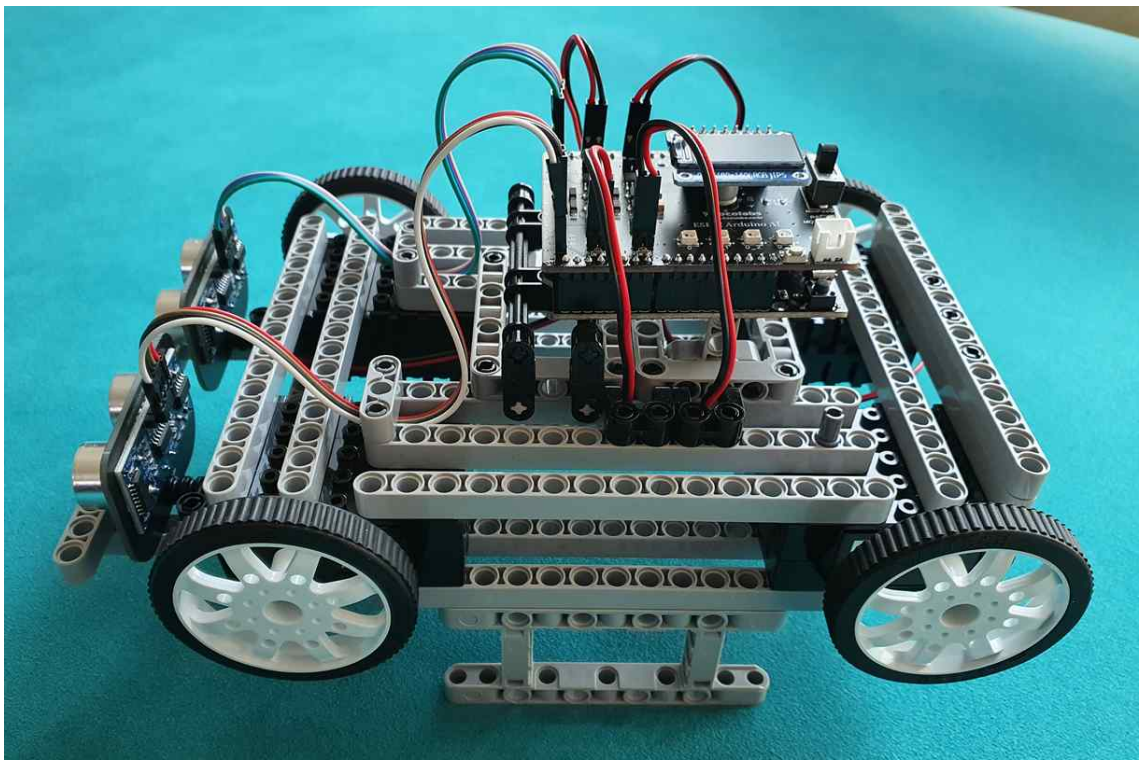
3. 다음과 같이 자동차 몸체의 하단에 끼워줍니다.



다음은 아래에서 본 모양입니다.



다음은 차량 지지대를 장착한 모양입니다.



*** 차량 지지대가 있으면 DC 모터 실습하기가 편합니다.

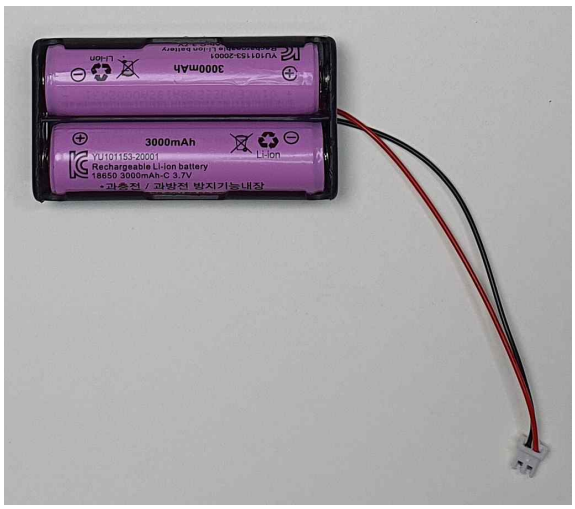
배터리 장착하기

다음은 배터리를 장착해 봅니다.

1. 다음과 같이 18650 배터리 2개, 18650 배터리 홀더를 준비합니다.

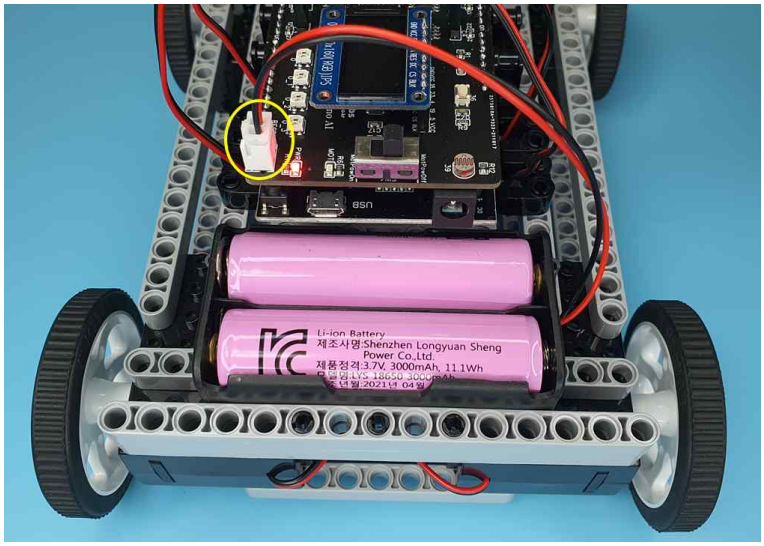


2. 다음과 같이 배터리를 배터리 홀더에 장착합니다. 양극, 음극을 잘 맞추어 장착합니다.



*** 이 책에서 사용하는 배터리는 KC인증 보호회로가 내장된 18650 배터리로 과전류, 과전압, 과열을 차단하여 화재 및 폭발의 위험으로부터 안전하게 사용할 수 있습니다.

3. 다음과 같이 자동차 뒷 부분에 끼워 넣어 고정해 준 후, 배터리 연결선을 아두이노 인공지능 자동차 쉴드에 연결합니다.

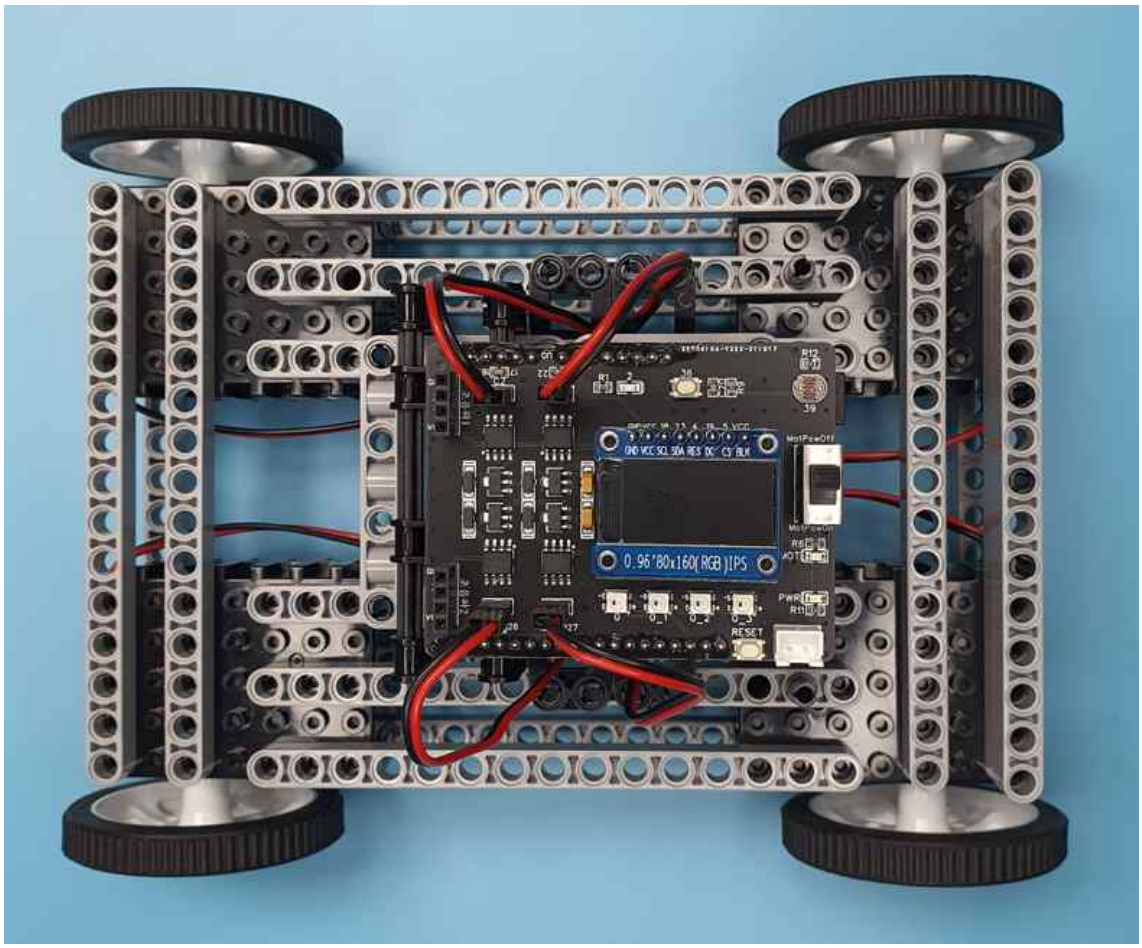


라인트레이서 센서부 조립하기

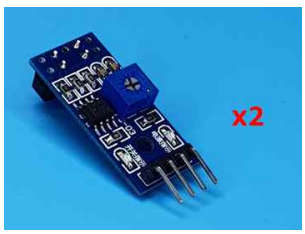
이제 라인트레이서 센서부를 조립하고 장착하는 방법을 살펴봅니다.

*** 라인트레이서 센서는 초음파 센서와 같은 위치에 장착됩니다. 따라서 초음파 센서 인공지능 자동차 실습을 마치고 라인트레이서 인공지능 자동차 실습시 이 단원을 이용하여 라인트레이서 인공지능 자동차를 조립합니다. 여기서는 일단 조립하는 방법만 소개합니다. 라인트레이서 인공지능 자동차 실습은 부록 4장에서 수행합니다.

1. 다음과 같이 초음파 센서 관련 부품을 떼어냅니다.

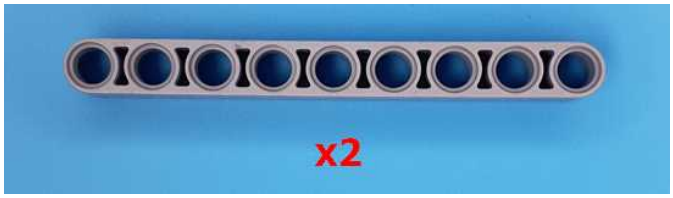


2 다음과 같이 적외선 송수신 센서, 볼트, 너트, 레고 부품을 준비합니다.

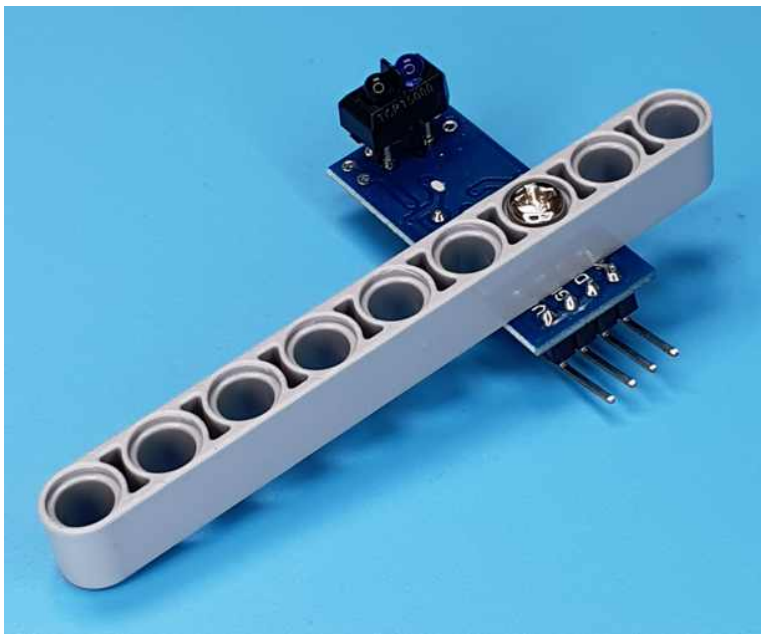




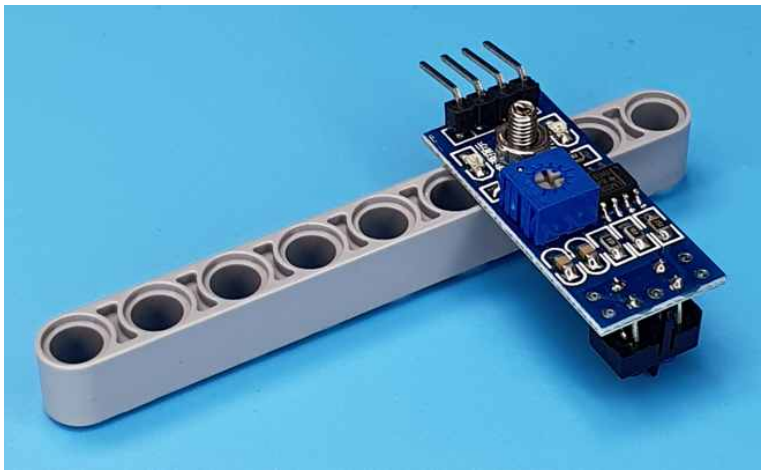




3. 다음과 같이 볼트를 레고와 적외선 센서에 끼워줍니다.

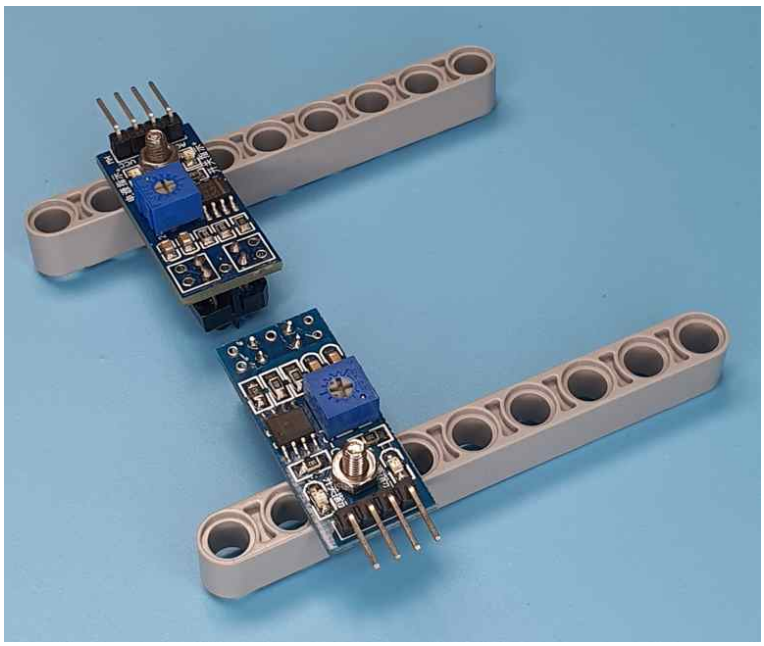


4. 다음과 같이 너트를 볼트에 장착합니다.

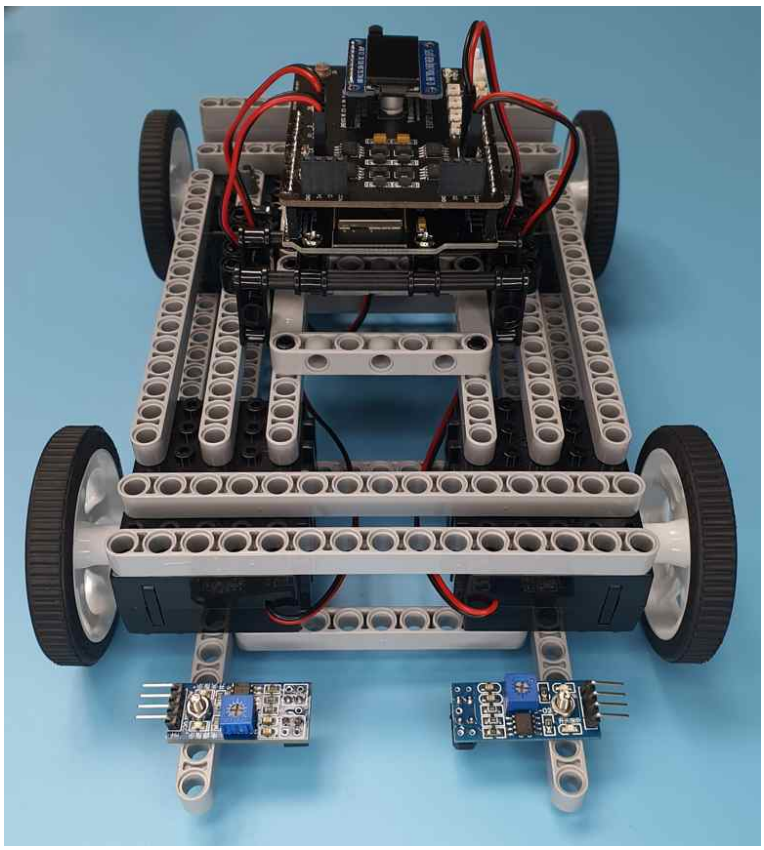


*** 스크류 드라이버를 이용하여 적당히 조여줍니다. 적외선 송수신 센서의 PCB 면이 손상되지 않도록 적당히 조여줍니다.

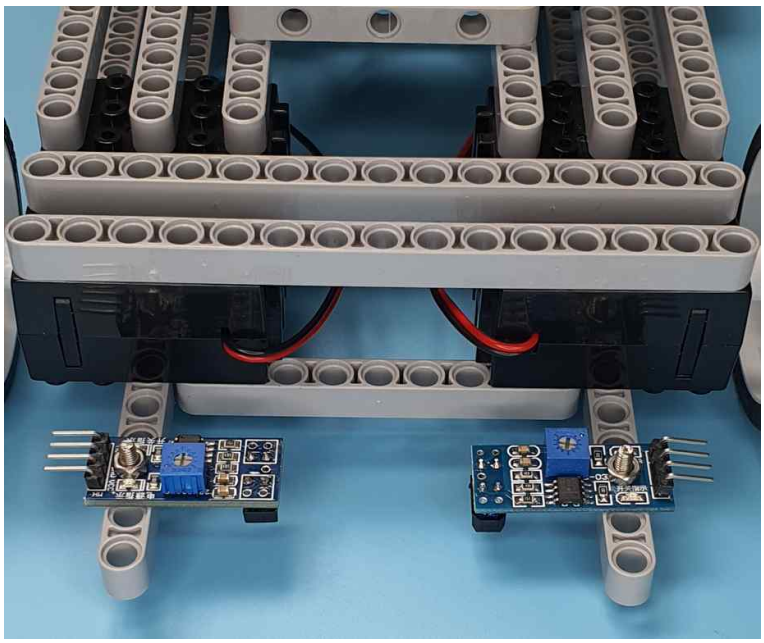
다음과 같이 2개를 조립합니다.



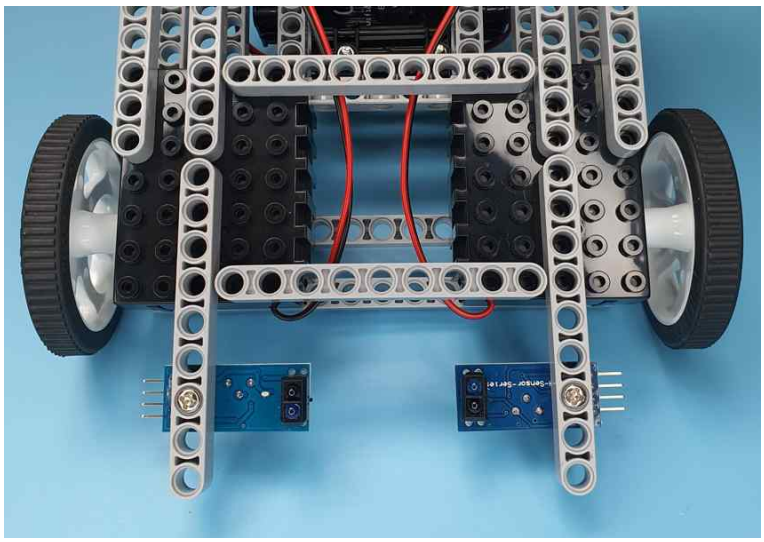
5. 다음과 같이 자동차 하단 앞 부분에 장착합니다.



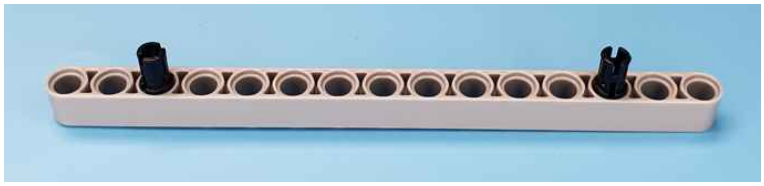
다음은 확대한 모양입니다.



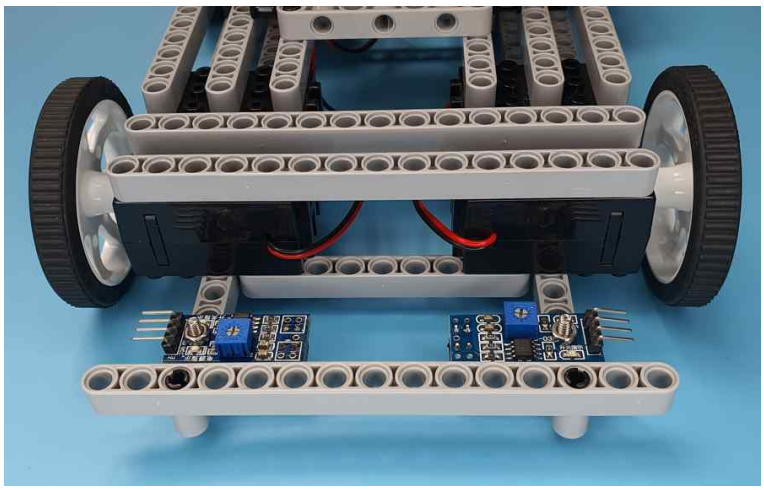
다음은 아래에서 본 모양입니다.



6. 초음파 센서부 조립에서 조립한 다음 부품을 준비합니다.



7. 다음과 같이 라인트레이서 센서 앞 부분에 장착합니다.

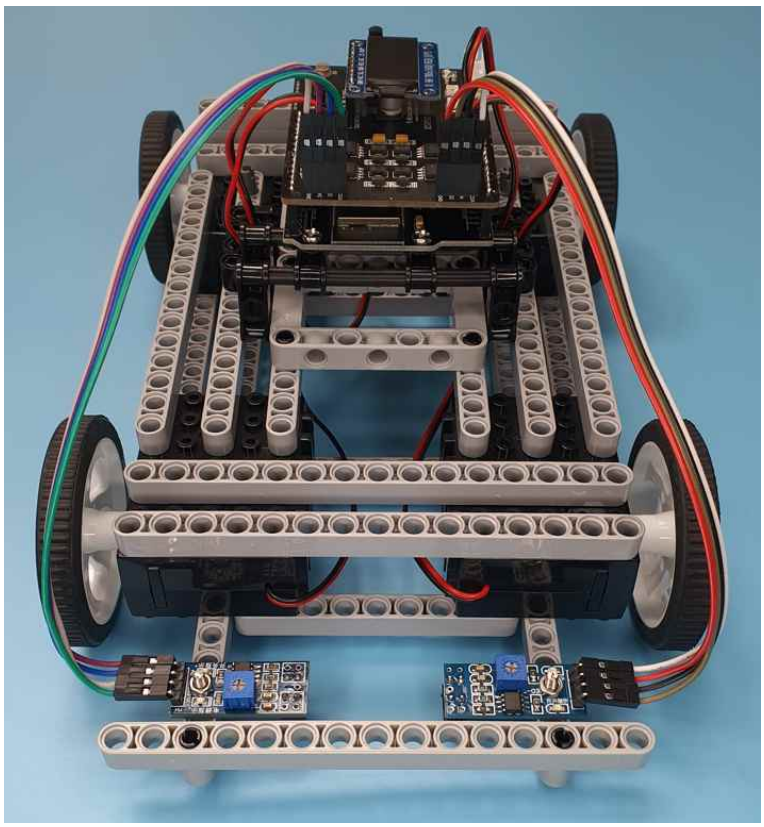


8. 다음과 같이 전선을 준비합니다.

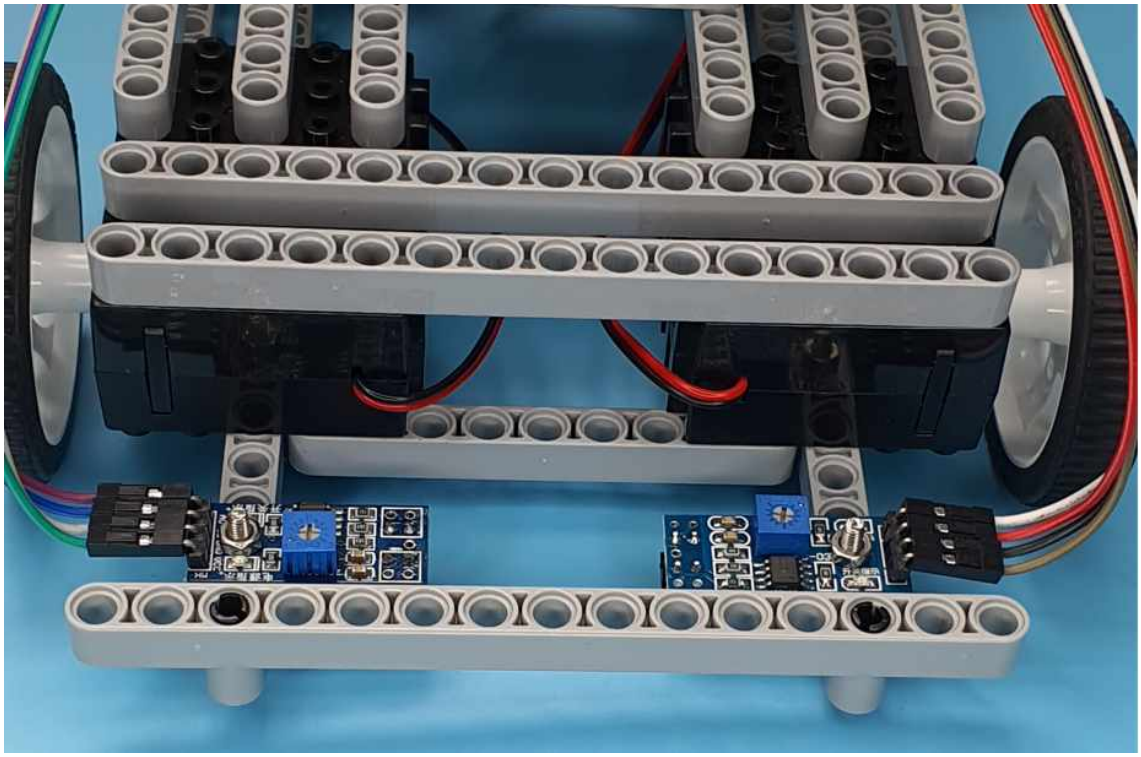


*** 초음파 센서부 조립에서 사용한 전선을 사용합니다.

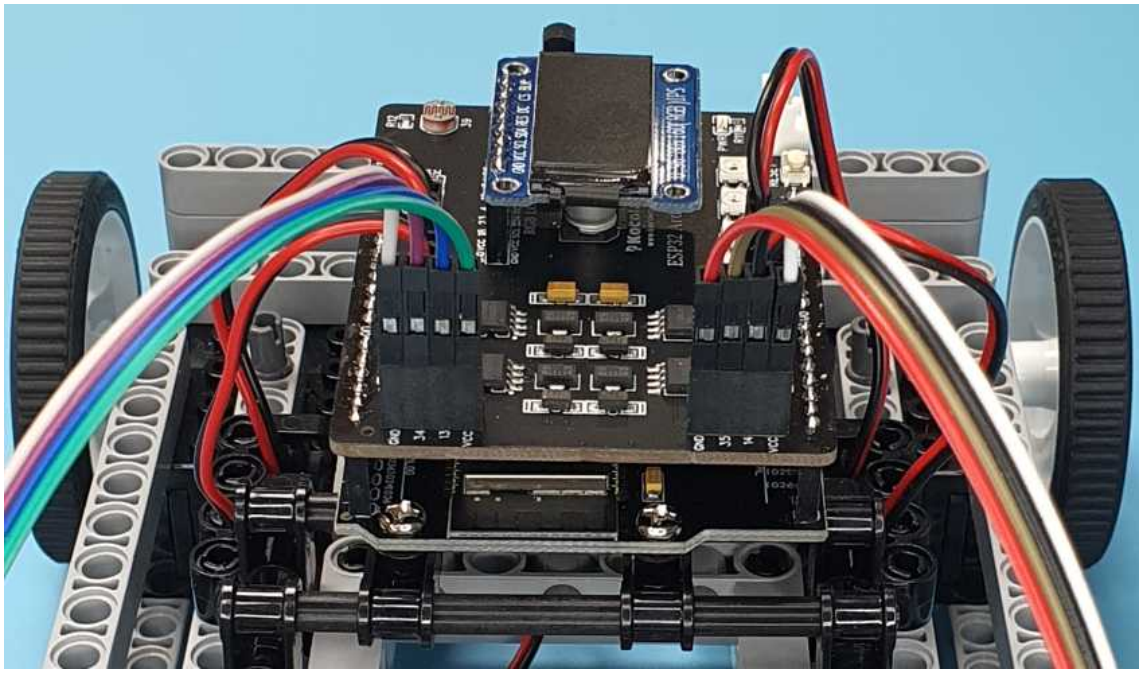
9. 다음과 같이 아두이노와 라인트레이서 센서를 연결합니다.



다음은 라인트레이서 센서부를 확대한 모양입니다.



다음은 아두이노 부를 확대한 모양입니다.



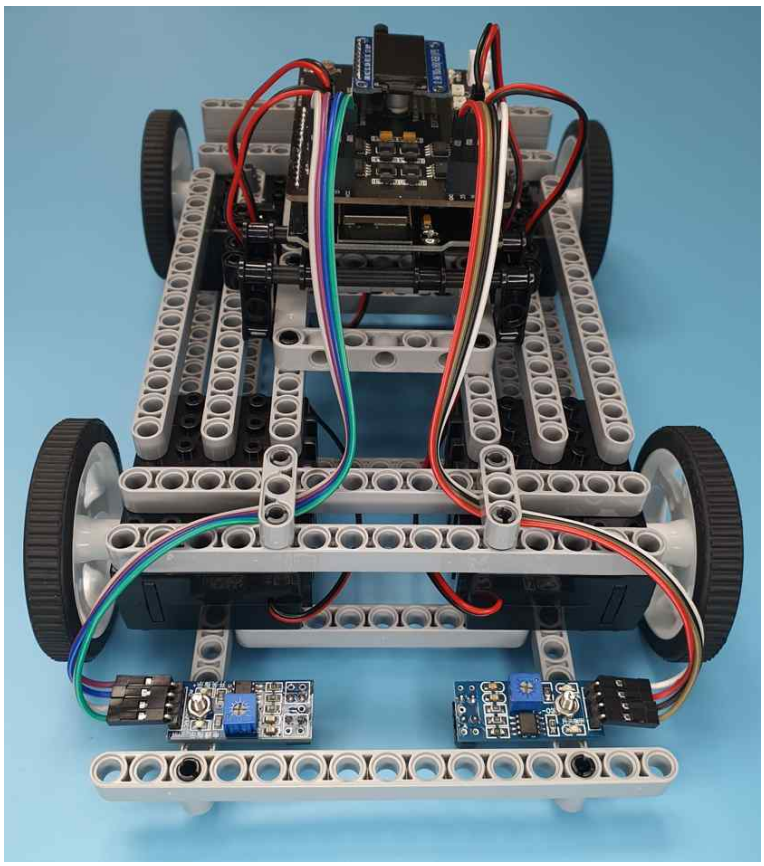
*** 아두이노부의 핀 이름과 라인트레이서 센서의 핀 이름에 맞춰 선을 연결해 줍니다.

아두이노부	라인트레이서 센서부
VCC	VCC
GND	GND
13, 14	DO
34, 35	AO

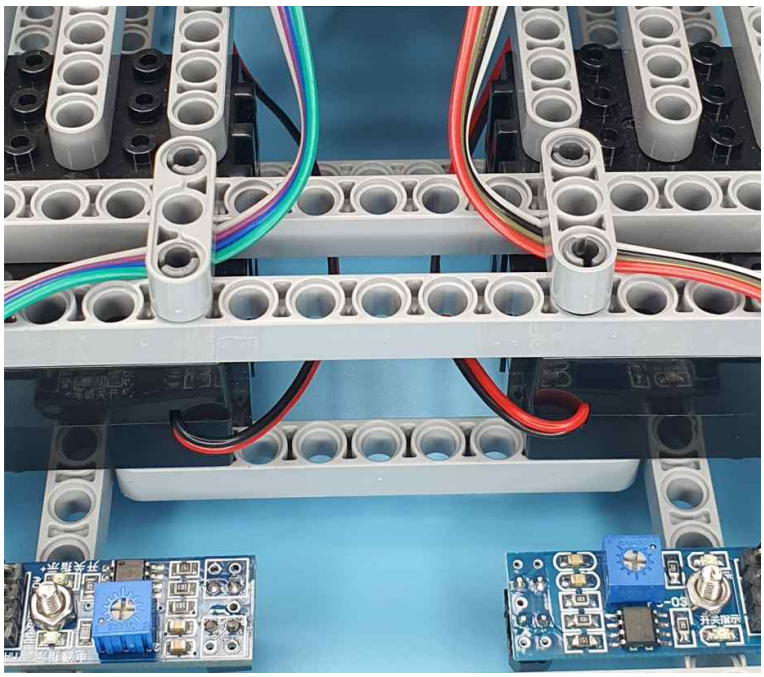
10. 초음파 센서부 조립에서 조립한 다음 부품을 준비합니다.



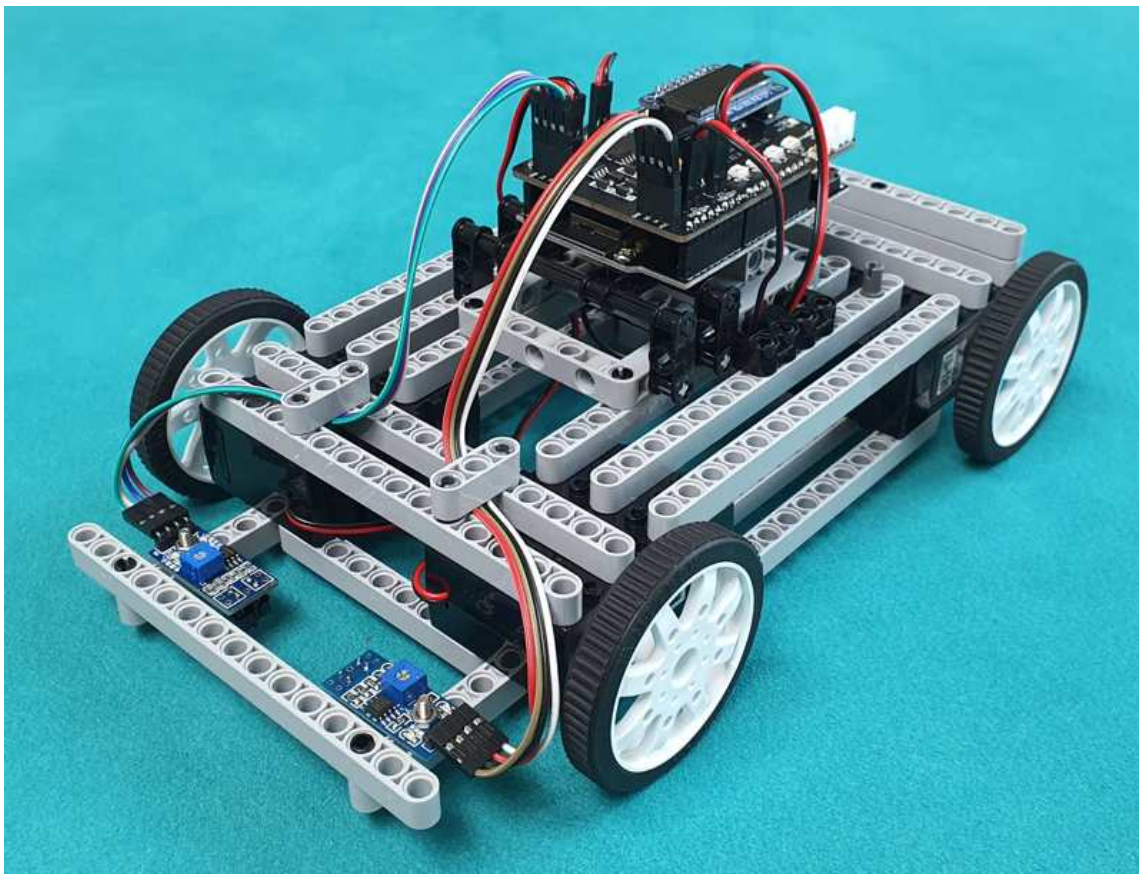
11. 다음과 같이 선을 고정해 줍니다.



다음은 선 고정부분을 확대한 모양입니다.



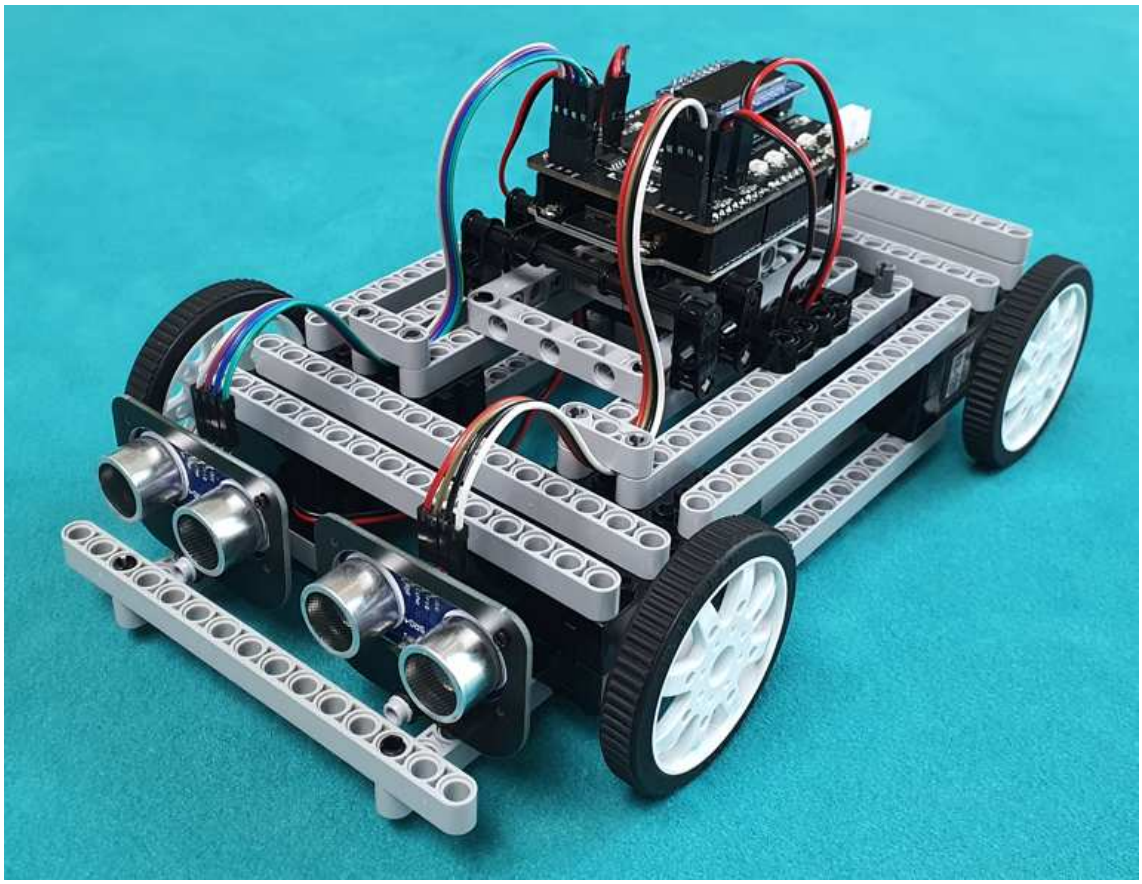
다음은 조립이 완료된 모양입니다.



이상 아두이노 인공지능 자동차 조립을 마칩니다.

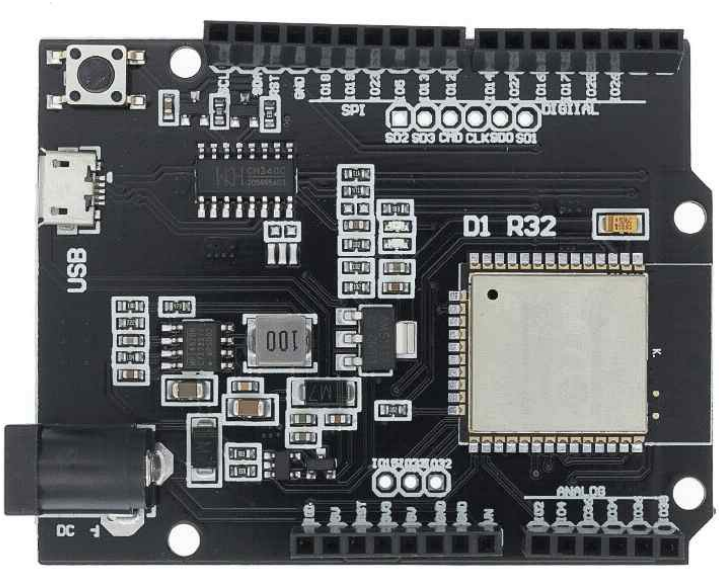
02 아두이노 인공지능 자동차 살펴보기

여기서는 아두이노 인공지능 실습을 위한 하드웨어 환경을 살펴봅니다.



01 ESP32 아두이노 살펴보기

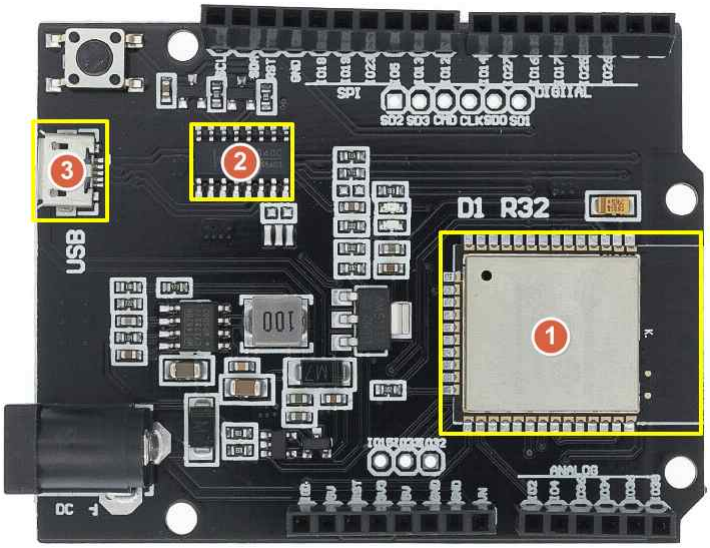
우리가 사용할 아두이노 보드는 [Wemos D1 R32]로 다음과 같습니다. [Wemos D1 R32]는 ESP32 칩을 내장한 ESP-WROOM-32 모듈 기반으로 구성된 개발용 보드입니다. [Wemos D1 R32]는 4MB 또는 16MB 크기의 플래시 메모리와 160 또는 240MHz의 클록, 50KB 정도의 사용 가능한 RAM(실제 크기는 520KB), Wifi 모듈과 Bluetooth 모듈을 내장하고 있습니다. [Wemos D1 R32]는 아두이노 소프트웨어 기반의 개발이 가능합니다.



다음은 [Wemos D1 R32]의 사양입니다.

CPU : Xtensa 듀얼 코어 32 비트 LX6, 240MHz, 600DMIPS
SRAM: 520 KB
Flash Memory: 4 MB
Wi-Fi: IEEE 802.11 b/g/n
Bluetooth : v4.2 BR/EDR, BLE
LED PWM : 16 채널
ADC : 12-bit, 18 채널
DAC : 8-bit x 2개
터치 센서 : 10개
UART : 3개
I2C : 2개
SPI : 4개
I2S : 2개
동작 전압: 3.3V
입력 전압: 7-12V

[Wemos D1 R32] 모듈은 와이파이, 블루투스 인터페이스와 시리얼 인터페이스를 가지고 있습니다. 다음 그림은 [Wemos D1 R32]의 주요 부분을 나타내고 있습니다.



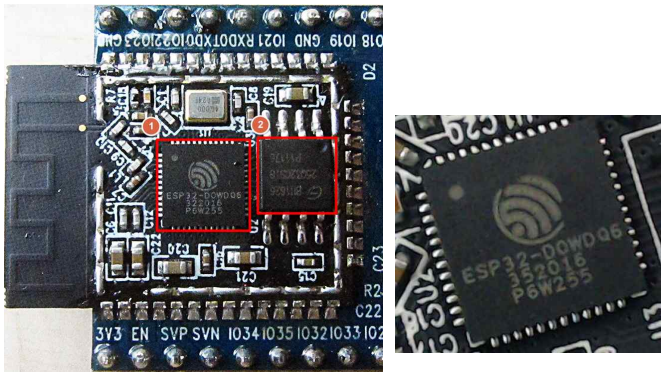
❶ 부분은 와이파이, 블루투스 기능을 포함한 ESP-WROOM-32 모듈을 붙인 부분입니다.



❷ 부분은 시리얼 영역으로 CH340 칩입니다. CH340 칩은 USB-to-UART 브리지 모듈로 ❸ 부분의 USB 단자와 ❶ 부분에 포함된 UART를 연결해주는 역할을 합니다.

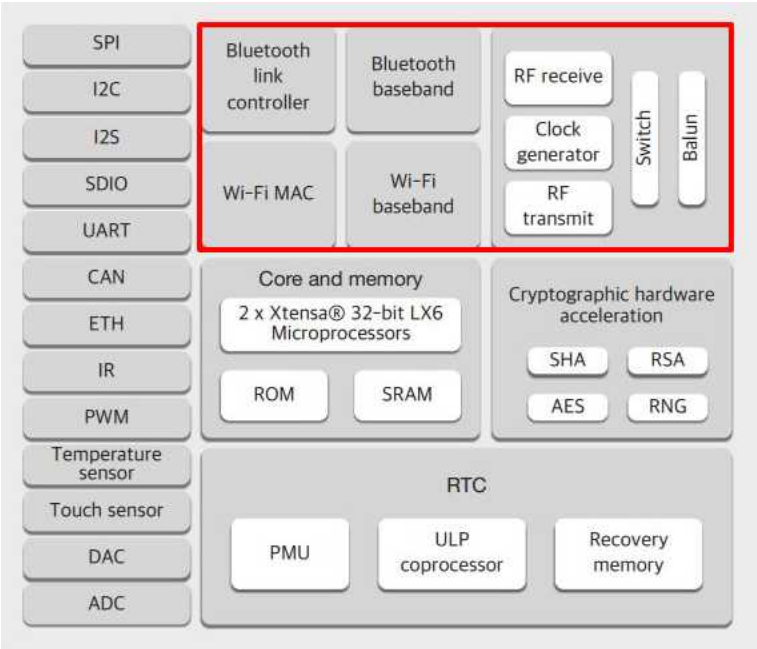
ESP-WROOM-32 모듈 살펴보기

ESP-WROOM-32 모듈의 금속 막을 벗기면 내부 모양은 다음과 같습니다. ❶ 부분은 ESP32 칩이고, ❷ 부분은 SPI 통신 방식을 사용하는 플래시 메모리입니다.



❶ ESP32 칩 내부에는 MCU, RAM, Wifi, Bluetooth, IO 등이 내장되어 있습니다. 그러나 프로그래밍 가능한 ROM은 없습니다. 그래서 사용자 프로그램을 저장하기 위해 ESP32 칩 외부에 SPI 플래시 메모리를 장착해야 합니다. ❷ 부분은 4MB 크기의 SPI 플래시 메모리입니다.

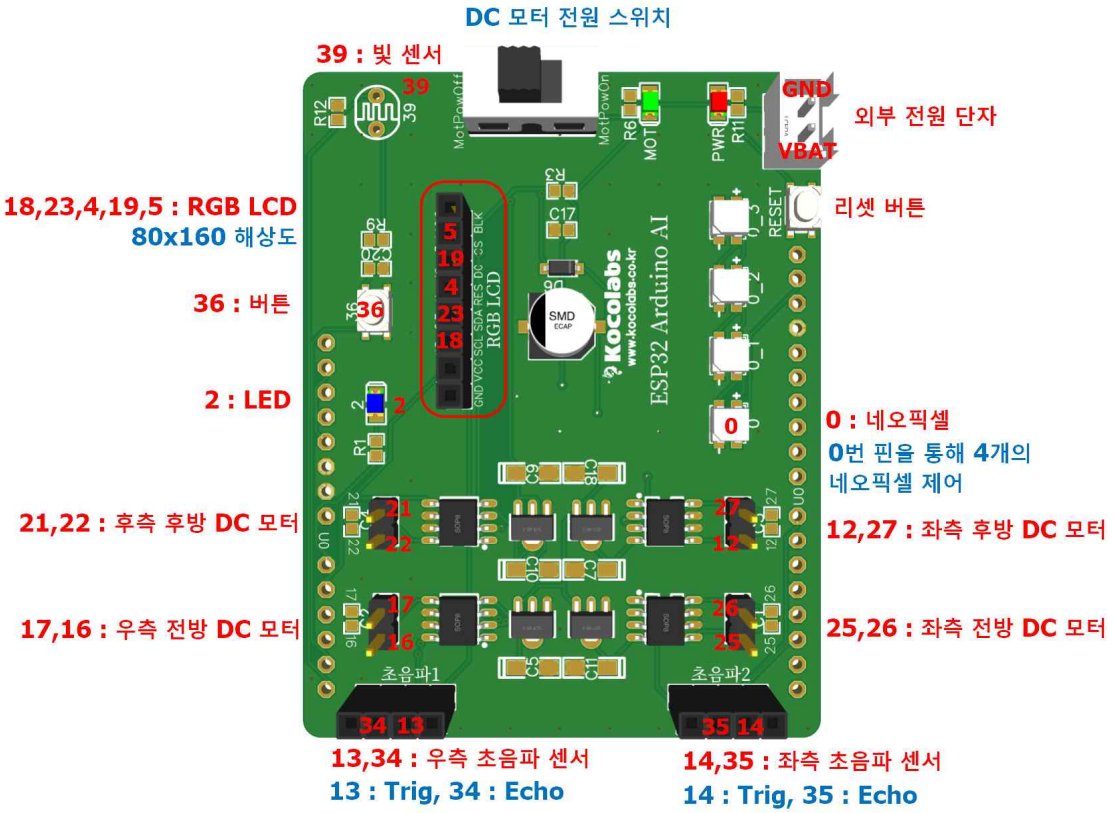
다음은 ESP32 칩의 내부 구조를 나타냅니다.



빨간색 사각 박스 부분은 Wifi, Bluetooth 영역이고 나머지 부분은 MCU 영역입니다. MCU 영역은 Xtensa 32 비트 LX6 마이크로 프로세서 2개로 구성됩니다.

02 아두이노 인공지능 쉴드 핀 살펴보기

다음은 아두이노 인공지능 쉴드입니다. 아두이노는 이 쉴드를 통해서 아두이노 RC 카의 DC 모터 제어, LED 제어, 네오픽셀 제어, 그래픽 LCD 제어, 초음파 센서 제어와 빛 센서 입력을 받게 됩니다. 아두이노 쉴드에는 각 부품이 연결된 아두이노 핀을 표시하고 있습니다. 예를 들어 네오픽셀은 아두이노 0번 핀에 연결되어 있고, 빛 센서는 39 핀에 연결되어 있습니다.



03 아두이노 개발 환경 구성하기

이제 아두이노 스케치를 구현하고, 컴파일하고, 업로드하기 위한 개발 환경을 구성하도록 합니다. 먼저 아두이노 개발을 위한 아두이노 소프트웨어를 설치하고, 다음은 ESP32 패키지를 설치합니다.

01 아두이노 소프트웨어 설치하기

먼저 아두이노 개발을 위한 아두이노 소프트웨어를 설치합니다.

다음과 같은 아두이노 소프트웨어를 설치해 봅니다.



우리는 이 프로그램을 이용하여

❶ 아두이노 스케치를 작성하고,
❷ 작성한 스케치를 컴파일하고,
❸ 컴파일한 스케치를 아두이노 보드상에 업로드하고,
❹ 시리얼 모니터를 통해 결과를 확인하게 됩니다.

1. [www.arduino.cc] 사이트에 접속합니다.



2. 홈페이지가 열립니다. [SOFTWARE] 메뉴를 선택합니다.



3. 새로 열린 페이지에서 아래로 조금 이동하여 다음 부분을 찾습니다.



*** 2021년 10월 기준 ARDUINO 1.8.16이 사용되며, 버전은 다운로드 시점에 따라 변경될 수 있습니다. 독자 여러분은 최신 버전을 설치하도록 합니다.

4. [Windows Win 7 and newer]를 마우스 클릭합니다.



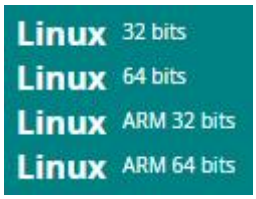
5. 그러면 다음 페이지로 연결됩니다. 하단에 있는 [JUST DOWNLOAD] 부분을 누릅니다.



*** 맥 OS 사용자의 경우엔 다음을 선택합니다.



*** 리눅스 OS 사용자의 경우엔 다음 중 하나를 선택합니다.

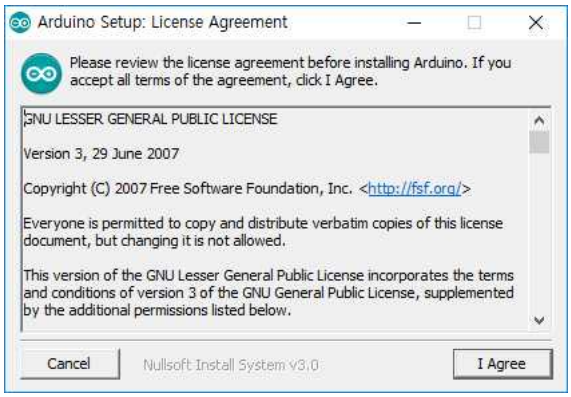


[Linux ARM 32 bits]나 [Linux ARM 64 bits]의 경우엔 라즈베리파이와 같이 ARM 기반 SOC에서 동작하는 리눅스에서 사용합니다.

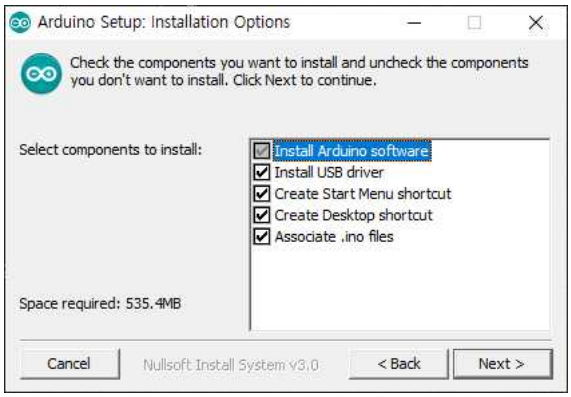
6. 다운로드가 완료되면 마우스 클릭하여 설치 프로그램을 실행시킵니다.



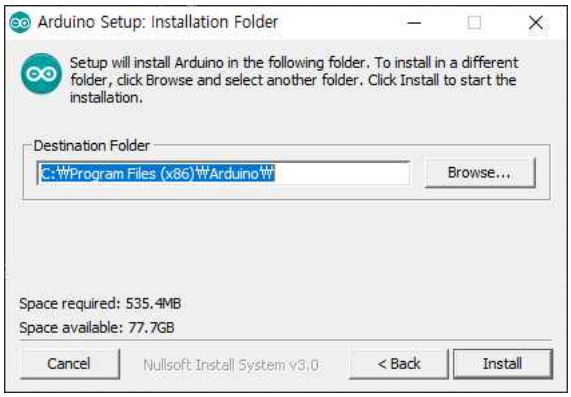
7. 다음과 같이 [Arduino Setup: License Agreement] 창이 뜹니다. 사용 조건 동의에 대한 내용입니다. [I Agree] 버튼을 눌러 동의합니다.



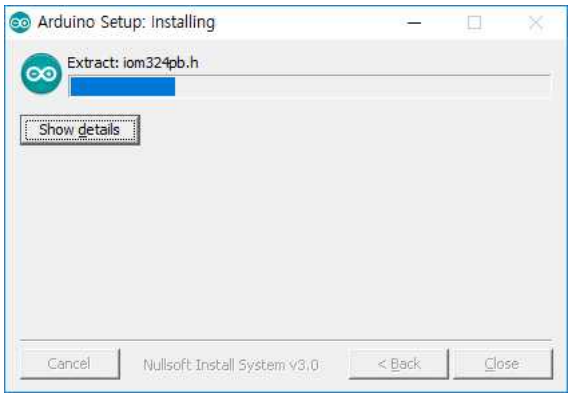
8. 다음과 같이 [Arduino Setup: Installation Options] 창이 뜹니다. 설치 선택에 대한 내용입니다. 기본 상태로 둔 채 [Next] 버튼을 누릅니다.



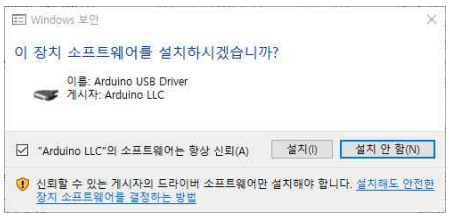
9. 다음과 같이 [Arduino Setup: Installation Folder] 창이 뜹니다. 설치 폴더 선택 창입니다. 기본 상태로 둔 채 [Install] 버튼을 누릅니다.



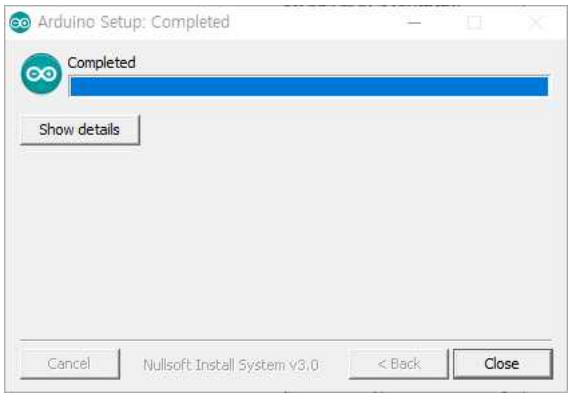
10. 그러면 다음과 같이 설치가 진행됩니다.



11. 설치 마지막 단계에 다음과 같은 창이 하나 이상 뜹니다. 아두이노 보드에 접근하기 위해 필요한 드라이버 설치 창입니다. [설치(I)] 버튼을 눌러줍니다.



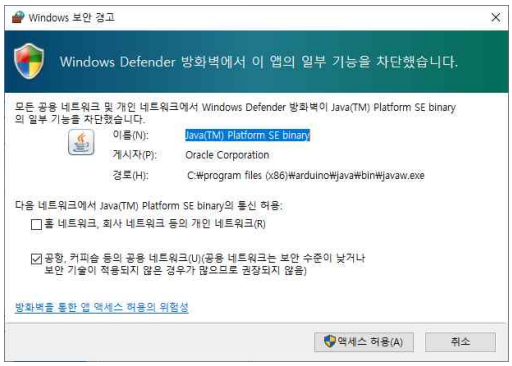
12. 다음과 같이 [Arduino Setup: Completed] 창이 뜹니다. 설치 완료 창입니다. [Close] 버튼을 눌러 설치를 마칩니다.



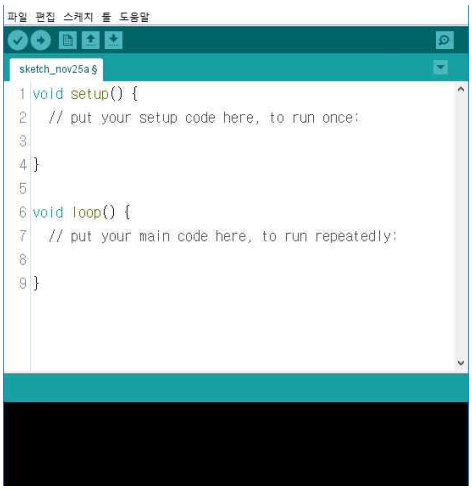
13. 바탕 화면에 다음 아이콘이 설치됩니다. 아이콘을 눌러 아두이노 소프트웨어를 실행시킵니다.



14. 처음엔 다음과 같은 보안 경고 창이 뜹니다. 아두이노 소프트웨어를 사용하기 위해 필요한 부분이기 때문에 [액세스 허용(A)] 버튼을 누릅니다.



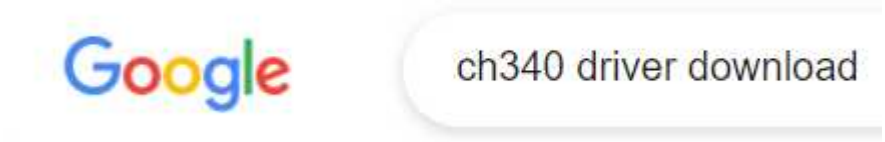
15. 그러면 다음과 같이 아두이노 소프트웨어 프로그램이 실행되는 것을 볼 수 있습니다.



02 USB 드라이버 설치하기

우리가 사용할 아두이노 보드 [Wemos D1 R32]를 USB에 연결하기 위해서는 CH340 드라이버를 설치해야 합니다.

1. 다음과 같이 [ch340 driver download]를 검색합니다.



2. 다음 사이트로 들어갑니다.



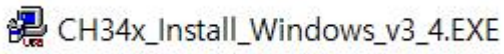
3. 다음 부분을 찾아 마우스 클릭합니다.



4. 다음과 같이 드라이버 프로그램을 다운로드 받습니다.



5. 압축을 푼 후, 다음 파일을 이용하여 드라이버를 설치합니다.



03 ESP32 아두이노 패키지 설치하기

여기서는 ESP32 아두이노 패키지를 설치하고, 아두이노 보드를 컴퓨터에 연결하는 방법, 아두이노 보드와 시리얼 포트를 선택하는 방법, 스케치를 작성하는 방법, 아두이노 오류 발생 시 대처 방법을 살펴봅니다.

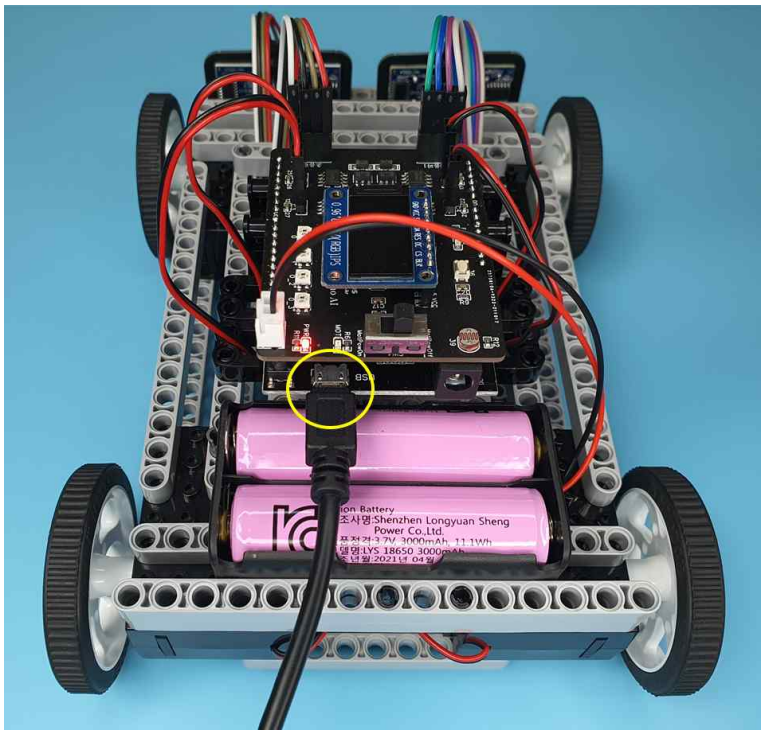
ESP32 보드 컴퓨터에 연결하기

이제 ESP32 보드와 컴퓨터를 연결해 봅니다. 아두이노 보드의 USB는 다음과 같이 세 가지 기능을 제공합니다.

1) 전원을 공급 받을 수 있고,
2) 시리얼 포트를 통해 컴파일한 프로그램을 업로드할 수 있고,
3) 시리얼 포트를 통해 디버깅 메시지를 볼 수 있습니다.

그래서 아두이노 보드는 USB 케이블 하나로 컴퓨터로 연결될 수 있으며, 간단한 인터페이스를 이용하여, 개발을 진행할 수 있습니다.

1. USB 케이블의 한쪽 끝(micro B 형)을 아두이노 보드에 연결합니다.



2. USB 케이블의 다른 쪽 끝을 컴퓨터에 연결합니다.



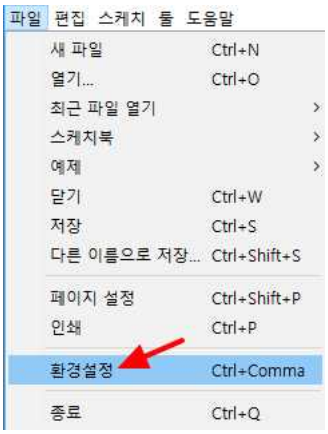
ESP32 패키지 설치하기

아두이노 소프트웨어에서 ESP32 아두이노 프로그래밍을 하기 위해서는 ESP32 아두이노 패키지를 설치해야 합니다. 다음과 같은 순서로 패키지를 설치합니다.

1. 아두이노 소프트웨어를 실행합니다.

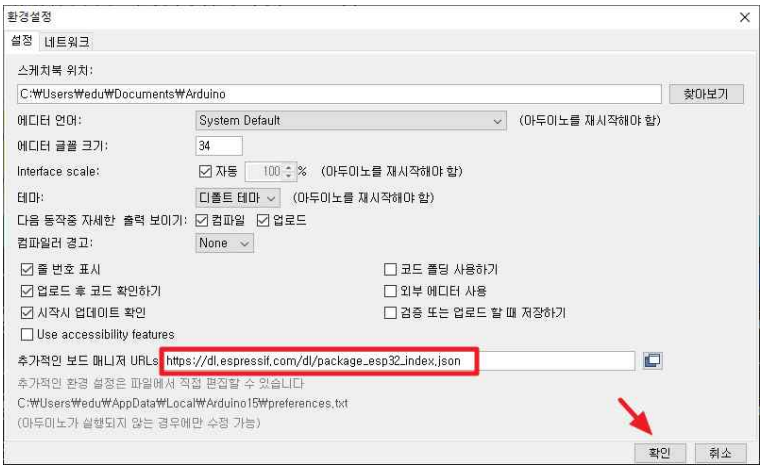


2. 아두이노 소프트웨어에서 [파일]-[환경설정] 메뉴를 선택합니다.



3. 환경설정 창의 [추가적인 보드 매니저 URLs] 입력 박스에 다음과 같이 입력한 후, [확인] 버튼을 눌러줍니다. 아래에서 dl은 디엘입니다.

https://dl.espressif.com/dl/package_esp32_index.json



*** AVR 계통의 아두이노가 아닌 추가적인 보드를 아두이노 소프트웨어에 추가할 때 사용하

는 방법입니다. 여기서는 아두이노 소프트웨어에 ESP32 아두이노 보드 개발 환경을 추가하기 위해 해당 사이트를 환경설정에서 추가하였습니다.

4. [툴]--[보드]--[보드 매니저...] 메뉴를 선택합니다.



5. [보드 매니저] 창이 뜨면 다음과 같이 esp32을 검색한 후, esp32 패키지를 선택한 후 [설치] 버튼을 누릅니다. 설치가 완료되면 [닫기] 버튼을 누릅니다.



6. [툴] 메뉴를 이용하여 보드, 포트를 다음과 같이 선택합니다.



*** 독자 여러분의 포트 번호는 다를 수 있으며, 해당 포트 번호를 사용하면 됩니다.
*** 우리는 포트를 통해서 아두이노 스케치 프로그램을 업로드하고, 시리얼 모니터를 통한 디버깅 메시지를 확인합니다.

04 아두이노 스케치 작성해 보기

여기서는

1) Hello PC 스케치를 작성한 후,
2) 컴파일하고,
3) 아두이노 보드에 업로드하고,
4) 시리얼 모니터를 통해 결과를 확인해 봅니다.

스케치 작성하기

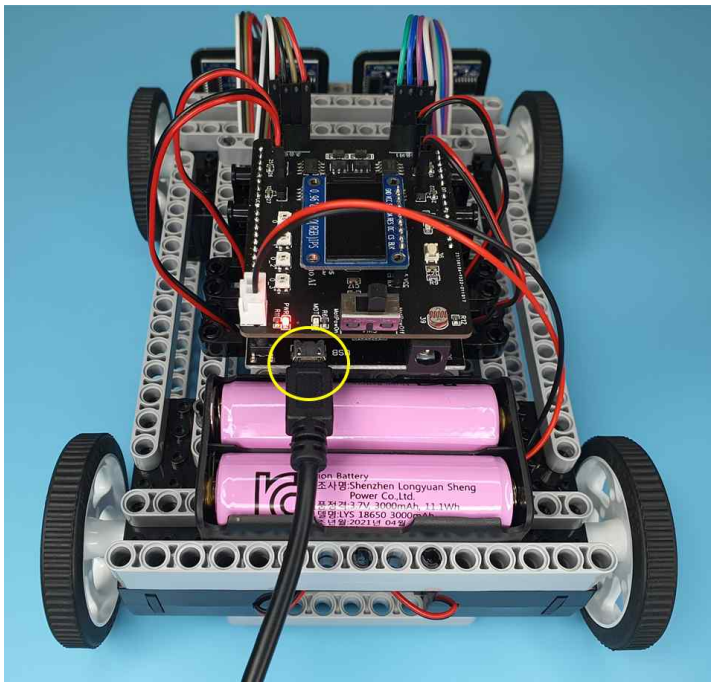
1. 다음과 같이 예제를 작성합니다.

sketch_jan31a §

```
1 void setup() {
2   Serial.begin(115200);
3 }
4
5 void loop() {
6   Serial.println("Hello PC^^. I'm an ESP32~");
7 }
```

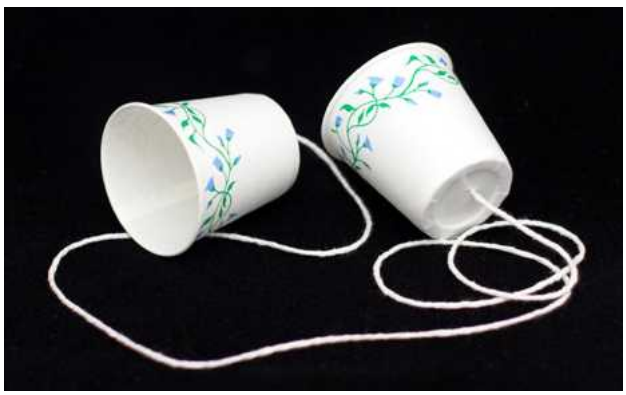
2 : 아두이노가 Serial.begin 명령을 수행하여 PC로 연결된 Serial의 통신 속도를 115200bps로 설정하게 합니다. 115200bps는 초당 115200 비트를 보내는 속도입니다. 시리얼 포트를 통해 문자 하나를 보내는데 10비트가 필요합니다. 그러므로 1초에 115200/10 = 11520 문자를 보내는 속도입니다. 11520 문자는 A4 용지 기준 5~6페이지 정도의 양입니다. 비트는 0 또는 1을 담을 수 있는 데이터 저장의 가장 작은 단위입니다.
6 : 아두이노가 Serial.println 명령을 수행하여 `"Hello PC^^. I'm an ESP32~"` 문자열을 PC로 출력하게 합니다. println은 print line의 약자입니다. ln의 l은 영문 대문자 아이(I)가 아니고 소문자 엘(l)입니다.

시리얼 통신은 다음 부분을 통해서 이루어집니다.





시리얼 통신의 원리는 종이컵과 실을 이용하여 말하고 들을 수 있는 원리와 같습니다. 우리가 하는 말이 실을 통해 순차적으로 전달되는 원리로 아두이노 보드와 컴퓨터도 통신을 하게 됩니다.

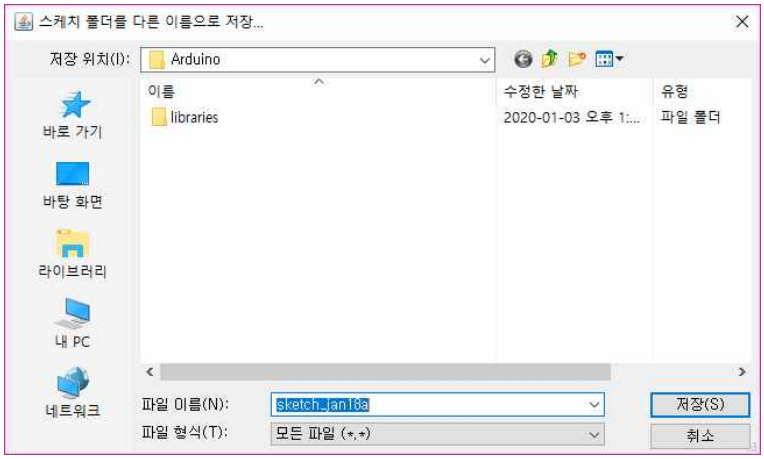


스케치 저장하기

2. 다섯 번째 아이콘인 [저장] 버튼을 누릅니다.



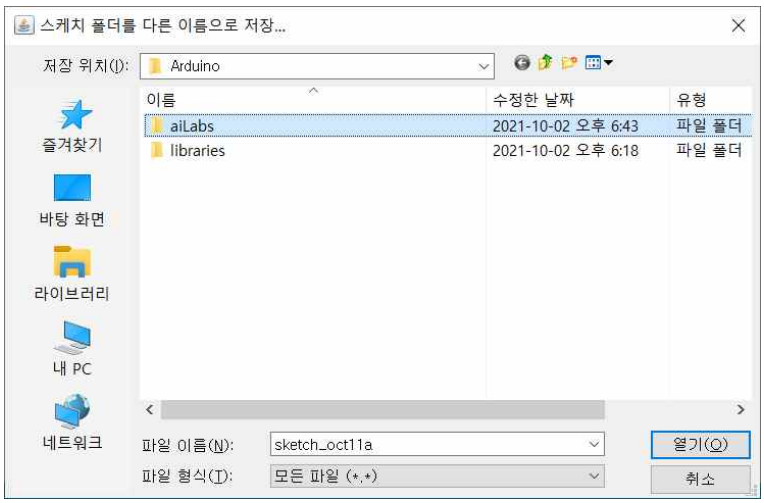
3. 그러면 다음과 같은 창이 뜹니다.



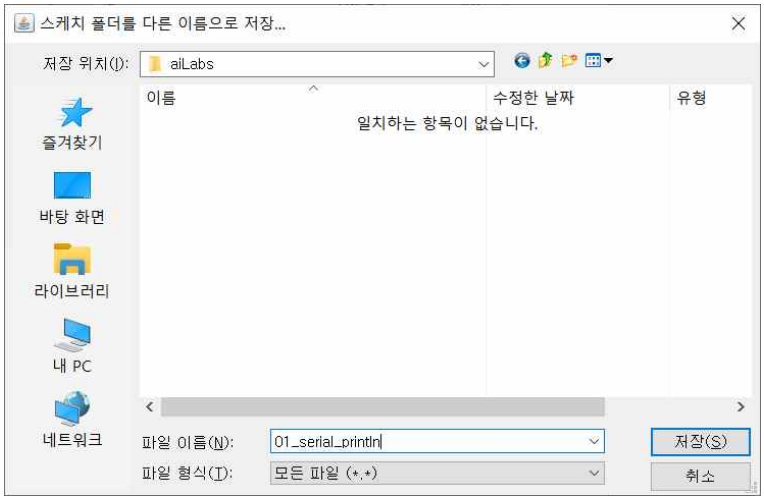
4. 프로젝트 디렉터리를 만들기 위해 오른쪽 상단에 있는 [새 폴더 만들기] 버튼을 누릅니다.



5. 디렉터리 이름을 [aiLabs]로 합니다. 한글 이름은 오류가 발생할 수 있으므로 사용하지 않습니다.



6. [aiLabs] 디렉터리로 이동하여 [01_serial_println]을 입력한 후, [저장] 버튼을 누릅니다.

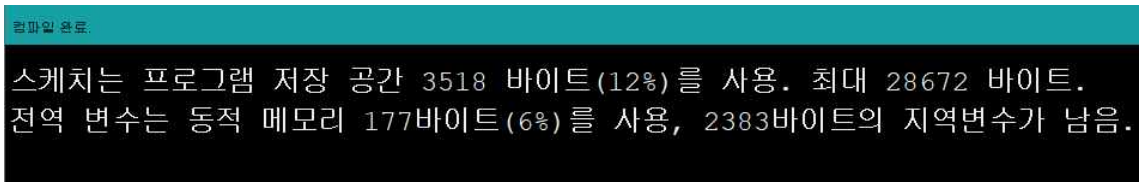


스케치 컴파일하기

7. 첫 번째 아이콘인 [확인] 버튼을 눌러 컴파일을 수행합니다.



8. [컴파일 완료]를 확인합니다.



용어설명 : 컴파일 작성한 스케치를 아두이노 보드 상에 있는 마이크로 컨트롤러가 읽을 수 있는 코드로 변형하는 작업이며, 컴파일러라는 프로그램이 이 작업을 수행합니다. 한글로 쓴 소설을 영어로 번역하여, 영어를 사용하는 사람들이 읽을 수 있도록 하는 작업과 같다고 보면 됩니다.

스케치 업로드하기

컴파일한 스케치를 아두이노 보드 상에 있는 마이컴에 쓰는 작업입니다. 업로드를 하면 전원을 꺼도 컴파일한 스케치의 내용은 마이컴 상에 남아 있습니다.

확인하기: '03 ESP32 아두이노 보드 패키지 설치하기'을 참고하여 업로드 직전에 아두이노 보드와 포트를 선택합니다.

9. [업로드] 버튼을 눌러줍니다.



10. 다음과 같이 업로드가 진행됩니다.

```
Hash of data verified.
Compressed 206544 bytes to 104402...
Writing at 0x00028000... (100 %)
```

11. [업로드 완료]를 확인합니다.

```
Leaving...
Hard resetting via RTS pin...
```

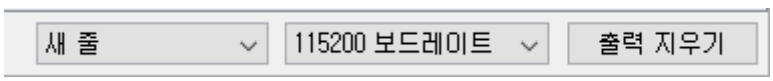
시리얼 모니터 확인

이제 결과를 시리얼 모니터를 통해 확인합니다.

12. [시리얼 모니터] 버튼을 눌러줍니다.



13. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



16. 다음과 같은 메시지가 반복적으로 뜨는 것을 확인합니다.

```
Hello PC^^. I'm an ESP32~
Hello PC^^. I'm an ESP32~
Hello PC^^. I'm an ESP32~
Hello PC^^. I'm an ESP32~
Hello PC^^. I'm an ESP32~
```

loop 함수가 반복돼서 호출되기 때문에 메시지도 반복돼서 뿌려지게 됩니다.

05 아두이노 오류 발생시 대처방법

앞으로 여러분은 아두이노를 다루는 과정에서 몇 가지 정형화된 오류를 반복적으로 접하게 됩니다. 이 때는 다음 순서로 문제를 해결해 보도록 합니다.

1. USB 포트 연결에 문제가 발생하는 경우가 있습니다. 이 경우엔 USB 연결을 해제한 후 다시 연결합니다.
2. 아두이노 소프트웨어에서 설정이 제대로 안되어 있는 경우가 있습니다. 이 경우엔 아두이노 소프트웨어의 tool 메뉴에서 보드와 포트가 제대로 선택되어 있는지 확인합니다.
3. 아두이노 스케치에 C 문법 오류가 있는 경우가 있습니다. 이 경우엔 반점(;-세미콜론), 괄호(소괄호(), 중괄호{}, 대괄호[]), 점(.), 함수 색상 순서로 확인해 봅니다.
4. 아두이노 보드 자체에 하드웨어적인 문제가 있는 경우가 있습니다. 이 경우엔 blink 예제로 아두이노 보드의 상태를 확인합니다.

06 아두이노 스케치 구조 이해하기

여기서는 아두이노 스케치의 구조에 대해서 자세히 살펴보도록 합니다.

아두이노 스케치는 다음과 같이 두 개의 기본 함수로 구성됩니다.

```
void setup() {
  // put your setup code here, to run once:

}

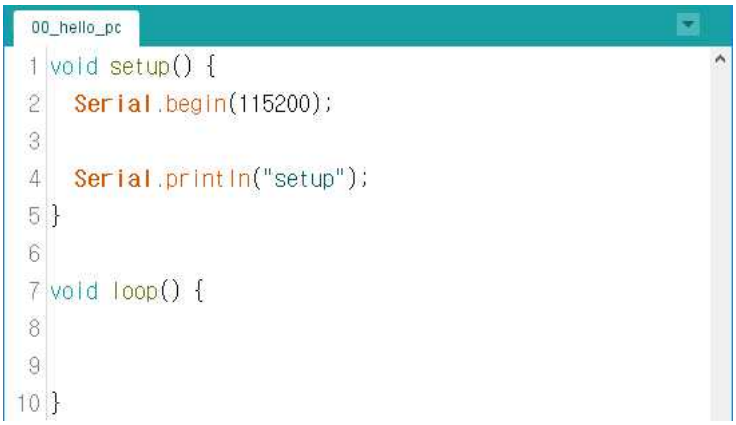
void loop() {
  // put your main code here, to run repeatedly:

}
```

setup 함수는 코드 실행을 시작할 때 한 번만 수행되며, 사용하고자 하는 하드웨어(입출력 모듈:센서, 모터 등)를 초기화시키는 부분입니다. loop 함수는 반복적으로 수행되며, 하드웨어를 반복적으로 동작 시키는 부분입니다.

*** 함수는 기능이라는 의미로 수학에서 유래하였으며 원하는 기능을 수행하기 위한 명령의 집합으로 구성됩니다.

1. setup 함수를 다음과 같이 수정해 봅니다.



2: 시리얼 포트의 통신 속도를 115200으로 설정합니다.
4: setup 문자열을 시리얼 포트로 출력합니다.

2. 업로드를 수행합니다.

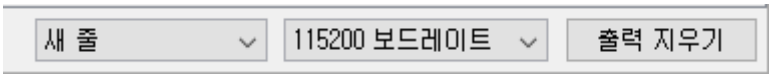


*** 업로드를 수행하면 컴파일 과정이 자동으로 포함됩니다.

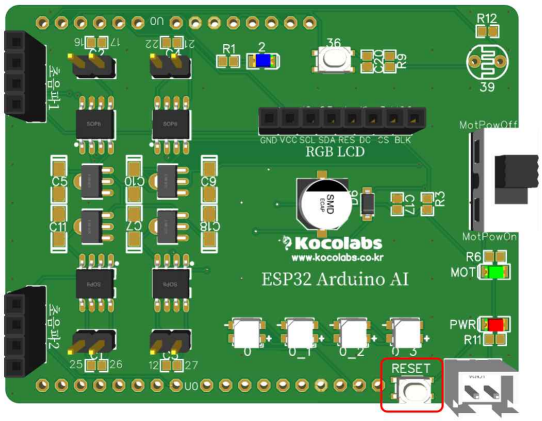
3. [시리얼 모니터] 버튼을 클릭합니다.



4. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



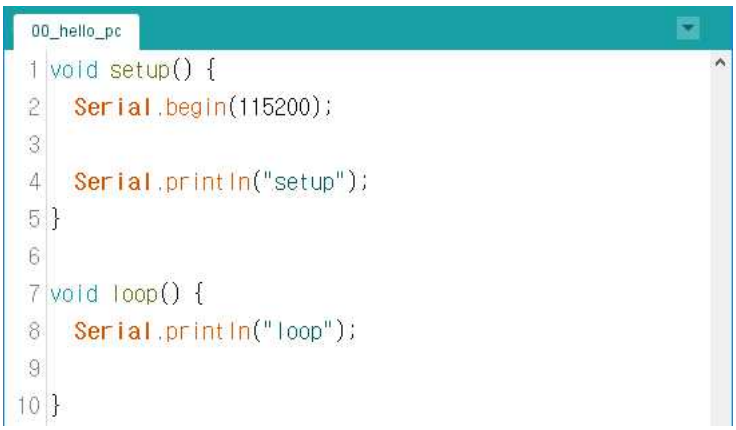
5. 다음에 표시된 [RESET] 버튼을 눌러 재부팅을 수행합니다.



6. setup 문자열이 출력되는 것을 확인합니다.

```
setup
```

7. 이번엔 loop 함수를 다음과 같이 수정합니다.



00_hello_pc

```
1 void setup() {
2   Serial.begin(115200);
3
4   Serial.println("setup");
5 }
6
7 void loop() {
8   Serial.println("loop");
9
10 }
```

8: loop 문자열을 시리얼 포트로 출력합니다.

8. 업로드를 수행합니다.



*** 업로드를 수행하면 컴파일 과정이 자동으로 포함됩니다.

9. 시리얼 모니터 창에 loop 문자열이 계속해서 출력되는 것을 확인합니다.

```
loop
loop
loop
loop
loop
```

Chapter 02
아두이노 인공지능 자동차 살펴보기

이번 장에서는 아두이노 인공지능 자동차를 구성하는 시리얼 출력과 입력, LED, 버튼, 빛 센서, 초음파 센서, DC 모터, 네오픽셀, RGB LCD에 대한 테스트를 수행하도록 합니다.

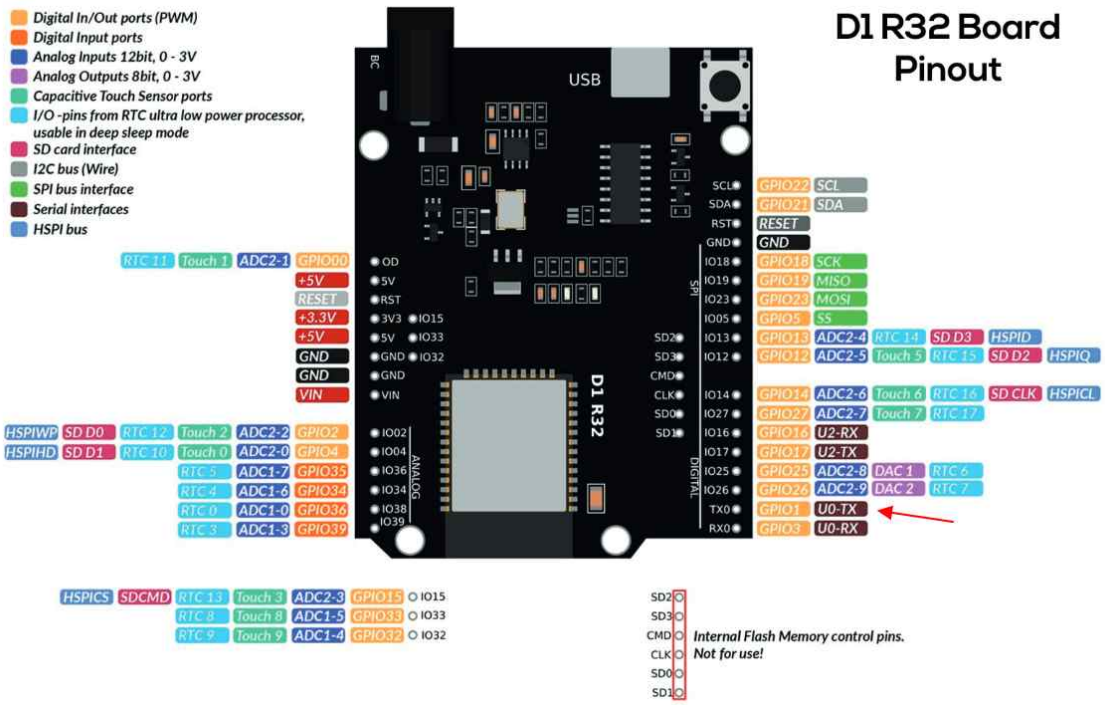
01 문자열 내보내기

앞에서 우리는 Serial.println 함수를 사용하여 다음과 같은 메시지를 PC로 보냈습니다.

```
Hello PC^^. I'm an ESP32~
Hello PC^^. I'm an ESP32~
Hello PC^^. I'm an ESP32~
Hello PC^^. I'm an ESP32~
Hello PC^^. I'm an ESP32~
```

아두이노가 여러분에게 인사를 한 것이죠. `Serial.println` 함수는 아주 유용한 함수입니다. 아두이노의 상태가 어떤지 우리에게 알려주는 주인공이 바로 `Serial.println` 함수입니다. 여러분은 앞으로 아두이노 스케치를 작성하다 버튼이나 센서의 값을 알고 싶은 경우가 있을 수 있습니다. 이 때 필요한 함수가 바로 `Serial.println` 함수입니다.

다음 그림에서 화살표가 가리키는 밤색 U0-TX 핀은 Serial.println 함수를 통해 PC로 문자열을 내보내는 핀입니다.



여러분은 아두이노 스케치를 통해 메시지를 출력할 때 다음 세 함수를 주로 사용하게 됩니다.

```
Serial.begin(speed)
Serial.println(val)
Serial.print(val)
```

Serial.begin

Serial.begin은 PC로 메시지를 보낼 때 데이터의 속도를 설정하는 함수입니다.

```
Serial.begin(speed);
             ❶
```

❶ PC로 메시지를 보낼 때 데이터 속도
(300, 600, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, 115200)

Serial.begin 함수는 PC로 메시지를 보낼 때 데이터의 속도를 설정합니다. speed 인자를 통해 설정할 수 있는 속도 값은 다음과 같습니다.

```
300, 600, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, 115200
```

우리 책에서는 주로 115200을 사용합니다. 115200bps는 초당 115200 비트를 보내는 속도입니다. 시리얼 포트를 통해 문자 하나를 보내는데 10비트가 필요합니다. 그러므로 1초에 115200/10 = 11520 문자를 보내는 속도입니다. 11520 문자는 A4 용지 기준 5~6페이지 정도의 양입니다. 비트는 0 또는 1을 담을 수 있는 데이터 저장의 가장 작은 단위입니다.

PC로 메시지를 보내기 위해서는 Serial.print 또는 Serial.println 함수를 사용합니다.

Serial.println

Serial.println은 PC로 메시지를 보내는 함수입니다.

```
Serial.println(val);
               ❶
```

❶ PC로 보낼 메시지로 정수, 실수, 문자열을 보낼 수 있다.

Serial.println 함수는 메시지 출력 후, 커서 위치를 다음 줄 첫 번째 칸으로 옮기는 엔터 키 입력 효과를 줍니다. val 인자를 통해 보낼 수 있는 값은 문자열, 정수 값, 실수 값을 보낼 수 있습니다.

Serial.print 함수는 메시지만 출력하고, 커서의 위치는 옮기지 않습니다.

다음과 같이 형식(format) 인자를 이용하여 메시지 형식을 좀 더 자세하게 줄 수도 있습니다.

```
Serial.println(val, format)
Serial.print(val, format)
```

format 인자의 경우 val 인자가 정수일 경우엔 진법(DEC, HEX, OCT, BIN)을 설정할 수 있으며, 실수인 경우엔 소수점 이하 표시할 자리수를 설정할 수 있습니다. 뒤에서 예제를 통해 사용법을 살펴봅니다.

*** 우리는 이 함수들을 이용하여 문자열과 숫자를 출력하게 됩니다. 숫자의 경우는 정수와 실수로 나눌 수 있습니다. 정수의 경우는 주로 10진수와 16진수로 표현할 수 있으며, 아두이노의 경우엔 2진수 표현도 가능합니다. 실수의 경우는 자릿수를 얼마나 나타낼지를 결정할 수 있습니다.

*** 아두이노 ESP32의 경우 C에서 사용하는 printf 함수를 지원합니다.

01 여러 형식의 자료 내보내기

여기서는 Serial.println 함수를 이용하여 문자열, 숫자, 문자를 출력해봅니다.

1. 다음과 같이 예제를 작성합니다.
211.ino

```
01 void setup() {
02       Serial.begin(115200);
03
04       Serial.println("Hello PC^^. I'm an ESP32~");
05       Serial.println(78);
06       Serial.println(1.23456);
07       Serial.println('N');
08 }
09
10 void loop() {
11
12 }
```

04 : 문자열을 출력합니다.
05 : 정수 78을 10진수 문자열로 변환하여 출력합니다.
06 : 실수 1.23456을 10진 실수 문자열로 변환하여 출력합니다.
07 : 문자 N을 문자열로 변환하여 출력합니다.

2. [툴] 메뉴를 이용하여 보드, 포트를 다음과 같이 선택합니다.



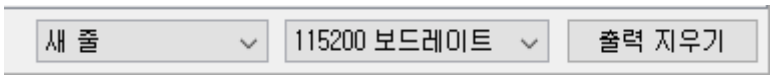
3. 업로드를 수행합니다.



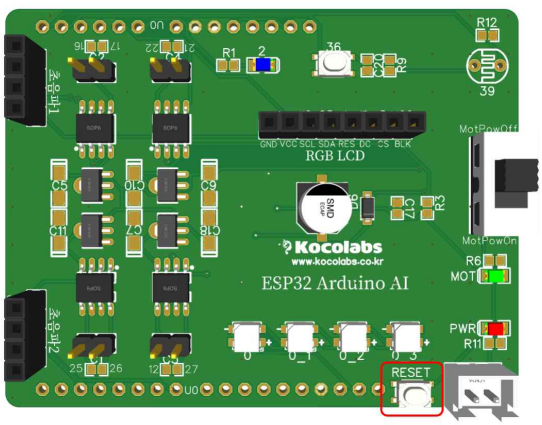
4. [시리얼 모니터] 버튼을 클릭합니다.



5. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



6. 다음에 표시된 [RESET] 버튼을 눌러 재부팅을 수행합니다.



7. 출력결과를 확인합니다.

```
Hello PC^^. I'm an ESP32~
78
1.23
N
```

1.23456 실수의 경우 기본적으로 소수점 아래 두 자리만 출력하는 것을 볼 수 있습니다.

용어설명: C의 자료형 C에서 일반적으로 사용하는 자료 형은 int, double, char *, char입니다. int는 정수 값을 담을 수 있는 자료 형을, double은 실수 값을 담을 수 있는 자료 형을, char *는 문자열의 첫 문자의 주소를 담을 수 있는 자료 형을, char는 한 문자를 담을 수 있는 자료 형을 나타냅니다. 정수의 경우엔 10진수와 16진수 두 종류가 있습니다. 10진수의 경우엔 개수나 번호 등에 사용되며, 16진수는 메모리 주소 값이나 특정한 비트의 값을 나타낼 때 사용합니다. 10진수는 주로 사칙연산자나 비교연산자와 같이 사용되며, 16진수는 주로 비트연산자와 같이 사용됩니다.

02 여러 형식의 숫자 내보내기

여기서는 Serial.println 함수를 이용하여 10진수와 16진수 정수를 출력해 봅니다. 또, 10진 실수의 소수점이하 출력을 조절해 봅니다.

1. 다음과 같이 예제를 작성합니다.

212.ino

```
01 void setup() {
02       Serial.begin(115200);
03
04       Serial.println(78, DEC);
05       Serial.println(78, HEX);
06       Serial.println(78, BIN);
07
08       Serial.println(1.23456, 0);
09       Serial.println(1.23456, 2);
10       Serial.println(1.23456, 4);
11 }
12
13 void loop() {
14
15 }
```

04 : 정수 78을 10진수 문자열로 변환하여 출력합니다.

05 : HEX 형식은 정수를 16진수 문자열로 변환하는 형식입니다. 여기서는 정수 78을 16진수 문자열로 변환하여 출력합니다.

06: 정수 78을 2진수 문자열로 변환하여 출력합니다.
08 : 여기서는 실수 1.23456을 소수점 이하 0개까지 10진 실수 문자열로 변환하여 출력합니다.
09 : 여기서는 실수 1.23456을 소수점 이하 2개까지 10진 실수 문자열로 변환하여 출력합니다.
10 : 여기서는 실수 1.23456을 소수점 이하 4개까지 10진 실수 문자열로 변환하여 출력합니다.

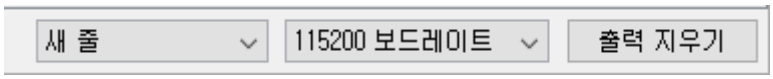
2. 업로드를 수행합니다.



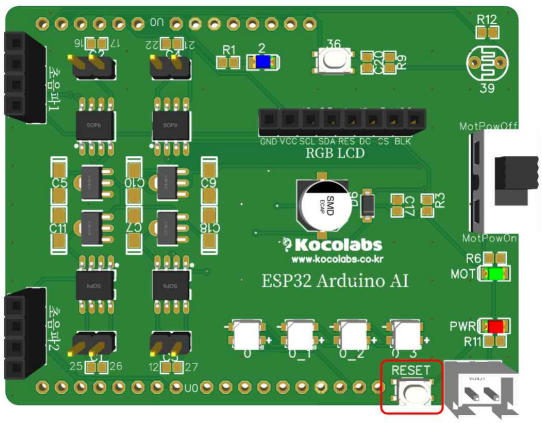
3. [시리얼 모니터] 버튼을 클릭합니다.



4. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



5. 다음에 표시된 [EN] 버튼을 눌러 재부팅을 수행합니다.



6. 출력결과를 확인합니다.

```
78
4E
1001110
1
1.23
1.2346
```

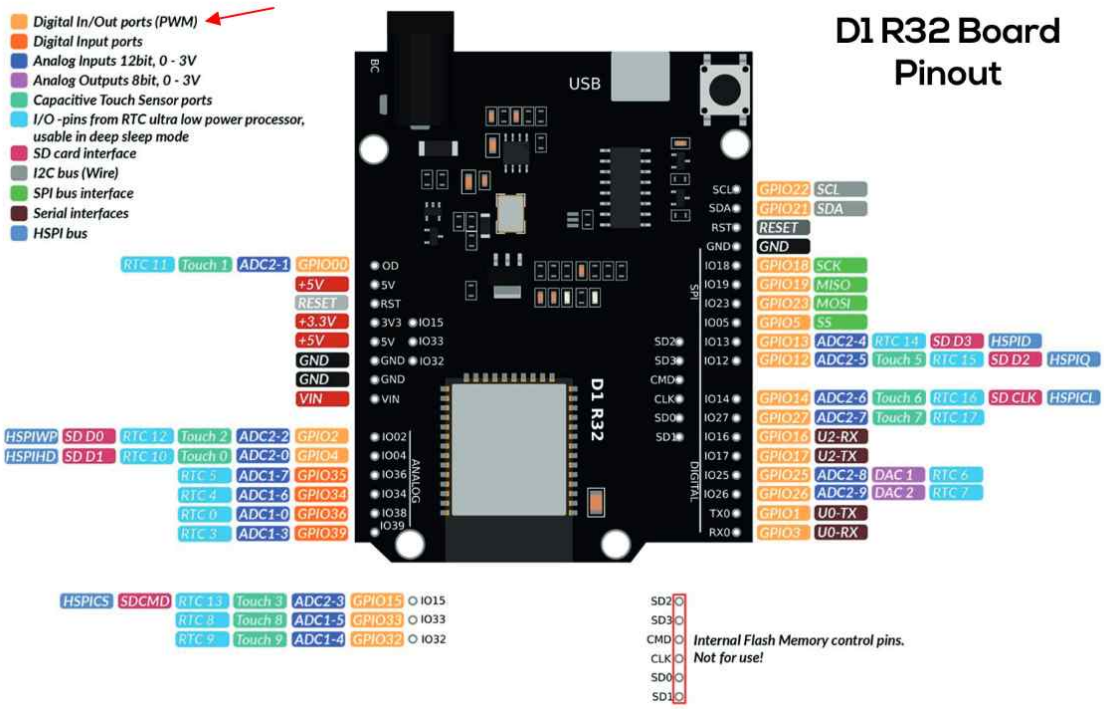

Kocolabs
www.kocolabs.co.kr

02 LED 켜고 끄기

여러분은 다음과 같이 유튜브 등에서 아두이노를 이용하여 LED를 깜빡이는 동영상을 본적이 있나요?



다음 그림에서 화살표가 가리키는 주황색 핀은 LED를 제어할 수 있는 핀입니다.



여러분은 아두이노 스케치를 통해 LED를 제어할 때 다음 세 함수를 주로 사용하게 됩니다.

```
pinMode(pin, mode)
digitalWrite(pin, value)
delay(ms)
```

pinMode

pinMode란 특정 핀을 출력 또는 입력 모드로 설정하는 명령어입니다.

```
pinMode(pin, mode);
        ❶    ❷
```

❶ 설정하고자 하는 핀 번호
❷ 설정하고자 하는 모드로 입력일 때는 INPUT, 출력일 때는 OUTPUT

pinMode 함수는 특정한 핀을 출력으로 사용할지 입력으로 사용할지를 설정합니다. pin 인자로는 보드 상에 나와 있는 숫자를 사용합니다. 본 책에서 사용하는 아두이노 ESP32의 경우 앞에서 살펴본 그림에서 화살표가 가리키는 주황 색깔의 핀에 해당하는 숫자를 사용합니다. mode 인자로는 OUTPUT, INPUT, INPUT_PULLUP을 사용할 수 있습니다. LED를 켜기 위해서는 0 또는 1을 LED로 쓰는 개념이기 때문에 OUTPUT으로 설정합니다. 버튼의 경우 버튼의 값을 읽는 개념이기 때문에 INPUT으로 설정합니다. 버튼의 경우 외부에 저항을 이용하여 회로를 구성하는데, 외부에 저항을 사용하지 않고 아두이노의 칩 내부에 있는 저항을 이용할 경우엔 INPUT_PULLUP으로 설정합니다.

digitalWrite

digitalWrite란 특정 핀을 HIGH 또는 LOW로 설정하는 명령어입니다.

```
digitalWrite(pin, value);
             ❶    ❷
```

❶ 제어하고자 하는 핀 번호
❷ HIGH 또는 LOW

digitalWrite 함수는 디지털 핀으로 HIGH(=1) 또는 LOW(=0) 값을 씁니다. 아두이노 ESP32의 주황 색깔의 핀의 경우 pinMode 함수를 통해 해당 핀이 OUTPUT으로 설정되었을 때, HIGH 값의 경우엔 해당 핀이 3.3V로 설정되며, LOW 값의 경우엔 0V로 설정됩니다.

delay

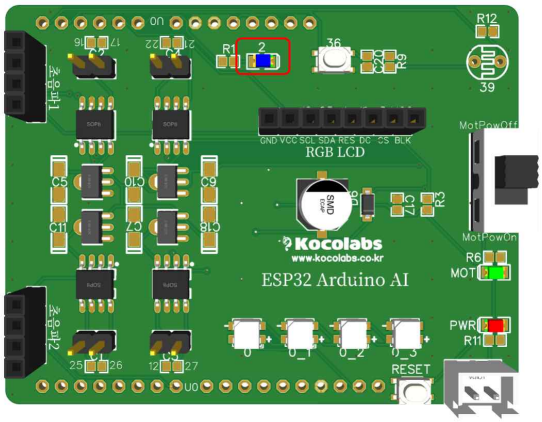
delay란 인자로 주어진 시간만큼 프로그램의 진행을 멈춥니다.

```
delay(ms);
      ❶
```

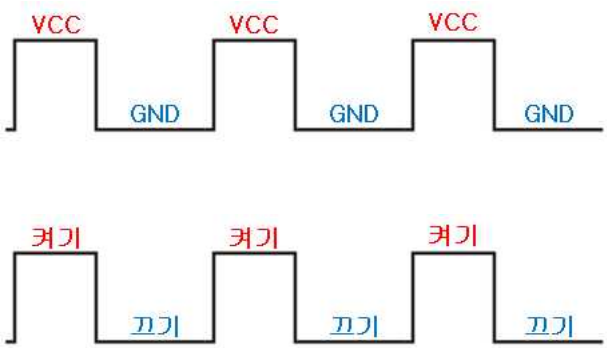
❶ 멈춰야할 밀리초(ms : unsigned long 형)
*** unsigned long은 변수형의 한 종류로 아두이노 스케치에서 0~4,294,967,295 (2^32 - 1) 범위의 0과 양의 정수 값을 갖습니다.

여기서는 digitalWrite 함수를 이용하여 아두이노 ESP32의 2번 핀에 연결된 LED를 켜보고 꺼보는 예제를 수행해 봅니다. 다음 그림에서 빨간 사각형으로 표시된 파란 LED가 2번 핀에

연결된 LED입니다.



또 반복적으로 켜고 끄는 주기를 짧게 해가며 아래 그림과 같은 사각 파형에 대해서도 알아보도록 합니다.



01 LED 켜고 끄기 확인하기

먼저 digitalWrite 함수를 이용하여 LED를 주기적으로 켜고 꺼봅니다. 여기서는 아두이노 ESP32의 2번 핀에 연결된 LED를 주기적으로 켜고 꺼 봅니다.

1. 다음과 같이 예제를 작성합니다.
221.ino

```
01 const int LED = 2;
02
03 void setup() {
04       pinMode(LED, OUTPUT);
05 }
06
07 void loop() {
08       digitalWrite(LED, HIGH);
```

```
09        delay(500);
10        digitalWrite(LED, LOW);
11        delay(500);
12 }
```

01 : LED 상수에 2번 핀을 할당합니다.

04 : pinMode 함수를 이용하여 LED를 출력으로 설정하고 있습니다. pinMode 함수는 digitalWrite 함수를 이용하여 HIGH, LOW 값을 쓰고자 할 때 사용하는 함수입니다.

08 : digitalWrite 함수를 이용하여 LED에 HIGH 값을 씁니다. 그러면 LED는 켜지게 됩니다.

09 : 500 밀리초간 지연을 줍니다. delay 함수는 아두이노가 아무것도 수행하지 않고 일정시간을 기다리게 하는 함수입니다. 함수의 인자로 주어지는 500은 밀리 초 단위입니다. 여기서는 500 밀리 초 동안 아두이노가 아무것도 수행하지 않습니다.

10 : digitalWrite 함수를 이용하여 LED에 LOW 값을 씁니다. 그러면 LED는 꺼지게 됩니다.

11 : 500 밀리초간 지연을 줍니다.

2. [툴] 메뉴를 이용하여 보드, 포트를 다음과 같이 선택합니다.



3. 업로드를 수행합니다.



4. LED의 동작을 확인합니다. 1초 주기로 LED가 켜졌다 꺼졌다 하는 것을 확인합니다. 즉, 1Hz의 주파수로 LED가 점멸하는 것을 확인합니다.



LED의 점등은 LED(=2) 핀을 통해 나오는 HIGH 값에 의해 발생합니다. LED의 소등은 LED 핀을 통해 나오는 LOW 값에 의해 발생합니다. 즉, LED 핀으로는 위 그림과 같이 HIGH값과 LOW 값이 1초 주기로 나오게 되며, 이 값들에 의해 LED는 점멸을 반복하게 됩니다. 그리고 이 경우 여러분은 LED가 점멸 하는 것을 느낄 수 있습니다.

*** Hz : 같은 동작이 1초에 1 번씩 반복될 때 우리는 1Hz로 동작한다고 합니다. 같은 동작이 1초에 2 번씩 반복될 때 우리는 2Hz로 동작한다고 합니다.

02 LED 켜고 끄기 간격 줄여보기

여기서는 digitalWrite 함수를 이용하여 아래와 같은 사각 파형에 대한 주파수와 상하비의 개념을 이해해 보도록 합니다.



주파수란 1초간 반복되는 사각 파형의 개수를 의미하며, 상하 비란 사각 파형의 HIGH 값과 LOW 값의 비를 의미합니다.

이제 LED의 점멸 간격을 줄여보도록 합니다. 그러면 여러분은 좀 더 조밀하게 LED가 점멸하는 것을 느낄 것입니다.

1. 다음과 같이 이전 예제를 수정합니다.
222.ino

```
01 const int LED = 2;
02
03 void setup() {
04       pinMode(LED, OUTPUT);
05 }
06
07 void loop() {
08       digitalWrite(LED, HIGH);
09       delay(50);
10       digitalWrite(LED, LOW);
```

```
11        delay(50);
12 }
```

09, 11 : 500을 50으로 변경합니다.

2. 업로드를 수행합니다.



3. LED의 동작을 확인합니다. 이 예제의 경우 LED는 초당 10번 점멸하게 됩니다. 즉, 10Hz의 주파수로 점멸하게 됩니다.



그림과 같은 파형이 초당 10개가 생성됩니다. 이 경우에도 여러분은 반복적으로 LED가 점멸하는 것을 느낄 것입니다. 그러나 그 간격은 더 조밀하게 느껴질 것입니다.

03 LED 켜고 끄기를 밝기로 느껴보기

LED의 점멸 간격을 더 줄여보도록 합니다. 여기서 여러분은 LED의 점멸을 느끼지 못하게 될 것입니다. 오히려 LED가 일정한 밝기로 켜져 있다고 느낄 것입니다.

1. 다음과 같이 예제를 수정합니다.
223.ino

```
01 const int LED = 2;
02
03 void setup() {
04        pinMode(LED, OUTPUT);
05 }
06
07 void loop() {
08        digitalWrite(LED, HIGH);
09        delay(5);
10        digitalWrite(LED, LOW);
11        delay(5);
12 }
```

09, 11 : 50을 5로 변경합니다.

2. 업로드를 수행합니다.



3. LED의 동작을 확인합니다. 이 예제의 경우 LED는 초당 100번 점멸 하게 됩니다. 즉, 100Hz의 주파수로 점멸하게 됩니다.



그림과 같은 파형이 초당 100개가 생성됩니다. 이제 여러분은 LED가 점멸하는 것을 느끼지 못할 것입니다. 오히려 LED가 일정하게 켜져 있다고 느낄 것입니다.

일반적으로 이러한 파형이 초당 50개 이상이 되면, 즉, 50Hz 이상의 주파수로 LED 점멸을 반복하면 우리는 그것을 느끼기 어렵습니다.

04 LED 어둡게 하기

이제 delay 함수를 조절하여 LED의 밝기를 어둡게 해 봅니다. 이전 예제의 경우 LED는 100Hz의 속도로 50%는 점등을, 50%는 소등을 반복하였습니다. 그리고 이 경우 우리는 LED의 밝기를 평균값인 50%의 밝기로 느꼈습니다. 만약 LED에 대해 10%는 점등을, 90%는 소등을 반복한다면 우리는 LED의 밝기를 어떻게 느낄까요? 평균 10%의 밝기로 느끼게 되지 않을까요? 예제를 통해 확인해 보도록 합니다.

1. 다음과 같이 예제를 수정합니다.
224.ino

```
01 const int LED = 2;
02
03 void setup() {
04       pinMode(LED, OUTPUT);
05 }
06
07 void loop() {
08       digitalWrite(LED, HIGH);
```

```
09        delay(1);
10        digitalWrite(LED, LOW);
11        delay(9);
12 }
```

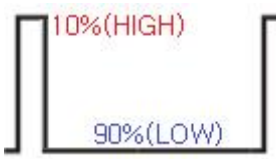
09 : 5를 1로 변경합니다.

11 : 5를 9로 변경합니다.

2. 업로드를 수행합니다.



3. LED의 동작을 확인합니다. 이 예제의 경우도 LED는 초당 100번 점멸 하게 됩니다. 즉, 100Hz의 주파수로 점멸하게 됩니다. 그러나 10%는 점등 상태로, 90%는 소등 상태로 있게 됩니다. 그래서 우리는 LED의 밝기가 이전 예제보다 낮다고 느끼게 됩니다.



그림에서 LED는 실제로 10%만 점등 상태이지만 100Hz의 주파수로 점멸하기 때문에 우리는 10%의 평균 밝기로 느끼게 됩니다. 10%는 HIGH 값에 의해 켜져 있고 90%는 LOW 값에 의해 꺼져있으며, 이 경우 (HIGH:LOW)=(1:9)가 되게 됩니다. 즉, 상하비가 1:9가 됩니다.

05 LED 밝게 하기

이제 반대로 LED의 밝기를 밝게 해 봅니다.

1. 다음과 같이 예제를 수정합니다.
225.ino

```
01 const int LED = 2;
02
03 void setup() {
04        pinMode(LED, OUTPUT);
05 }
06
07 void loop() {
08        digitalWrite(LED, HIGH);
09        delay(9);
```

```
10        digitalWrite(LED, LOW);
11        delay(1);
12 }
```

09 : 1를 9로 변경합니다.

11 : 9를 1로 변경합니다.

2. 업로드를 수행합니다.



3. LED의 동작을 확인합니다. 이 예제의 경우도 LED는 초당 100번 점멸 하게 됩니다. 즉, 100Hz의 주파수로 점멸하게 됩니다. 그러나 90%는 점등 상태로, 10%는 소등 상태로 있게 됩니다. 그래서 우리는 LED가 이전 예제에 비해 아주 밝다고 느끼게 됩니다.



그림에서 LED는 실제로 90%만 점등 상태이지만 100Hz의 주파수로 점멸하기 때문에 우리는 90%의 평균 밝기로 느끼게 됩니다. 90%는 HIGH 값에 의해 켜져 있고 10%는 LOW 값에 의해 꺼져 있으며, 이 경우 (HIGH:LOW)=(9:1)이 되게 됩니다. 즉, 상하비가 9:1이 됩니다.

상하비가 8:2가 되면 우리는 LED가 80%의 밝기로 켜져 있다고 느끼게 됩니다. 9:1에 해당되는 부분을 차례대로 다음과 같이 바꾸어 볼 수 있습니다.

```
0:10, 1:9, 2:8, 3:7 ... 10:0
```

우리는 HIGH와 LOW의 상하 비에 따라 LED의 밝기를 조절할 수 있습니다.

06 LED 밝기 조절해 보기

여기서는 1초 간격으로 다음의 상하비로 LED의 밝기를 조절해 보도록 합니다.

```
0:10, 1:9, 2:8, 3:7 ... 10:0
```

즉, HIGH의 개수는 0부터 10까지 차례로 늘어나며, 반대로 LOW의 개수는 10부터 0까지 차례로 줄게 됩니다.

0.01초 간격으로 LED 밝기 11 단계 조절해보기

먼저 0.01초 간격으로 LED의 밝기를 11단계로 조절해 봅니다.

1. 다음과 같이 예제를 수정합니다.

226.ino

```
01 const int LED = 2;
02
03 void setup() {
04       pinMode(LED, OUTPUT);
05 }
06
07 void loop() {
08       for(int t_high =0;t_high <=10;t_high ++) {
09                digitalWrite(LED, HIGH);
10                delay(t_high);
11                digitalWrite(LED, LOW);
12                delay(10 -t_high);
13       }
14 }
```

08 : t_high 변수를 0부터 10까지 1씩 증가시켜가면서, 중괄호 안쪽(8줄~13줄)의 동작을 수행합니다.

09, 10 : LED를 켜고 t_high 시간만큼 기다립니다.

11, 12 : LED를 끄고 (10-t_high) 시간만큼 기다립니다.

10, 12 : t_high + (10 - t_high) = 10이 되어 for문을 한 번 도는 데는 10밀리 초 정도가 되며 for문 전체를 도는 데는 110밀리 초 정도가 됩니다.

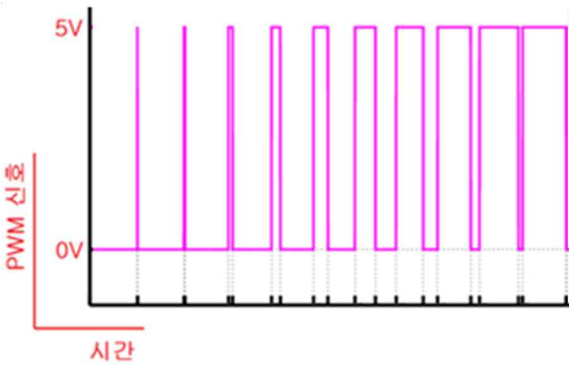
2. 업로드를 수행합니다.



3. LED의 동작을 확인합니다. 10밀리 초 간격으로 다음의 비율로 LED가 밝아집니다.

```
0%, 10% 20%, 30%, ... 100%
```

아래와 같은 형태의 파형이 반복되면서 LED의 밝기가 변합니다.



이 예제의 경우 밝기의 변화가 너무 빨라 밝기가 변하는 것을 느끼기 힘듭니다. 깜빡임으로 느낄 수 있습니다.

0.1초 간격으로 LED 밝기 11 단계 조절해보기

다음은 0.1초 간격으로 LED의 밝기를 11단계로 조절해 봅니다.

1. 다음과 같이 예제를 수정합니다.

226_2.ino

```
01 const int LED = 2;
02
03 void setup() {
04       pinMode(LED, OUTPUT);
05 }
06
07 void loop() {
08       for(int t_high =0;t_high <=10;t_high ++) {
09                int cnt =0;
10                while(true) {
11                         digitalWrite(LED, HIGH);
12                         delay(t_high);
13                         digitalWrite(LED, LOW);
14                         delay(10 -t_high);
15
16                         cnt ++;
17                         if(cnt ==10) break;
18                }
19       }
20 }
```

08 : for 문을 사용하여 t_high 변수 값을 0부터 10까지 주기적으로 변경하고 있습니다.

t_high 변수 값은 16, 18 번째 줄에서 사용되며, LED을 통해 HIGH, LOW 값이 나가는 시간 값을 가집니다.

10 : 조건이 없는 while 문을 수행합니다. while 문을 나오는 조건은 21 번째 줄에 있으며, 1초 간격으로 나오게 됩니다.

09 : cnt 변수 생성 후, 0으로 초기화합니다.

16 : cnt 변수를 하나씩 증가시킵니다.

17 : cnt 변수가 10이 되면 break 문을 수행하여 while 문을 벗어납니다.

08 : cnt 변수를 선언한 후, 0으로 초기화합니다.
09 : 무한루프를 돌면서
15 : cnt 값을 하나씩 증가시킵니다.
16 : cnt 값이 10이 되면 내부 while 문을 나옵니다.

이렇게 하면 09~17줄을 cnt값이 0에서 9까지 10회 반복하게됩니다. 그러면 0.001*t_high 값을 유지하는 시간을 10밀리초(0.01초)에서 100밀리초(0.1초)로 늘릴 수 있습니다. for 문을 수행하는 시간도 110밀리초(0.11초)에서 1100밀리초(1.1초)로 늘릴 수 있으며, 우리는 LED 밝기의 변화를 느낄 수 있습니다.

2. 업로드를 수행합니다.



3. LED의 동작을 확인합니다. 1.1 초 주기로 다음의 비율로 LED가 밝아집니다.

0%, 10% 20%, 30%, ... 100%

03 LED 밝기 조절하기

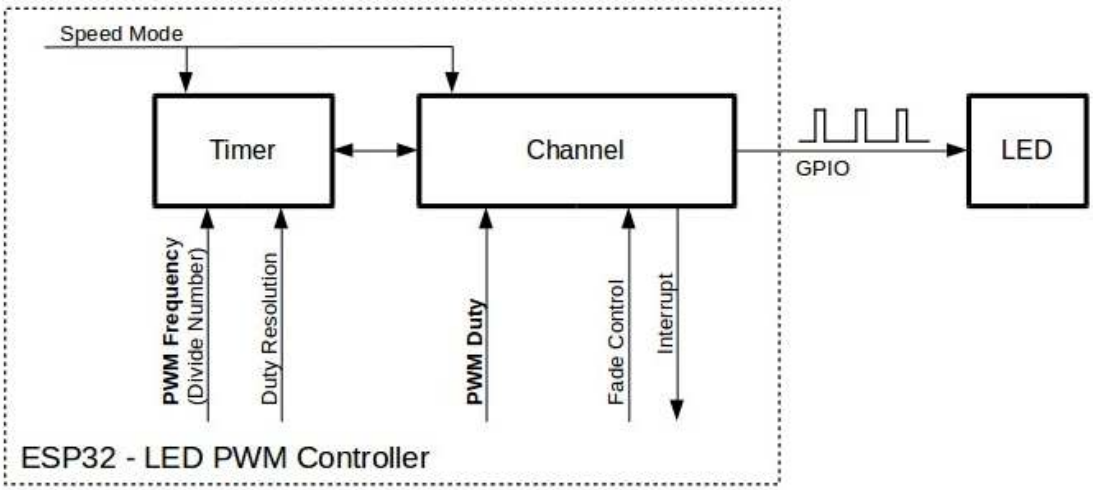
이전 예제에서 우리는 100Hz의 속도로 1:9의 상하비로 LED의 밝기를 조절해 보았습니다. 2번 핀에 ledcWrite 함수를 사용할 경우 더 조밀한 상하비로 LED의 밝기를 조절할 수 있습니다. 예를 들어, 0~1023 사이의 HIGH 값으로 LED의 밝기를 조절할 수 있습니다.

아두이노 ESP32의 경우 analogWrite 함수 대신에 ledcWrite 함수를 지원합니다. ledcWrite 함수는 사각파형에 대한 세밀한 제어 기능을 제공합니다.

아두이노 ESP32의 경우 ledcWrite 함수는 PWM 핀에 대해 아래와 같은 형태의 사각 파형을 내보내며, 특히 상하비를 결정하는 역할을 합니다. 상하비는 한 주기당 3.3V 비율을 의미합니다.

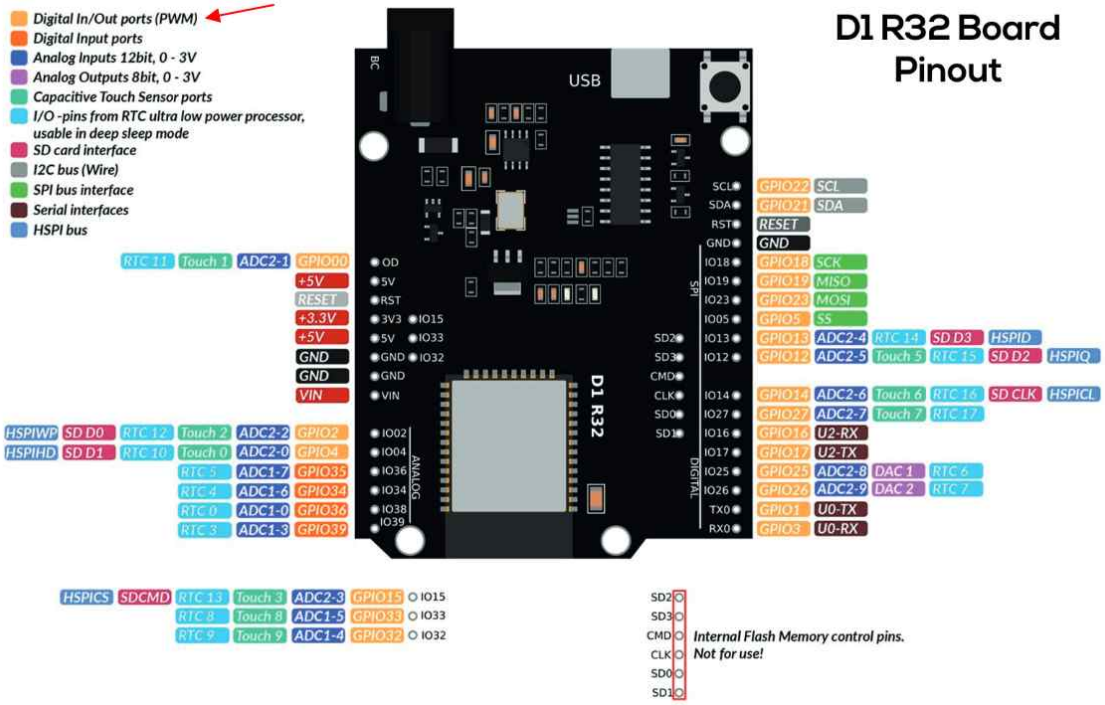


다음 그림은 아두이노 ESP32 내부의 LED PWM Controller 모듈입니다.



ledcWrite 함수는 내부 LED PWM Controller 모듈에 명령을 주어 해당 핀으로 일정한 모양의 사각파형을 내보내게 합니다. LED PWM Controller 모듈은 새로운 ledcWrite 명령을 받을 때까지 해당 핀으로 똑같은 사각 파형을 내보냅니다.

다음 그림에서 화살표가 가리키는 주황색 핀은 사각 파형(PWM 파형)을 내보낼 수 있는 핀입니다.



여러분은 아두이노 스케치를 통해 사각 파형을 내보낼 때 다음 세 함수를 사용하게 됩니다.

```
ledcAttachPin(pin, channel)
ledcSetup(channel, frequency, resolution)
ledcWrite(channel, dutycycle)
```

ledcAttachPin

ledcAttachPin이란 특정 핀으로 사각파형을 내보낼 채널을 연결하는 명령어입니다.

ledcAttachPin(pin, channel);

❶ ❷

❶ 제어하고자 하는 핀 번호입니다. 정수형을 씁니다.
❷ 사각 파형을 내보낼 채널 번호입니다. 0~15 사이의 값을 쓸 수 있습니다. 정수형을 씁니다.

ledcSetup

ledcSetup이란 특정 채널에 주파수, 듀티 사이클 정밀도를 설정하는 명령어입니다.

ledcSetup(channel, frequency, resolution);

❶ ❷ ❸

❶ 사각 파형을 내보낼 채널 번호입니다. 정수형을 씁니다.
❷ 사각 파형의 주파수입니다. 정수형을 씁니다.
❸ 사각 파형의 듀티사이클의 정밀도입니다. 예를 들어, resolution 값이 10 이면 듀티사이클 값은

0~1023($2^{10}-1$)사이의 값을 갖습니다.

ledcWrite

ledcWrite란 특정 채널에 듀티사이클을 적용하는 명령어입니다.

ledcWrite(channel, dutycycle);

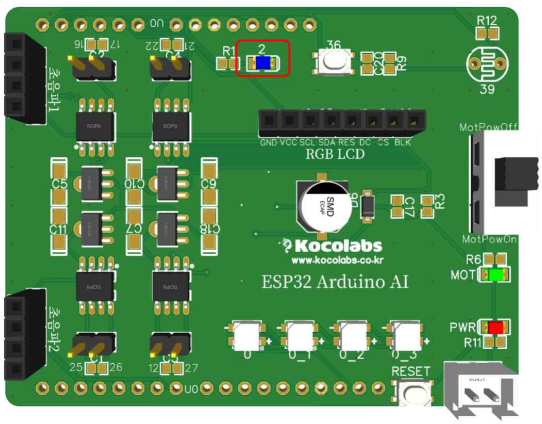
❶ ❷

❶ 사각 파형을 내보낼 채널 번호입니다. 0~16 사이의 값을 쓸 수 있습니다. 정수형을 씁니다.
❷ 듀티 사이클 값입니다. ledcSetup 함수에서 설정한 해상도 인자에 따라 결정됩니다. 예를 들어, resolution 값이 10 이면 듀티사이클 값은 0~1023($2^{10}-1$)사이의 값을 갖습니다.

아두이노 ESP32의 PWM 핀은 내부적으로 0~15의 채널에 연결 시킬 수 있으며, 사각 파형은 채널을 통해 핀으로 전달됩니다.

01 주파수 조절로 LED 켜고 끄기 반복해 보기

먼저 주파수 조절로 2번 핀에 연결된 LED를 켰다 껐다를 반복해 봅니다.



1. 다음과 같이 예제를 작성합니다.

231.ino

```
01 const int led_pin = 2;
02 const int led_channel = 1;
03 const int led_freq = 1;
04 const int led_resolution = 10;
05
06 void setup() {
07       ledcAttachPin(led_pin, led_channel);
08       ledcSetup(led_channel, led_freq, led_resolution);
```

```
09
10        ledcWrite(led_channel, 100);
11 }
12
13 void loop() {
14
15 }
```

01 : led_pin 정수 상수를 선언하고, 2번 핀으로 초기화합니다.

02 : led_channel 정수 상수를 선언하고, 1번 채널로 초기화합니다. 채널 번호는 0~15를 사용할 수 있습니다.

03 : led_freq 정수 상수를 선언하고, 1로 초기화합니다. 이렇게 하면 주파수를 1로 설정하게 됩니다.

04 : led_resolution 정수 상수를 선언하고, 10으로 초기화합니다. 이렇게 하면 사각 파형의 상하비 값을 0~1023($2^{10}-1$)사이의 값으로 조절할 수 있습니다.

07 : ledcAttachPin 함수를 호출하여 led_pin을 led_channel에 붙입니다.

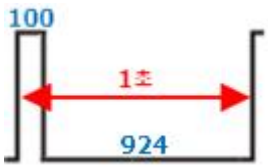
08 : ledcSetup 함수를 호출하여 led_channel의 주파수와 상하비 해상도를 설정합니다.

10 : ledcWrite 함수를 호출하여 led_channel로 100의 상하비 값을 씁니다. 이렇게 하면 100만큼 HIGH, 924만큼 LOW인 사각 파형이 생성됩니다.

2. 업로드를 수행합니다.



3. LED의 동작을 확인합니다. 1초 주기로 LED가 켜졌다 꺼졌다 하는 것을 확인합니다. 즉, 1Hz의 주파수로 LED가 켜졌다 꺼졌다 하는것을 확인합니다. HIGH 구간은 100만큼, LOW 구간은 924만큼의 파형이 주기적으로 생성됩니다.



LED 켜기 끄기 간격 줄여보기

이제 LED의 깜빡임 간격을 줄여보도록 합니다. 그러면 여러분은 좀 더 조밀하게 LED가 깜빡이는 것을 느낄 것입니다.

1. 이전 예제를 다음과 같이 수정합니다.

231_2.ino

```
01 const int led_pin = 2;
02 const int led_channel = 1;
03 const int led_freq = 10;
04 const int led_resolution = 10;
05
06 void setup() {
07       ledcAttachPin(led_pin, led_channel);
08       ledcSetup(led_channel, led_freq, led_resolution);
09
10       ledcWrite(led_channel, 100);
11 }
12
13 void loop() {
14
15 }
```

03 : led_freq 정수 상수 값을 10으로 설정하여 주파수를 10으로 맞추어 주고 있습니다. 이 경우 주파수는 10Hz가 됩니다.

2. 업로드를 수행합니다.



3. LED의 동작을 확인합니다. 이 예제의 경우 LED는 초당 10번 켜졌다 꺼졌다 하게 됩니다. 즉, 10Hz의 주파수로 깜빡이게 됩니다.



반복적인 LED 켜기 끄기를 일정한 밝기로 느껴보기

LED의 깜빡임 간격을 더 줄여보도록 합니다. 여기서 여러분은 LED의 깜빡임을 느끼지 못하게 될 것입니다. 오히려 LED가 일정하게 켜져 있다고 느낄 것입니다.

1. 이전 예제를 다음과 같이 수정합니다.

231_3.ino

```
01 const int led_pin = 2;
02 const int led_channel = 1;
03 const int led_freq = 100;
04 const int led_resolution = 10;
05
06 void setup() {
07       ledcAttachPin(led_pin, led_channel);
08       ledcSetup(led_channel, led_freq, led_resolution);
09
10       ledcWrite(led_channel, 100);
11 }
12
13 void loop() {
14
15 }
```

03 : led_freq 정수 상수 값을 100으로 설정하여 주파수를 100으로 맞추어 주고 있습니다. 이 경우 주파수는 100Hz가 됩니다.

2. 업로드를 수행합니다.



3. LED의 동작을 확인합니다. 이 예제의 경우 LED는 초당 100번 켜졌다 꺼졌다 하게 됩니다. 즉, 100Hz의 주파수로 깜빡이게 됩니다.



그림과 같은 파형이 초당 100개가 생성됩니다. 이제 여러분은 LED가 깜빡이는 것을 느끼지 못할 것입니다. 오히려 LED가 일정한 밝기로 켜져있다고 느낄 것입니다.

02 상하비 조절로 LED 밝기 조절해 보기

여기서는 상하비 조절로 1024단계로 LED 밝기를 변경해 봅니다. 다음의 상하비로 LED의 밝기를 조절해 보도록 합니다.

```
0:1024, 1:1023, 2:1022, 3:1021 ... 1024:0
```

즉, HIGH의 개수는 0부터 1024까지 차례로 늘어나며, 반대로 LOW의 개수는 1024부터 0까지 차례로 줄게 됩니다.

1. 이전 예제를 다음과 같이 수정합니다.

232.ino

```
01 const int led_pin = 2;
02 const int led_channel = 1;
03 const int led_freq = 100;
04 const int led_resolution = 10;
05
06 void setup() {
07       ledcAttachPin(led_pin, led_channel);
08       ledcSetup(led_channel, led_freq, led_resolution);
09
10       ledcWrite(led_channel, 0);
11 }
12
13 void loop() {
14
15       for(int t_high=0;t_high<=1024;t_high++) {
16                ledcWrite(led_channel, t_high);
17                delay(1);
18       }
19
20 }
```

10 : ledcWrite 함수를 호출하여 led_channel의 HIGH 구간을 0으로 설정합니다.

15 : for 문을 사용하여 t_high 변수 값을 0부터 1024까지 1 간격으로 변경하며 16~17줄을 수행합니다.

16 : ledcWrite 함수를 호출하여 LED에 0, 1, 2, ... 1024의 HIGH 값을 줍니다.

17 : 1 밀리 초간 기다립니다.

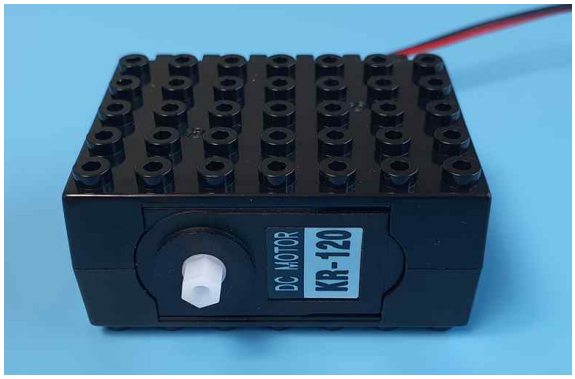
2. 업로드를 수행합니다.



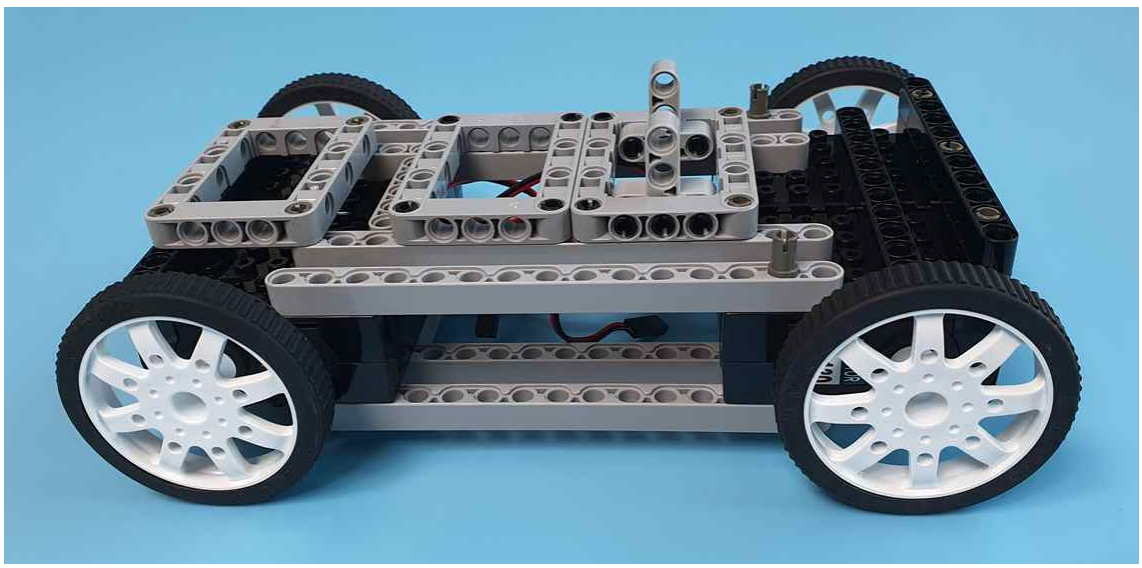
3. LED의 동작을 확인합니다. 약 1초 주기로 0~1024 단계로 LED의 밝기가 증가하는 것을 볼 수 있습니다.

04 DC 모터 제어하기

이 책에서 다루는 아두이노 인공지능 자동차는 ㈜케이로봇(www.k-robot.com)의 제품인 KR-120 레고형 DC 모터 기어박스를 사용합니다.



㈜케이로봇의 KR-120 레고형 DC 모터는 레고 블록과 호환이 되어 다음 그림과 같이 손쉽게 자동차 프레임을 구성할 수 있습니다.



㈜케이로봇은 다양한 형태의 품질좋은 DC모터와 서보모터를 생산하는 국내 회사입니다. 독자 여러분도 필요한 모터가 있다면 ㈜케이로봇(k-robot.com)을 방문해 보시기 바랍니다.


MTS-D450SB


KR300MG (세레이션)
300rpm (2.4kg.fcm)


KR300MG (육각)
300rpm (2.4kgf.cm)

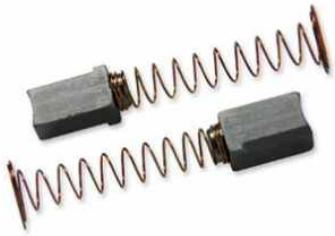

KR120
120rpm (2.2kgf.cm)


KRS-A110SB
Analog Servo

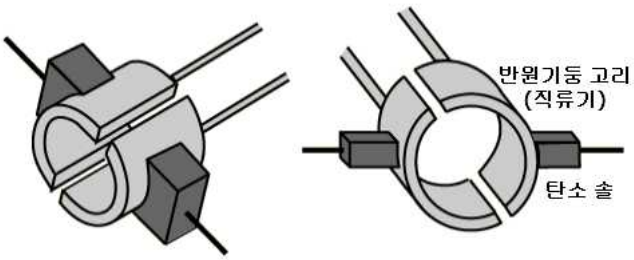
DC 모터 기어박스에는 일반적으로 다음과 같은 형태의 DC 모터가 들어가 있습니다.



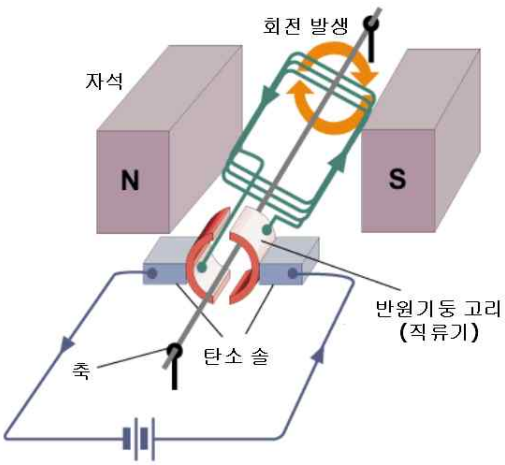
DC 모터는 모터를 구동시키기 위해 다음과 같은 형태의 탄소 브러시가 사용됩니다.



이 브러시는 다음과 같은 형태로 직류기(commutator)를 통해 코일과 연결됩니다.

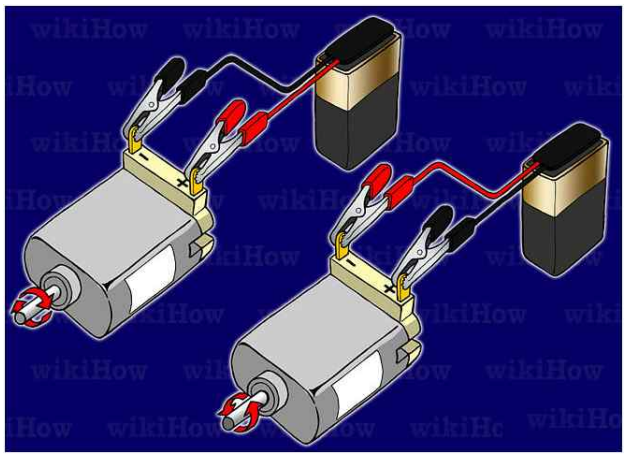


즉, 다음 그림과 같이 전지로부터의 전류가 탄소 솔과 직류기를 통해 코일로 전류가 흐르면서 회전을 하게 됩니다.

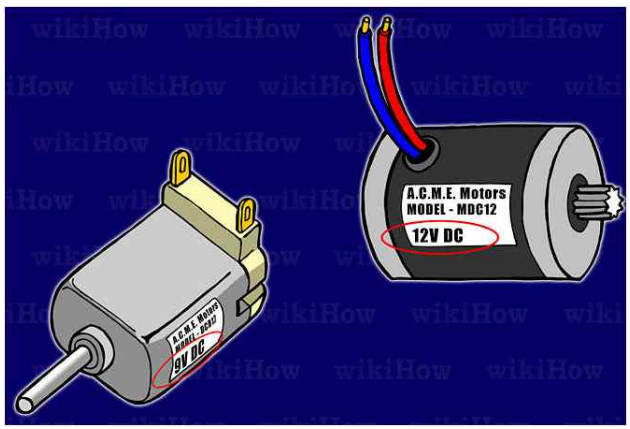


DC 모터는 두 개의 핀으로 구성되며, 극성은 없습니다. 연결 방식에 따라 모터의 회전 방향

만 바뀌게 됩니다.

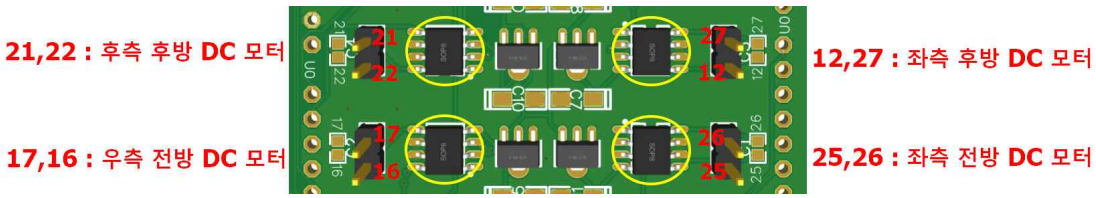


DC 모터는 일반적으로 정격 전압에 따라 출력이 달라집니다. 아래 그림에서는 각각 9V와 12V 정격 전압을 사용하는 DC 모터를 나타내고 있습니다.



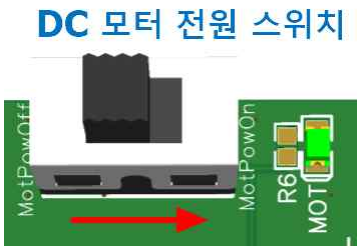
5V(또는 3.3V)의 낮은 전압을 사용하는 아두이노로 9V나 12V의 높은 전압을 사용하는 모터를 제어하기 위해서는 추가적인 모터 드라이버가 필요합니다. 모터 드라이버의 주된 역할은 제어를 위한 아두이노의 5V(또는 3.3V) 전원과 모터의 동력을 만드는 데 필요한 9V나 12V의 전원을 분리하는 것입니다. 즉, 아두이노를 모터 전원으로부터 분리해 보호하는 역할을 합니다.

ESP32 Arduino AI 쉴드 상에는 다음과 같이 4개의 모터 드라이버가 장착되어 있습니다.



모터 드라이버는 총 4개의 모터를 개별적으로 제어할 수 있습니다. 좌측 전방과 후방의 모터는 아두이노 ESP32의 25, 26, 12, 17 번핀으로 제어합니다. 우측 전방과 후방의 모터는 아두

이노 ESP32의 17, 16, 21, 22 번핀으로 제어합니다. 모터는 기본적으로 배터리를 통해 구동되어야 합니다. 따라서 모터를 구동시키기 위해서는 배터리를 장착한 후, 다음 스위치를 [모터 켜기] 위치로 이동시켜야 합니다.



01 DC 모터 제어해 보기

여기서는 4개의 DC 모터에 대해 전진, 후진, 정지를 수행해 봅니다.

왼쪽 앞 바퀴 전후진 테스트 코딩하기

먼저 왼쪽 앞 바퀴에 대한 전후진 테스트를 수행하도록 합니다.

1. 다음과 같이 예제를 작성합니다.

241.ino

```
01 const int dcMotors[] = {25,26};
02 const int forward[] = {HIGH,LOW};
03 const int backward[] = {LOW,HIGH};
04 const int stop[] = {LOW,LOW};
05
06 void setup() {
07       for(int i=0;i<sizeof(dcMotors)/sizeof(dcMotors[0]);i++)
08                pinMode(dcMotors[i], OUTPUT);
09
10       for(int cnt=0;cnt<5;cnt++) {
11                for(int i=0;i<sizeof(dcMotors)/sizeof(dcMotors[0]);i++)
12                         digitalWrite(dcMotors[i], forward[i%2]);
13
14                delay(500);
15
16                for(int i=0;i<sizeof(dcMotors)/sizeof(dcMotors[0]);i++)
17                         digitalWrite(dcMotors[i], backward[i%2]);
18
```

```
19                delay(500);
20        }
21
22        for(int i=0;i<sizeof(dcMotors)/sizeof(dcMotors[0]);i++)
23                digitalWrite(dcMotors[i], stop[i%2]);
24 }
25
26 void loop() {
27
28 }
```

01 : dcMotors 정수 상수 배열을 선언한 후, 25, 26번 핀으로 초기화합니다. 25, 26번 핀은 왼쪽 앞 바퀴를 제어하는 핀입니다.
02 : forward 정수 상수 배열을 선언한 후, LOW, HIGH로 초기화합니다. 25번 핀을 LOW로, 26번 핀을 HIGH로 설정하면 왼쪽 바퀴가 전진합니다.
03 : backward 정수 상수 배열을 선언한 후, HIGH, LOW로 초기화합니다. 25번 핀을 HIGH로, 26번 핀을 LOW로 설정하면 왼쪽 바퀴가 후진합니다.
04 : stop 정수 상수 배열을 선언한 후, LOW, LOW로 초기화합니다. 25번 핀을 LOW로, 26번 핀을 LOW로 설정하면 왼쪽 바퀴가 정지합니다.
07~08 : dcMotors의 각 핀에 대해 pinMode 함수를 호출하여 출력으로 설정합니다. sizeof(dcMotors)는 dcMotors 배열의 전체 크기를 나타내며, sizeof(dcMotors[0])은 dcMotors의 0번 항목의 크기를 나타내므로 sizeof(dcMotors)/sizeof(dcMotors[0])는 dcMotors 배열의 항목의 개수를 나타냅니다.
10 : 11~19줄을 5회 수행합니다.
11~12 : dcMotors의 각 핀에 대해 digitalWrite 함수를 호출하여 전진으로 설정합니다.
14 : delay 함수를 호출하여 500밀리초간 기다립니다.
16~17 : dcMotors의 각 핀에 대해 digitalWrite 함수를 호출하여 후진으로 설정합니다.
14 : delay 함수를 호출하여 500밀리초간 기다립니다.
22~23 : dcMotors의 각 핀에 대해 digitalWrite 함수를 호출하여 멈춤으로 설정합니다.

2. 업로드를 수행합니다.



3. 왼쪽 앞바퀴의 동작을 확인합니다. 왼쪽 앞바퀴가 0.5초 간격으로 전진과 후진을 번갈아가며 동작하는 것을 확인합니다. 이 동작을 5회 수행합니다.

*** 전진과 후진이 반대로 수행될 경우 핀 번호를 다음과 같이 바꿔주도록 합니다. 또는 모터 연결 단자를 반대로 연결합니다.

```
const int dcMotors[] = {26,25};
```

왼쪽 뒤 바퀴 추가하기

다음은 왼쪽 뒤 바퀴에 대한 전후진 테스트를 추가하도록 합니다.

1. 다음과 같이 예제를 수정합니다.

241_2.ino

```
01 const int dcMotors[] = {25,26,12,27};
```

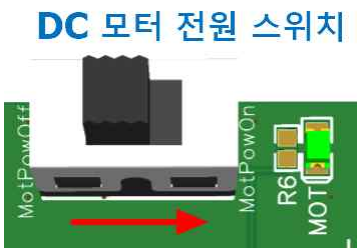
01 : dcMotors 정수 상수 배열에 12, 27을 추가합니다. 12, 27번 핀은 왼쪽 뒤 바퀴를 제어하는 핀입니다.

2. 업로드를 수행합니다.



3. 왼쪽 뒤바퀴의 동작을 확인합니다. 왼쪽 뒤바퀴가 0.5초 간격으로 전진과 후진을 번갈아가며 동작하는 것을 확인합니다. 이 동작을 5회 수행합니다.

*** DC 모터를 구동시키기 위해서는 배터리를 장착하고 다음과 같이 DC 모터 전원 스위치를 MotPowOn으로 위치시킵니다.



오른쪽 앞 바퀴 추가하기

다음은 오른쪽 앞 바퀴에 대한 전후진 테스트를 추가하도록 합니다.

1. 다음과 같이 예제를 수정합니다.

241_3.ino

```
01 const int dcMotors[] = {25,26,12,27,16,17};
```

01 : dcMotors 정수 상수 배열에 16, 17을 추가합니다. 16, 17번 핀은 오른쪽 앞 바퀴를 제어하는 핀입니다.

2. 업로드를 수행합니다.



3. 오른쪽 앞바퀴의 동작을 확인합니다. 오른쪽 앞바퀴가 0.5초 간격으로 전진과 후진을 번갈아가며 동작하는 것을 확인합니다. 이 동작을 5회 수행합니다.

오른쪽 뒤 바퀴 추가하기

다음은 오른쪽 뒤 바퀴에 대한 전후진 테스트를 추가하도록 합니다.

1. 다음과 같이 예제를 수정합니다.

241_4.ino

```
01 const int dcMotors[] = {25,26,12,27,16,17,22,21};
```

01 : dcMotors 정수 상수 배열에 22, 21을 추가합니다. 22, 21번 핀은 오른쪽 뒤 바퀴를 제어하는 핀입니다.

2. 업로드를 수행합니다.



3. 오른쪽 뒤바퀴의 동작을 확인합니다. 오른쪽 뒤바퀴가 0.5초 간격으로 전진과 후진을 번갈아가며 동작하는 것을 확인합니다. 이 동작을 5회 수행합니다.

*** 외부 전원을 사용하여 4개의 모터를 동시에 구동할 경우에 ESP32로 공급되는 전력이 순간적으로 부족하여 brownout 리셋이 발생할 수 있습니다. ESP32의 경우 내부적으로 WiFi와 BLE 하드웨어 모듈을 내장하고 있으며, 이러한 하드웨어 모듈들은 안정적인 전력 공급을 필요로 합니다. 따라서 ESP32 칩 내부에는 Brownout Detector 기능이 있어 전원 전압이 내부적으로 설정한 전압값 이하로 떨어지면 자동으로 리셋을 발생시켜 ESP32를 재부팅하게 됩니다. 외부 전원만으로 ESP32를 구동할 경우에, 특히 모터와 같이 순간적으로 구동 전력을 많이 사용하는 하드웨어와 같이 구동할 경우에 ESP32에 brownout 리셋이 발생할 가능성이 높습니다. 이 경우, ESP32로 전원을 따로 공급하거나 또는 다음과 같이 Brownout Detector를 비활성화시켜 문제를 해결합니다.

```
01 #include "soc/soc.h"
02 #include "soc/rtc_cntl_reg.h"
03
04 void setup() {
05       WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);
06       // 여러분의 코드를 작성합니다.
07 }
08
```

```
09 void loop() {
10      // 여러분의 코드를 작성합니다.
11 }
```

01 : soc/soc.h 파일을 포함합니다. soc.h 파일은 05줄에서 호출하는 함수 WRITE_PERI_REG에 대한 정보를 담고 있습니다.

02 : soc/rtc_cntl_reg.h 파일을 포함합니다. rtc_cntl_reg.h 파일은 05줄에서 사용하는 레지스터 RTC_CNTL_BROWN_OUT_REG에 대한 정보를 담고 있습니다.

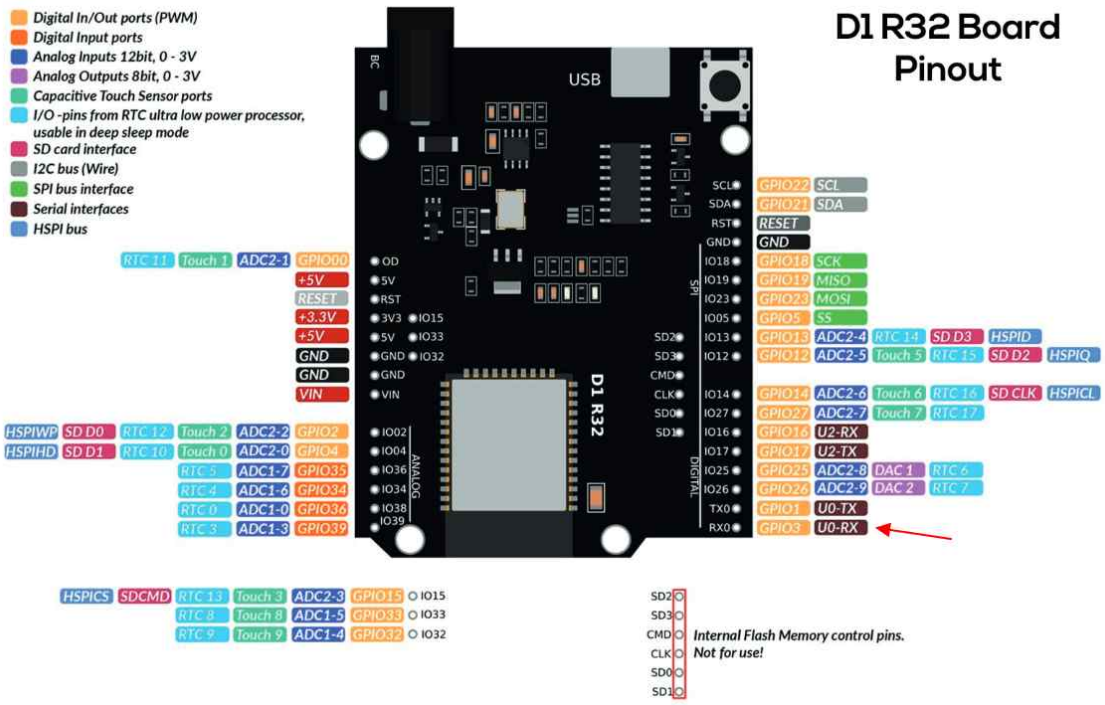
05 : WRITE_PERI_REG 함수를 호출하여 RTC_CNTL_BROWN_OUT_REG 레지스터 변수에 0을 써 줍니다. 이렇게 하면 Brownout Detector를 비활성화시켜 brownout 리셋을 막게 됩니다.

05 사용자 입력 받기

우리는 앞에서 Serial.println 함수를 통해서 아두이노에서 PC로 출력을 수행하는 방법을 살펴보았습니다. 그러면 PC로부터 아두이노로 입력을 수행하는 방법은 없을까요? 아두이노가 PC로부터 입력을 받아야 사용자가 원하는 것을 아두이노에게 시킬 수 있지 않을까요? 이 때 필요한 함수가 바로 Serial.read 함수입니다.

여기서는 시리얼 입력을 살펴봅니다. 시리얼 입력은 사용자의 입력을 받기 위해 필요하며 Serial.available 함수와 Serial.read 함수를 이용합니다. 사용자의 입력을 받기 때문에 아주 중요한 기능입니다.

아두이노 ESP32 는 USB 단자를 통해 PC로부터 메시지를 받습니다. 다음 그림에서 화살표가 가리키는 밤색 U0-RX 핀은 Serial.read 함수를 통해 PC로부터 메시지를 받는 핀입니다.



여러분은 아두이노 스케치를 통해 아두이노가 PC로부터 메시지를 받게 할 때 다음 세 함수를 주로 사용하게 됩니다.

Serial.begin(speed)
Serial.available()

Serial.read()

Serial.begin 함수는 앞에서 이미 살펴보았습니다. 여기서는 따로 설명하지 않습니다.
Serial.available은 PC로부터 도착한 데이터의 바이트 수를 돌려줍니다.
Serial.read는 PC로부터 받은 메시지의 첫 번째 바이트를 읽는 함수입니다.

01 사용자 입력 받기

여기서는 Serial.available 함수와 Serial.read 함수를 이용하여 PC를 통해 사용자로부터 문자를 입력받은 후, PC로 사용자 입력을 돌려보내 봅니다.

1. 다음과 같이 예제를 작성합니다.

251.ino

```
01 void setup() {
02       Serial.begin(115200);
03 }
04
05 void loop() {
06       if(Serial.available()>0) {
07                char userInput = Serial.read();
08                Serial.print(userInput);
09       }
10 }
```

06 : Serial.available 함수를 호출하여 시리얼을 통해 도착한 문자가 있는지 확인합니다. 도착한 문자가 있을 경우 6~9줄을 수행합니다. Serial.available 함수는 시리얼 입력 버퍼에 도착한 데이터의 개수를 주는 함수입니다.
07 : Serial.read 함수를 호출하여 키보드 입력 문자 하나를 userInput 변수로 받습니다. Serial.read 함수는 시리얼 입력 버퍼에 도착한 데이터를 한 바이트 읽어내는 함수입니다.
08 : Serial.print 함수를 호출하여 사용자로부터 전달된 문자를 출력합니다.

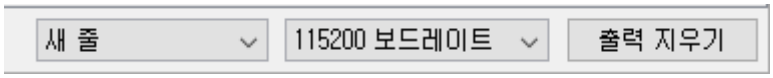
2. 업로드를 수행합니다.



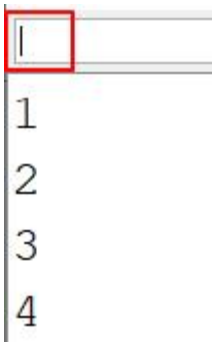
3. [시리얼 모니터] 버튼을 눌러줍니다.



4. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



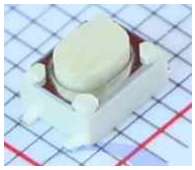
5. 시리얼 모니터 창의 빨간 박스 입력 창에 1, 2, 3, 4를 입력해 봅니다.



1,2,3,4 문자가 표시되는 것을 확인합니다.

06 버튼 상태 읽기

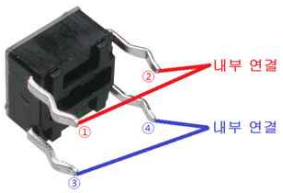
우리가 다루는 아두이노 인공지능 자동차에는 다음과 같은 푸시 버튼이 장착되어 있습니다.



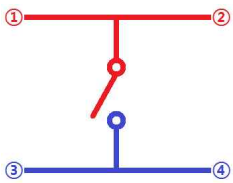
일반적인 푸시 버튼의 모양은 다음과 같습니다.



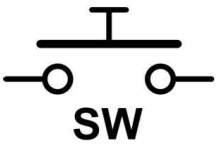
다음과 같이 두 쌍의 핀이 있으며, 각 쌍은 내부적으로 연결되어 있습니다.



내부적인 연결은 다음과 같습니다.

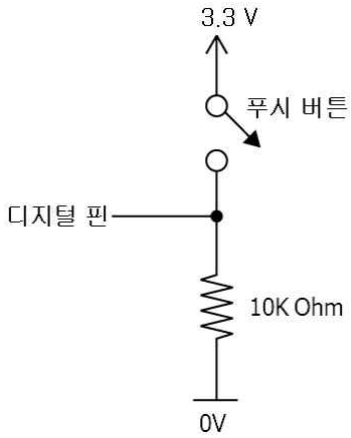


푸시 버튼을 나타내는 기호는 다음과 같고, 극성은 없습니다.



풀다운 저항 회로

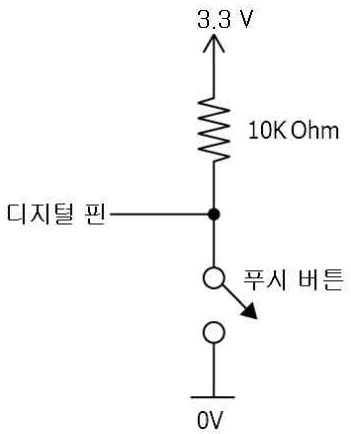
다음 회로는 풀다운 저항 회로로 버튼의 기본 상태 값이 0인 회로입니다. 풀다운은 0V로 당겨 내린다는 의미로 해석할 수 있습니다.



이 경우 디지털 핀은 버튼이 눌리지 않았을 때는 10K Ohm 저항을 통해 0V로 연결되며, 논리적으로 0 값이 입력됩니다(10K Ohm 저항 대신에 220 Ohm, 330 Ohm, 1K Ohm 저항을 사용하는 경우도 있습니다. 그러나 저항 값이 너무 낮으면 흐르는 전류량이 많아져 전력 소모가 심해집니다). 버튼을 눌렀을 경우에 디지털 핀은 3.3V로 연결되며, 논리적으로 1 값이 입력됩니다. 저항이 없는 상태에서 버튼을 누를 경우 3.3V와 0V가 직접 연결되는 단락 회로(short-circuit)가 만들어지며, 이 경우 저항이 0 Ω에 가까운 회로가 만들어집니다. 이럴 경우 옴의 법칙(I = V/R)에 의해 아주 큰 전류가 흐르게 되고, 보호회로가 없을 경우에 칩이 망가질 수 있습니다. 저항은 단락 회로를 방지하는 역할을 하게 됩니다.

풀업 저항 회로

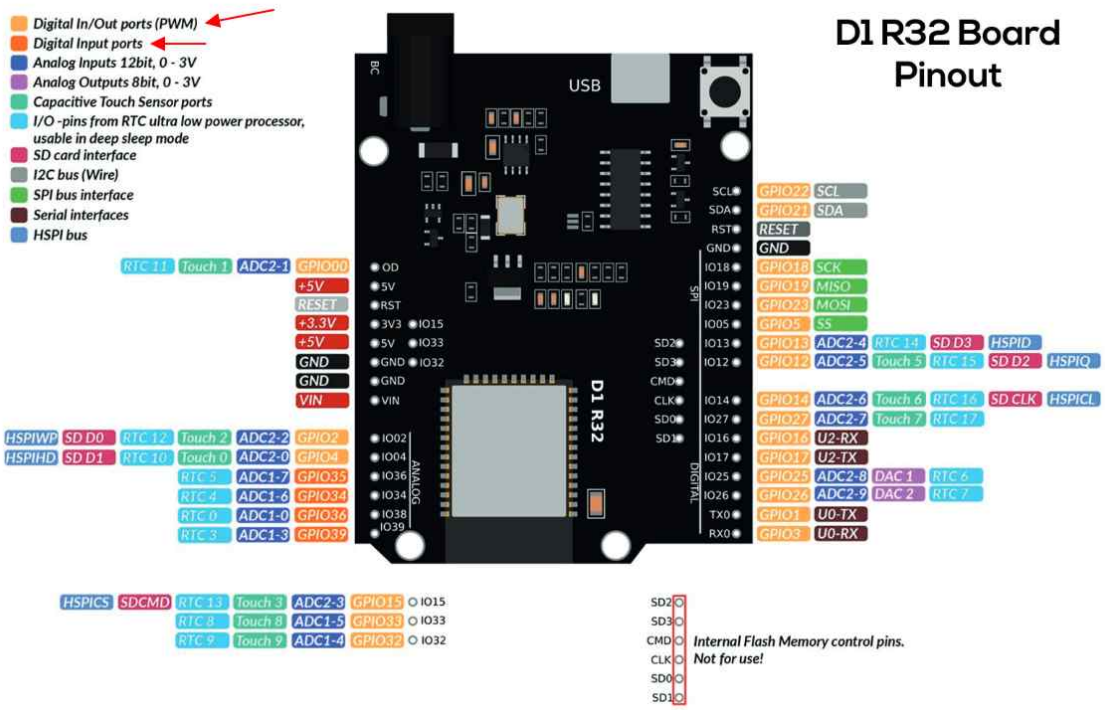
다음 회로는 풀업 저항 회로로 버튼의 기본 상태 값이 0인 회로입니다. 풀업은 3.3V로 당겨 올린다는 의미로 해석할 수 있습니다.



이 경우 디지털 핀은 버튼이 눌리지 않았을 때는 220 Ohm 저항을 통해 3.3V로 연결되며, 논리적으로 1 값이 입력됩니다. 220 Ohm 저항 대신에 330 Ohm, 1K Ohm, 10K Ohm 저항

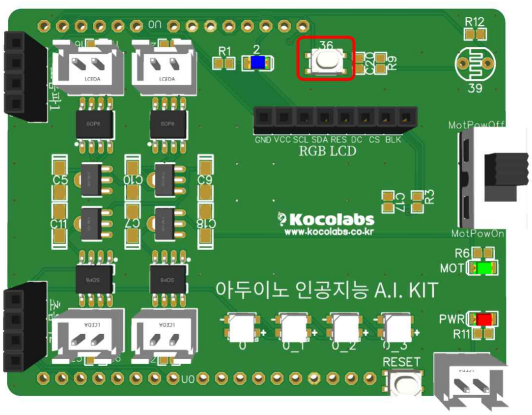
을 사용할 수도 있습니다. 버튼을 눌렀을 경우에 디지털 핀은 0V로 연결되며, 논리적으로 0 값이 입력됩니다.

다음 그림에서 화살표가 가리키는 주황색과 짙은 주황색 핀은 버튼을 연결할 수 있는 핀입니다.



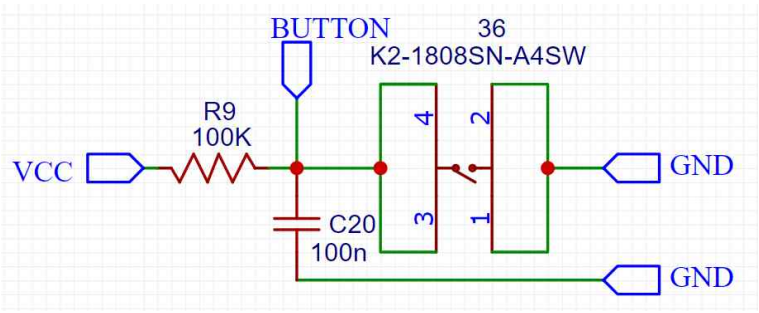
*** 특히 두 번째 화살표가 가리키는 짙은 주황색으로 표시된 핀(34,35,36,39)은 입력만 가능한 핀입니다.

ESP32 Arduino AI 쉴드 상에 다음과 같이 1개의 푸시 버튼이 장착되어 있습니다.



푸시 버튼은 36번 핀에 연결되어 있습니다.

다음은 ESP32 Arduino AI 쉴드의 버튼 회로도입니다.



36 번 핀을 통해 버튼의 상태를 읽을 수 있습니다. 버튼의 회로는 풀업 저항 회로이며 버튼을 뗀 상태에서 1값이 입력됩니다. 버튼을 누르면 0값이 입력됩니다.

여러분은 아두이노 스케치를 통해 버튼이 눌렸는데 떼졌는지 알고자 할 때 다음 두 함수를 사용해야 합니다.

```
pinMode(pin, mode)
digitalRead(pin)
```

pinMode 함수는 앞에서 살펴보았습니다.

digitalRead

digitalRead란 특정 핀의 값을 읽는 명령어입니다.

```
digitalRead(pin);
            ❶
```

❶ 읽고자 하는 핀 번호
HIGH 또는 LOW 값을 돌려줍니다.

digitalRead 함수는 할당된 핀이 VCC(=5V 또는 3.3V) 또는 GND에 연결된 상태에 따라 논리적으로 1, 0을 읽는 함수입니다. 할당된 핀이 VCC(=5V 또는 3.3V)에 연결되었을 경우엔 1이, GND에 연결되었을 때는 0을 읽게 됩니다.

01 0, 1 읽어보기

여기서는 버튼의 상태값을 읽어봅니다.

1. 다음과 같이 예제를 작성합니다.
261.ino

```
01 const int button = 36;
02
03 void setup() {
04        Serial.begin(115200);
05        pinMode(button, INPUT);
06 }
07
08 void loop() {
09        Serial.println(digitalRead(button));
10        delay(100);
11 }
```

01 : button 정수 상수를 선언한 후, 36으로 초기화합니다.

05 : pinMode 함수를 호출하여 button을 입력으로 설정합니다.

09 : digitalRead 함수를 호출하여 button 값을 읽습니다. digitalRead 함수는 버튼의 상태에 따라 0또는 1값을 내어줍니다.

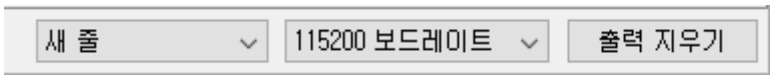
2. 업로드를 수행합니다.



3. [시리얼 모니터] 버튼을 눌러줍니다.



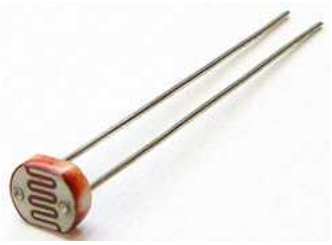
4. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



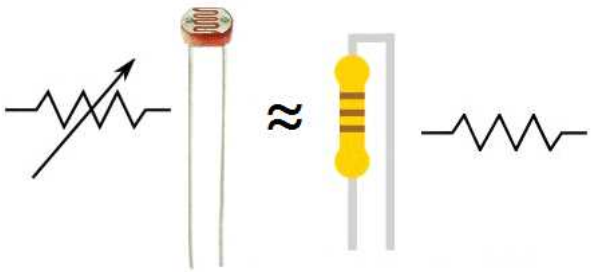
5. 결과 값을 확인합니다. 버튼을 누르고 떼며 결과 값을 확인합니다.

07 빛 센서로 밝기 감지하기

우리가 다루는 아두이노 인공지능 자동차에는 다음과 같은 빛 감지 센서가 장착되어 있습니다.

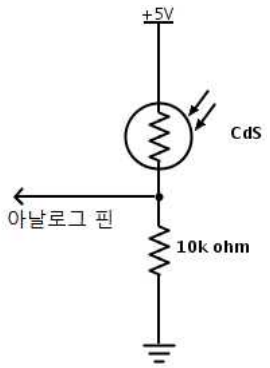


빛 감지 센서는 두 개의 핀을 갖고, 극성은 없습니다.



빛 감지 센서는 빛의 양에 따라 값이 변하는 가변저항과 같습니다.

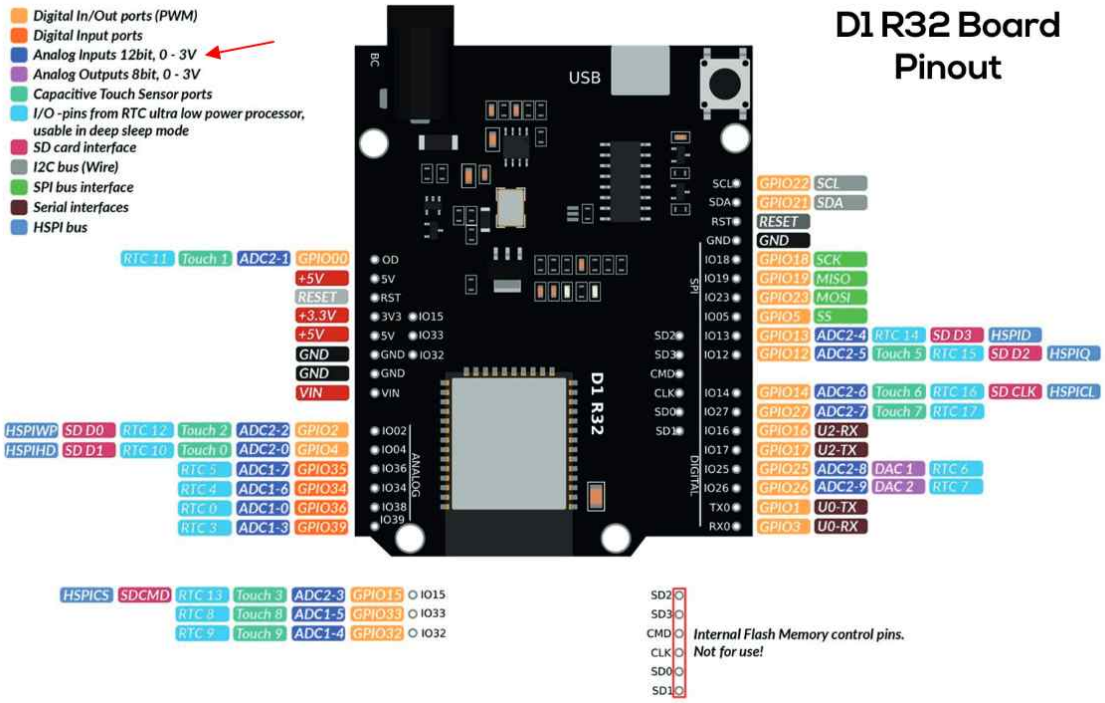
빛 감지 센서의 회로는 일반적으로 다음과 같이 구성합니다.



10K Ohm 저항과 직렬로 연결합니다. 빛 감지 센서로 입력되는 빛의 양에 따라 저항 값이 달라집니다. 빛의 양이 적을수록, 즉 어두울 수록 저항 값은 높아지고, 빛의 양이 많을 수록 저항 값은 낮아집니다. 밝을 때의 저항 값은 5~10K Ohm 정도이며, 어두울 때는 200K Ohm까

지 올라갑니다.

다음 그림에서 화살표가 가리키는 파란색 핀은 센서를 연결할 수 있는 핀입니다.



ESP32 Arduino AI 쉴드 상에서 빛센서는 아두이노 ESP32의 39 핀과 연결되어 있습니다.



여러분은 아두이노 스케치를 통해 센서값을 읽을 때 다음 함수를 사용해야 합니다.

analogRead

analogRead란 특정 아날로그 핀의 값을 읽는 명령어입니다. **analogRead(pin);** ❶ ❶ 읽고자 하는 아날로그 핀 번호 0~1023 사이의 값을 돌려줍니다.

analogRead 함수는 마이컴 내부의 ADC 모듈을 제어하여 센서 값을 읽는 역할을 합니다. 아

두이노 ESP32의 경우 아날로그 입력 핀으로 입력되는 0~3V의 전압 값을 논리적으로 0~4095 사이의 정수 값으로 바꿔줍니다. 우리는 analogRead 함수를 이용하여 ADC 모듈에 연결된 센서 값을 읽을 수 있습니다.

01 빛 센서 읽기

여기서는 빛 감지 센서를 읽어 보도록 합니다.

1. 다음과 같이 예제를 작성합니다.

271.ino

```
01 const int cds = 39;
02
03 void setup() {
04       Serial.begin(115200);
05 }
06
07 void loop() {
08       Serial.println(analogRead(cds));
09       delay(100);
10 }
```

01 : cds 정수 상수를 선언한 후, 39로 초기화합니다.
08 : analogRead 함수를 호출하여 cds 핀을 읽어 출력합니다. 아두이노 ESP32의 경우 analogRead 함수는 cds 상태에 따라 0~4095 사이의 값을 출력합니다.
09 : delay 함수를 호출하여 100밀리초를 기다립니다.

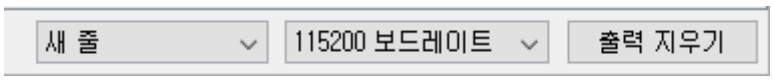
2. 업로드를 수행합니다.



3. [시리얼 모니터] 버튼을 눌러줍니다.



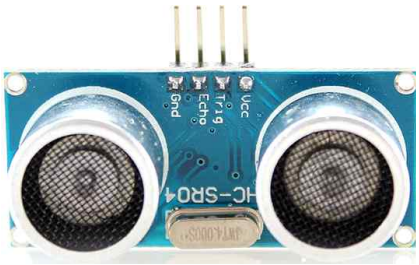
4. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



5. 결과 값을 확인합니다. 빛 센서를 손가락으로 가려가며 결과 값을 확인합니다.

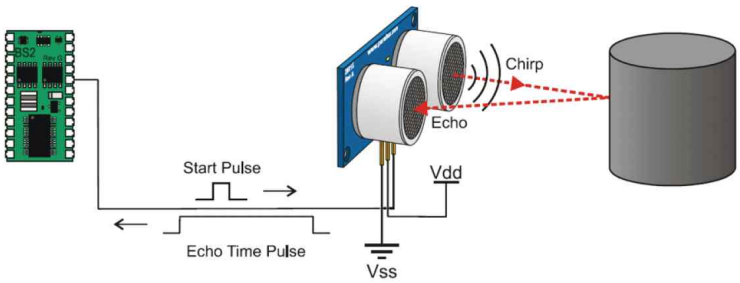
08 초음파 센서로 거리 측정하기

다음은 이 책에서 사용하는 초음파 센서입니다.



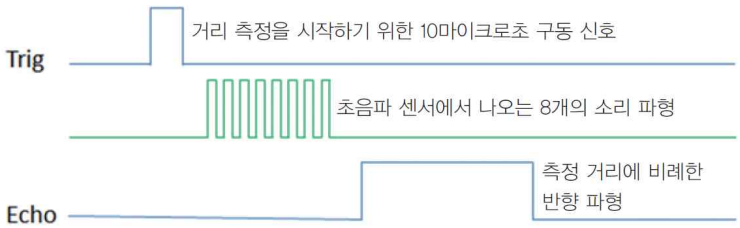
모델명은 HC-SR04로 측정 가능 거리는 2 ~ 400cm 사이가 됩니다.

초음파 거리 센서는 다음과 같은 방법으로 거리를 측정합니다.



한쪽에 있는 초음파 출력기에서 출발한 초음파가 물체에 닿아 반사되어 다른 한쪽에 있는 초음파 입력기에 닿게 되는데, 이 때, 물체의 거리에 따라 초음파가 도달하는 시간이 달라지게 됩니다. 초음파에 대한 출력은 초음파 센서에 부착된 제어기에서 거리 감지 시작을 알리는 시작 파형을 내보내면서 시작됩니다. 물체와의 거리는 거리에 따라 달라지는 반향 시간 파형으로 확인할 수 있습니다.

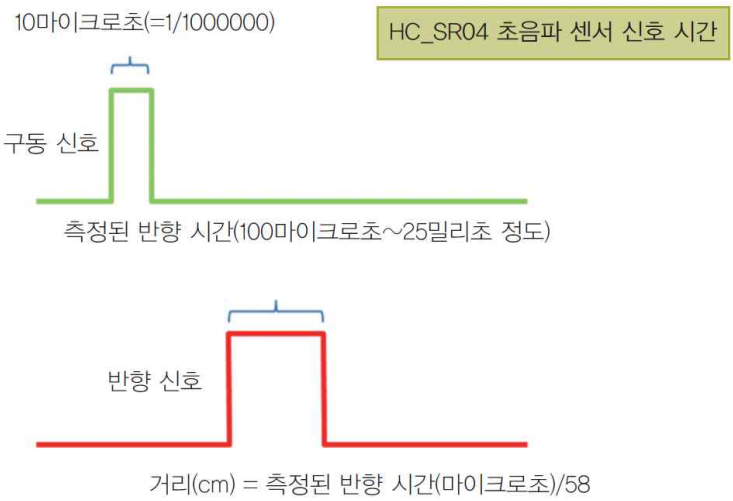
다음은 초음파 센서가 거리를 측정하기 위해 필요한 파형입니다.



Trig 파형은 거리 감지를 시작하기 위해 아두이노에서 초음파 센서로 내보내는 신호로 10 마이크로 초 동안 HIGH 신호를 줍니다. 그러면 초음파 센서는 8개의 초음파를 40kHz의 속도로 외부로 내보내게 됩니다. 초음파 센서는 곧이어 Echo 핀을 HIGH 신호로 올려 거리

측정이 시작되었다는 것을 아두이노로 알립니다. 내보낸 초음파 신호가 반향되어 센서에 도달하여 거리 측정이 완료되면 초음파 센서는 Echo 핀을 LOW 신호로 내려 거리 측정이 완료되었다는 것을 아두이노로 알립니다.

다음은 Echo 핀의 최소, 최대 파형 길이와 파형 길이에 따라 측정된 거리를 계산하는 그림입니다.



Echo 파형의 길이는 최소 100 마이크로 초에서 최대 25 밀리 초가 될 수 있습니다. 거리에 대한 계산은 cm 기준으로 마이크로 초 단위로 측정된 시간을 58로 나누면 됩니다. 예를 들어, Echo 파형의 길이가 100 마이크로 초일 경우 100/58 = 1.72 cm가 됩니다. Echo 파형의 길이가 25 밀리 초일 경우 25*1000/58 = 431.03 cm가 됩니다.

ESP32 Arduino AI 쉴드 상에는 다음과 같이 4개의 초음파 센서 단자가 장착되어 있습니다.



우측 초음파 센서는 13, 34 번핀으로 제어합니다. 13번핀은 초음파 센서의 trigger 핀, 34번 핀은 초음파 센서의 echo 핀에 연결하도록 합니다. 좌측 초음파 센서는 14, 35 번핀으로 제어합니다. 14번핀은 초음파 센서의 trigger 핀, 35번 핀은 초음파 센서의 echo 핀에 연결하도록 합니다.

01 초음파 센서 읽고 거리 계산하기

여기서는 초음파 센서를 읽어 거리를 계산해 봅니다.

우측 초음파 센서 테스트 코딩하기

먼저 우측 초음파 센서에 대한 거리 측정을 수행해 봅니다.

1. 다음과 같이 예제를 작성합니다.

281.ino

```
01 const int trigPin = 13;
02 const int echoPin = 34;
03
04 void setup() {
05
06       Serial.begin(115200);
07
08       pinMode(trigPin, OUTPUT);
09       pinMode(echoPin, INPUT);
10
11 }
12
13 void loop() {
14
15       digitalWrite(trigPin, LOW);
16       delayMicroseconds(2);
17
18       digitalWrite(trigPin, HIGH);
19       delayMicroseconds(10);
20       digitalWrite(trigPin, LOW);
21
22       long duration = pulseIn(echoPin, HIGH);
23
24       int distance= duration / 58;
25
26       Serial.print("Distance: ");
27       Serial.println(distance);
28
29 }
```

01 : trigPin 정수 상수를 선언한 후, 13번 핀으로 초기화합니다.
02 : echoPin 정수 상수를 선언한 후, 34번 핀으로 초기화합니다.
08 : pinMode 함수를 호출하여 trigPin을 출력으로 설정합니다.

09 : pinMode 함수를 호출하여 echoPin을 입력으로 설정합니다.
15~16 : trigPin을 LOW로 설정합니다.
18~20 : trigPin을 10마이크로초 동안 HIGH로 설정합니다.
22 : pulseIn 함수를 호출하여 echoPin의 HIGH 구간을 측정한 후, duration 변수에 저장합니다.
24 : 거리를 계산합니다.
26~27 : 거리를 출력합니다.

2. 업로드를 수행합니다.



3. 초음파 센서를 적당한 거리에서 손으로 가려가며 거리 값이 변하는 것을 확인합니다.

좌측 초음파 센서 테스트 코딩하기

다음은 좌측 초음파 센서에 대한 거리 측정을 수행해 봅니다.

1. 다음과 같이 예제를 수정합니다.

282.ino

```
01 const int trigPin = 14;
02 const int echoPin = 35;
```

01 : trigPin을 14로 변경합니다.
02 : echoPin을 35로 변경합니다.

2. 업로드를 수행합니다.



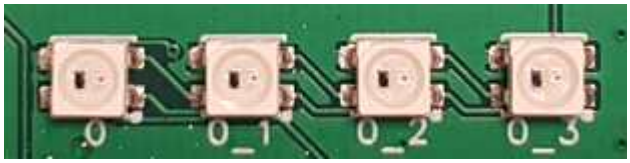
3. 초음파 센서를 적당한 거리에서 손으로 가려가며 거리 값이 변하는 것을 확인합니다.

09 RGB 네오픽셀 LED 제어하기

다음은 RGB 네오픽셀 LED입니다. RGB 네오픽셀 LED는 WS2812 LED라고도 합니다. RGB 네오픽셀 LED는 R,G,B 각각에 대해 8비트 색깔을 낼 수 있으며, 전체 24비트로 색깔을 표현할 수 있습니다. 네오픽셀은 그 모양을 다양하게 구성하여 프로젝트에 활용하기 좋습니다. 그리고 하나의 제어 핀을 통해 직렬로 몇 개든지 연결이 가능합니다.



ESP32 Arduino AI 쉴드 상에는 다음과 같이 4개의 RGB 네오픽셀이 장착되어 있습니다. 4개의 RGB 네오픽셀은 아두이노 ESP32의 0 번 핀에 직렬로 연결되어 있습니다. 여기서는 0 번 핀에 직렬로 연결된 4개의 RGB 네오픽셀을 제어해 봅니다.



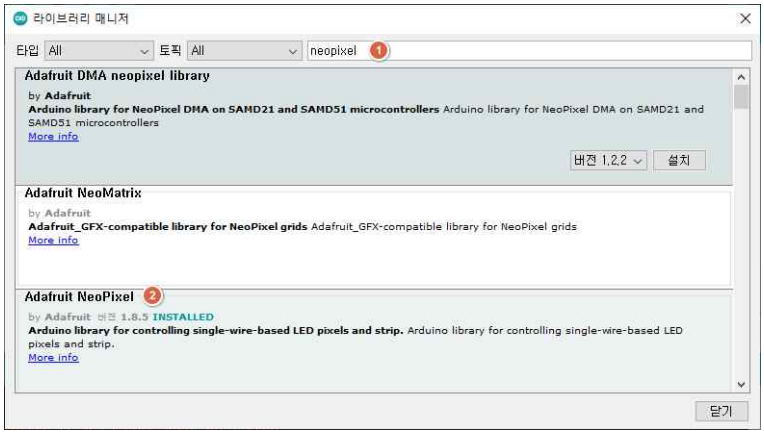
01 NeoPixel 라이브러리 설치하기

먼저 NeoPixel 라이브러리를 아두이노 소프트웨어에 설치합니다.

1. [스케치]--[라이브러리 포함하기]--[라이브러리 관리...] 메뉴를 선택합니다.



2. neopixel을 검색한 후, [Adafruit NeoPixel] 라이브러리를 설치합니다.



02 RGB 네오픽셀 LED 제어해 보기

여기서는 NeoPixel 라이브러리를 이용하여 RGB 네오픽셀 LED를 제어해 봅니다.

LED 하나 켜고 꺼 보기

먼저 첫 번째 LED를 켜고 꺼봅니다.

1. 다음과 같이 예제를 작성합니다.

292.ino

```
01 #include <Adafruit_NeoPixel.h>
02
03 #define PIN        0
04 #define NUMPIXELS 4
05
06 Adafruit_NeoPixel pixels(NUMPIXELS, PIN);
07
08 void setup() {
09       pixels.begin();
10       pixels.clear();
```

```
11        pixels.setBrightness(255/100.0*20);
12
13        for(int cnt=0;cnt<5;cnt++) {
14                  pixels.setPixelColor(0, pixels.Color(255, 0, 0));
15                  pixels.show();
16                  delay(500);
17
18                  pixels.setPixelColor(0, pixels.Color(0, 255, 0));
19                  pixels.show();
20                  delay(500);
21        }
22
23        pixels.setPixelColor(0, pixels.Color(0, 0, 0));
24        pixels.show();
25 }
26
27 void loop() {
28
29 }
```

01 : Adafruit_NeoPixel.h 파일을 포함합니다.
03 : 첫 번째 네오픽셀이 연결된 핀 번호를 정의합니다.
04 : 네오픽셀 LED의 개수를 정의합니다.
06 : Adafruit_NeoPixel 객체인 pixels를 생성합니다. 첫 번째 인자는 네오픽셀 LED의 개수, 두 번째 인자는 네오픽셀이 연결된 핀 번호입니다.
09 : begin 함수를 호출하여 네오픽셀을 초기화합니다.
10 : clear 함수를 호출하여 모든 네오픽셀을 끕니다.
11 : setBrightness 함수를 호출하여 네오픽셀을 밝기를 20%로 설정합니다. 네오픽셀의 밝기는 최소 0에서 최대 255단계로 조절할 수 있습니다.
13 : 14~20줄을 5회 수행합니다.
14 : setPixelColor 함수를 호출하여 첫 번째 네오픽셀((0번 네오픽셀)의 색깔을 빨간색으로 설정합니다. pixels.Color 함수의 인자는 차례대로 R, G, B를 나타냅니다. R, G, B 각각의 값은 0~255 범위의 값을 줄 수 있습니다.
15 : show 함수를 호출하여 설정된 네오픽셀의 색깔을 실제로 LED에 표시합니다.
16 : delay 함수를 호출하여 500 밀리 초 동안 기다립니다.
18 : setPixelColor 함수를 호출하여 첫 번째 네오픽셀((0번 네오픽셀)의 색깔을 초록색으로 설정합니다.
19 : show 함수를 호출하여 설정된 네오픽셀의 색깔을 실제로 LED에 표시합니다.
20 : delay 함수를 호출하여 500 밀리 초 동안 기다립니다.
23 : setPixelColor 함수를 호출하여 첫 번째 네오픽셀((0번 네오픽셀)의 색깔을 검정색으로 설정합니다. 이렇게 하면 네오픽셀이 꺼진 상태로 설정합니다.
24 : show 함수를 호출하여 설정된 네오픽셀의 색깔을 실제로 LED에 표시합니다.

2. 업로드를 수행합니다.



3. 네오픽셀을 동작을 확인합니다. 0 번째 RGB 네오픽셀 LED가 0.5초 간격으로 빨간색과 초록색을 번갈아 바뀌는 것을 확인합니다. 이 동작을 5회 수행합니다.

전체 LED 켜고 꺼 보기

이번엔 전체 LED를 켜고 꺼봅니다.

1. 다음과 같이 예제를 수정합니다.
292_2.ino

```
01 #include <Adafruit_NeoPixel.h>
02
03 #define PIN         0
04 #define NUMPIXELS 4
05
06 Adafruit_NeoPixel pixels(NUMPIXELS, PIN);
07
08 void setup() {
09       pixels.begin();
10       pixels.clear();
11       pixels.setBrightness(255/100.0*20);
12
13       for(int cnt=0;cnt<5;cnt++) {
14                pixels.fill(pixels.Color(255,0,0));
15                pixels.show();
16                delay(500);
17
18                pixels.fill(pixels.Color(0, 0, 255));
19                pixels.show();
20                delay(500);
21       }
22       pixels.fill(pixels.Color(0, 0, 0));
23       pixels.show();
24 }
25
26 void loop() {
```

```
27
28 }
```

14, 18, 22 : setPixelColor 함수를 fill 함수로 변경합니다. fill 함수는 전체 LED의 색깔을 설정하는 함수입니다. 18줄에서는 전체 LED를 파란색으로 설정합니다.

2. 업로드를 수행합니다.



3. 네오픽셀을 동작을 확인합니다. 전체 RGB 네오픽셀 LED가 0.5초 간격으로 빨간색과 파란색을 번갈아 바뀌는 것을 확인합니다. 이 동작을 5회 수행합니다.

전체 LED 여러 색깔 켜고 꺼 보기

이번엔 전체 LED에 대해 여러 색깔로 켜고 꺼봅니다.

1. 다음과 같이 예제를 수정합니다.
292_3.ino

```
01 #include <Adafruit_NeoPixel.h>
02
03 #define PIN        0
04 #define NUMPIXELS 4
05
06 Adafruit_NeoPixel pixels(NUMPIXELS, PIN);
07
08 uint32_t BLACK = pixels.Color(0, 0, 0);
09 uint32_t RED = pixels.Color(255, 0, 0);
10 uint32_t YELLOW = pixels.Color(255, 150, 0);
11 uint32_t GREEN = pixels.Color(0, 255, 0);
12 uint32_t CYAN = pixels.Color(0, 255, 255);
13 uint32_t BLUE = pixels.Color(0, 0, 255);
14 uint32_t PURPLE = pixels.Color(180, 0, 255);
15 uint32_t WHITE = pixels.Color(255, 255, 255);
16 uint32_t COLORS[] = {BLACK, RED, YELLOW, GREEN, CYAN, BLUE, PURPLE, WHITE};
17
18 void setup() {
19         pixels.begin();
20         pixels.clear();
21         pixels.setBrightness(255/100.0*20);
22
23         for(int cnt=0;cnt<5;cnt++) {
24                 for(int n=0;n<sizeof(COLORS)/sizeof(COLORS[0]);n++) {
25                         pixels.fill(COLORS[n]);
26                         pixels.show();
27                         delay(300);
```

```
28                      }
29          }
30
31          pixels.fill(BLACK);
32          pixels.show();
33 }
34
35 void loop() {
36
37 }
```

08~15 : 네오픽셀 LED에 사용할 색깔을 정의합니다.

16 : 08~15줄에서 정의한 색깔들의 배열을 선언합니다.

23 : 24~28줄을 5회 수행합니다.

24 : COLORS 배열에 있는 색깔의 개수만큼 25~27줄을 수행합니다. sizeof(COLORS)는 COLORS 배열 전체의 크기를 의미하며, sizeof(COLORS[0])는 COLORS[0] 번 항목의 크기를 의미합니다. 배열 전체의 크기를 항목 한 개의 크기로 나누면 배열 항목의 개수가 됩니다.

25 : COLORS 배열의 n 번째 항목의 색깔로 네오픽셀 LED 전체를 설정합니다.

26 : show 함수를 호출하여 설정된 네오픽셀의 색깔을 실제로 LED에 표시합니다.

27 : delay 함수를 호출하여 300 밀리 초 동안 기다립니다.

2. 업로드를 수행합니다.



3. 네오픽셀을 동작을 확인합니다. 전체 RGB 네오픽셀 LED가 COLORS 배열에 있는 색깔 순서대로 켜졌다 꺼졌다 하는 것을 확인합니다. 이 동작을 5회 수행합니다.

무지개 색깔 내보기

이번엔 전체 LED에 대해 무지개 색깔로 켜고 꺼봅니다.

1. 다음과 같이 예제를 수정합니다.

292_4.ino

```
01 #include <Adafruit_NeoPixel.h>
02
03 #define PIN        0
04 #define NUMPIXELS 4
05
06 Adafruit_NeoPixel pixels(NUMPIXELS, PIN, NEO_GRB + NEO_KHZ800);
07
08 uint32_t BLACK = pixels.Color(0, 0, 0);
09 uint32_t RED = pixels.Color(255, 0, 0);
10 uint32_t YELLOW = pixels.Color(255, 150, 0);
11 uint32_t GREEN = pixels.Color(0, 255, 0);
```

```
12 uint32_t CYAN = pixels.Color(0, 255, 255);
13 uint32_t BLUE = pixels.Color(0, 0, 255);
14 uint32_t PURPLE = pixels.Color(180, 0, 255);
15 uint32_t WHITE = pixels.Color(255, 255, 255);
16 uint32_t COLORS[] = {BLACK, RED, YELLOW, GREEN, CYAN, BLUE, PURPLE, WHITE};
17
18 uint32_t wheel(int pos) {
19         //Input a value 0 to 255 to get a color value.
20         //The colours are a transition r - g - b - back to r.
21         if(pos < 0 || pos > 255) {
22                 return pixels.Color(0, 0, 0);
23         } else if(pos < 85) {
24                 return pixels.Color(255 - pos * 3, pos * 3, 0);
25         } else if(pos < 170) {
26                 pos -= 85;
27                 return pixels.Color(0, 255 - pos * 3, pos * 3);
28         } else {
29                 pos -= 170;
30                 return pixels.Color(pos * 3, 0, 255 - pos * 3);
31         }
32 }
33
34 void rainbow_cycle(int wait) {
35         int NUM_LEDS = pixels.numPixels();
36
37         for(int j=0;j<255;j++) {
38                 for(int i=0;i<NUM_LEDS;i++) {
39                         int rc_index = (i*256.0/NUM_LEDS)+j;
40                         pixels.setPixelColor(i, wheel(rc_index&255));
41                 }
42                 pixels.show();
43                 delay(wait);
44         }
45
46         pixels.clear();
47         pixels.show();
48 }
49
50 void setup() {
51         pixels.begin();
52         pixels.clear();
53         pixels.setBrightness(255/100.0*20);
54
55         rainbow_cycle(10);
56 }
57
58 void loop() {
59
60 }
```

18~32 : wheel 함수를 정의합니다. wheel 함수는 인자로 0~255 사이의 값을 받아 색깔 값을 내어줍니다. 색깔은 빨간색, 초록색, 파란색 다시 빨간색과 같은 단계로 변합니다. 함수에 대

한 자세한 설명은 하지 않습니다.
34~44 : rainbow_cycle 함수를 정의합니다.
35 : numPixels 함수를 호출하여 LED의 개수를 받습니다.

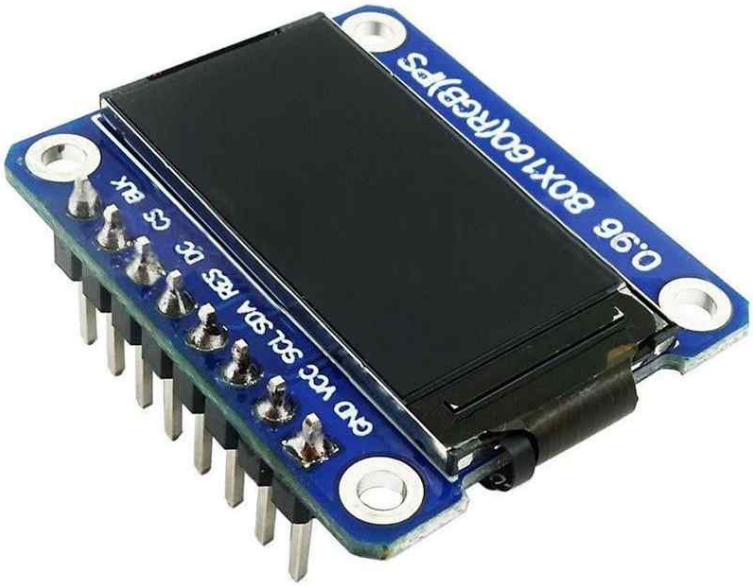
2. 업로드를 수행합니다.



3. 네오픽셀을 동작을 확인합니다. 각각의 LED가 여러 색깔로 예쁘게 표시되는 것을 확인합니다.

10 RGB LCD 제어하기

다음은 0.96 인치 RGB IPS LCD 디스플레이입니다. 이 LCD를 제어하기 위한 TFT 드라이버는 ST7735입니다. 작은 크기의 LCD이지만 160x80의 해상도와 16비트 RGB 컬러 픽셀을 가지고 있습니다. SPI 인터페이스를 이용하여 아두이노 ESP32와 연결되며, 전압은 3.3V를 사용합니다.



ESP32 Arduino AI 쉴드 상에는 다음과 같이 장착됩니다. RGB LCD는 IO18,23,4,19,5번 핀을 통해 아두이노 ESP32 에 연결됩니다.



RGB LCD 핀 설명

다음은 RGB LCD 핀 설명입니다.

핀	설명	라즈베리파이 피코 핀
GND	전원 기준 핀	GND
VCC	전원 공급 핀, 3.3V	VCC
SCL	SPI 클럭 핀	IO18
SDA	SPI 데이터 핀	IO23
RES	디스플레이 리셋 핀	IO4
DC	SPI 데이터 핀/ command 선택 핀	IO19

CS	SPI 칩 선택 핀, active low	IO5
BLK	LCD 백라이트 제어 핀	VCC

ESP32 Arduino AI 쉴드에서 BLK 핀의 경우 VCC와 연결되어 LCD 백라이트가 항상 켜진 상태를 유지하게 됩니다.

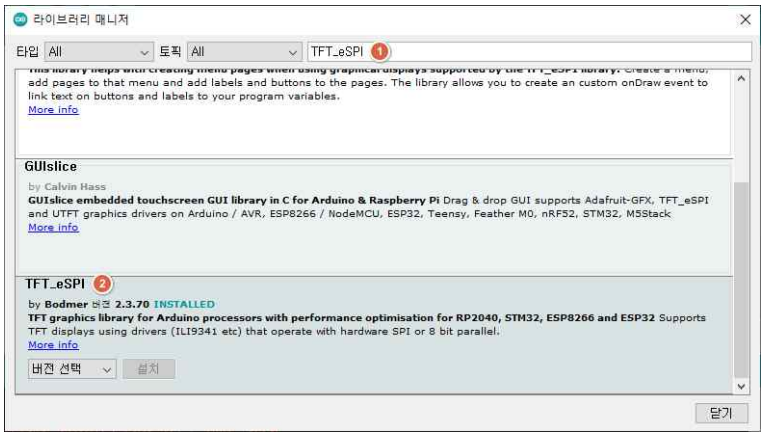
01 ST7735 라이브러리 설치하기

먼저 ST7735 라이브러리를 아두이노 소프트웨어에 설치합니다.

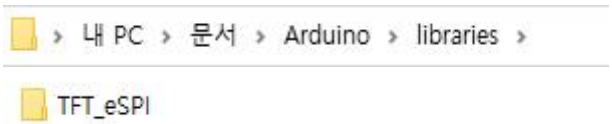
1. [스케치]--[라이브러리 포함하기]--[라이브러리 관리...] 메뉴를 선택합니다.

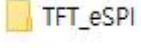


2. TFT_eSPI를 검색한 후, [TFT_eSPI] 라이브러리를 설치합니다.



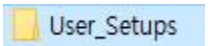
3. 다음 디렉터리로 이동합니다. TFT_eSPI 디렉터리를 확인합니다.





4. TFT_eSPI 디렉터리로 이동합니다. User_Setups 디렉터리를 확인합니다.

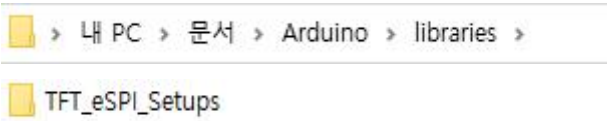




5. User_Setups 디렉터리 아래에서 다음 파일을 찾습니다.



6. 다음 디렉터리에 TFT_eSPI_Setups 디렉터리를 새로 생성합니다.



7. TFT_eSPI_Setups 디렉터리 아래로 Setup43_ST7735.h 파일을 복사한 후, 다음과 같이 이름을 Setup43_ST7735_ESP32_80x160.h으로 변경합니다.



8. 다음과 같이 파일의 내용을 수정한 후, 저장해 줍니다.

Setup43_ST7735_ESP32_80x160.h

```
// Setup for ESP32 and ST7735 80 x 160 TFT

// See SetupX_Template.h for all options available

#define ST7735_DRIVER


#define TFT_WIDTH  80
#define TFT_HEIGHT 160


#define ST7735_GREENTAB160x80

// For ST7735, ST7789 and ILI9341 ONLY, define the colour order IF the blue and red are swapped on your display
// Try ONE option at a time to find the correct colour order for your display

//  #define TFT_RGB_ORDER TFT_RGB  // Colour order Red-Green-Blue
//  #define TFT_RGB_ORDER TFT_BGR  // Colour order Blue-Green-Red

#ifdef ESP32
#define TFT_MISO 19
#define TFT_MOSI 23
#define TFT_SCLK 18
#define TFT_CS    5//15  // Chip select control pin
#define TFT_DC    19//2  // Data Command control pin
#define TFT_RST   4  // Reset pin (could connect to RST pin)
//#define TFT_RST  -1  // Set TFT_RST to -1 if display RESET is connected to ESP32 board RST
#else
// Display GND       to NodeMCU pin GND (0V)
// Display VCC       to NodeMCU 5V or 3.3V
// Display SCK       to NodeMCU pin D5
// Display SDI/MOSI  to NodeMCU pin D7
// Display BLK       to NodeMCU pin VIN
#define TFT_CS   PIN_D8  // Chip select control pin D8
#define TFT_DC   PIN_D3  // Data Command control pin
#define TFT_RST  PIN_D4  // Reset pin (could connect to NodeMCU RST, see next line)
#endif


#define LOAD_GLCD   // Font 1. Original Adafruit 8 pixel font needs ~1820 bytes in FLASH
#define LOAD_FONT2  // Font 2. Small 16 pixel high font, needs ~3534 bytes in FLASH, 96 characters
#define LOAD_FONT4  // Font 4. Medium 26 pixel high font, needs ~5848 bytes in FLASH, 96 characters
#define LOAD_FONT6  // Font 6. Large 48 pixel font, needs ~2666 bytes in FLASH, only characters 1234567890:-.apm
#define LOAD_FONT7  // Font 7. 7 segment 48 pixel font, needs ~2438 bytes in FLASH, only characters 1234567890:.
#define LOAD_FONT8  // Font 8. Large 75 pixel font needs ~3256 bytes in FLASH, only characters 1234567890:-.
//#define LOAD_FONT8N // Font 8. Alternative to Font 8 above, slightly narrower, so 3 digits fit a 160 pixel TFT
#define LOAD_GFXFF  // FreeFonts. Include access to the 48 Adafruit_GFX free fonts FF1 to FF48 and custom fonts

#define SMOOTH_FONT


//#define SPI_FREQUENCY  20000000
  #define SPI_FREQUENCY  27000000 // Actually sets it to 26.67MHz = 80/3

#define TFT_RGB_ORDER TFT_BGR
```

24 : 15를 5로 변경합니다.

25 : 2를 19로 변경합니다.
54 : TFT_RGB_ORDER를 TFT_BGR로 정의합니다.

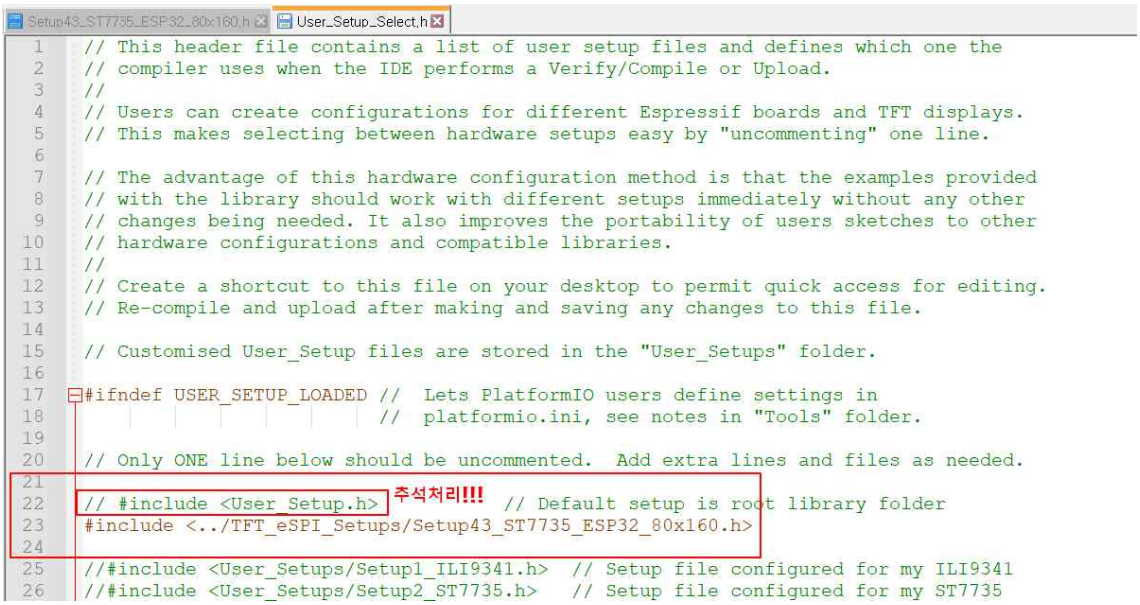
9. TFT_eSPI 디렉터리로 이동합니다.



10. 다음 파일을 찾습니다.



11. 다음과 같이 파일을 수정한 후, 저장해 줍니다.



22 : 주석 처리합니다. 이 부분을 주석처리하지 않으면 LCD가 비정상적으로 동작합니다.
23 : Setup43_ST7735_ESP32_80x160.h 파일을 포함합니다.

02 RGB LCD 제어해 보기

여기서는 TFT_eSPI 라이브러리를 이용하여 RGB LCD를 제어해 봅니다. TFT_eSPI 라이브러리에는 다음과 같이 몇 가지 색깔이 정의되어 있습니다.

```
251  /*****************************************************************************
252  **                        Section 6: Colour enumeration                     **
253  *****************************************************************************/
254  // Default color definitions
255  #define TFT_BLACK       0x0000      /*   0,   0,   0 */
256  #define TFT_NAVY        0x000F      /*   0,   0, 128 */
257  #define TFT_DARKGREEN   0x03E0      /*   0, 128,   0 */
258  #define TFT_DARKCYAN    0x03EF      /*   0, 128, 128 */
259  #define TFT_MAROON      0x7800      /* 128,   0,   0 */
260  #define TFT_PURPLE      0x780F      /* 128,   0, 128 */
261  #define TFT_OLIVE       0x7BE0      /* 128, 128,   0 */
262  #define TFT_LIGHTGREY   0xD69A      /* 211, 211, 211 */
263  #define TFT_DARKGREY    0x7BEF      /* 128, 128, 128 */
264  #define TFT_BLUE        0x001F      /*   0,   0, 255 */
265  #define TFT_GREEN       0x07E0      /*   0, 255,   0 */
266  #define TFT_CYAN        0x07FF      /*   0, 255, 255 */
267  #define TFT_RED         0xF800      /* 255,   0,   0 */
268  #define TFT_MAGENTA     0xF81F      /* 255,   0, 255 */
269  #define TFT_YELLOW      0xFFE0      /* 255, 255,   0 */
270  #define TFT_WHITE       0xFFFF      /* 255, 255, 255 */
271  #define TFT_ORANGE      0xFDA0      /* 255, 180,   0 */
272  #define TFT_GREENYELLOW 0xB7E0      /* 180, 255,   0 */
273  #define TFT_PINK        0xFE19      /* 255, 192, 203 */ //Lighter pink, was 0xFC9F
274  #define TFT_BROWN       0x9A60      /* 150,  75,   0 */
275  #define TFT_GOLD        0xFEA0      /* 255, 215,   0 */
276  #define TFT_SILVER      0xC618      /* 192, 192, 192 */
277  #define TFT_SKYBLUE     0x867D      /* 135, 206, 235 */
278  #define TFT_VIOLET      0x915C      /* 180,  46, 226 */
```

문자열 표시해보기

먼저 화면에 문자열을 표시해봅니다.

1. 다음과 같이 예제를 작성합니다.
2102.ino

```
01 #include <SPI.h>
02 #include <TFT_eSPI.h>
03
04 TFT_eSPI tft = TFT_eSPI();
05
06 void setup() {
07
08       tft.begin();
09       tft.setRotation(3);
```

```
10
11        tft.fillScreen(TFT_DARKGREY);
12
13        tft.setCursor(0, 10);
14        tft.setTextFont(2);
15        tft.setTextColor(TFT_RED, TFT_BLACK);
16        tft.setTextSize(1);
17        tft.println("Hello World!");
18
19        tft.setTextFont(4);
20        tft.setTextColor(TFT_YELLOW);
21        tft.println(1234.56);
22
23 }
24
25 void loop() {
26
27 }
```

01 : SPI.h 파일을 포함합니다. ST7735 driver chip은 아두이노 ESP32 와 SPI 통신을 수행합니다.
02 : TFT_eSPI.h 파일을 포함합니다.
04 : TFT_eSPI 객체를 생성한 후, tft 변수에 할당합니다.
08 : begin 함수를 호출하여 TFT LCD를 초기화합니다.
09 : setRotation 함수를 호출하여 LCD의 출력 방향을 3으로 설정합니다. 설정 값은 0, 1, 2, 3을 가질 수 있습니다. 설정 값에 따라 LCD 출력 기준점이 달라집니다. 설정 값의 역할을 보기 위해 값을 변경해 봅니다.
11 : fillScreen 함수를 호출하여 화면의 색깔을 어두운 회색으로 설정합니다. 여기서는 15줄에서 수행되는 글자 배경색 설정의 효과를 보기 위해 어두운 회색으로 설정합니다.
13 : setCursor 함수를 호출하여 글자 출력의 픽셀 위치를 설정합니다. 첫 번째 인자는 가로 픽셀 위치, 두 번째 인자는 세로 픽셀 위치를 나타냅니다. 여기서는 가로 0픽셀, 세로 10픽셀 위치에 커서를 설정합니다.
14 : setTextFont 함수를 호출하여 폰트를 설정합니다. 폰트는 Setup43_ST7735_ESP32_80x160.h 파일에 정의되며 1, 2, 4, 6, 7, 8 값으로 설정할 수 있습니다. 6, 7, 8의 경우 숫자만 표시됩니다.
15 : setTextColor 함수를 호출하여 글자의 색깔을 설정합니다. 표시될 글자의 색깔은 빨간색, 배경색은 검정색으로 설정합니다.
16 : setTextSize 함수를 호출하여 글자의 크기를 설정합니다. 이 함수는 기본 폰트가 표시될 픽셀의 배수가 됩니다.
17 : println 함수를 호출하여 “Hello World!” 문자열을 출력합니다.
19 : setTextFont 함수를 호출하여 폰트를 설정합니다.
20 : setTextColor 함수를 호출하여 글자의 색깔을 설정합니다. 여기서는 글자의 색깔만 노란

색으로 설정합니다. 이 경우 글자의 배경색은 기본 배경색이 사용됩니다.
21 : println 함수를 호출하여 1234.56 숫자를 출력합니다.

2. 업로드를 수행합니다.



3. 다음과 같이 화면에 문자열이 출력되는 것을 확인합니다.



그림 그려보기

다음은 화면에 그림을 그려봅니다.

1. 다음과 같이 예제를 작성합니다.
2102_2.ino

```
01 #include <SPI.h>
02 #include <TFT_eSPI.h>
03
04 TFT_eSPI tft = TFT_eSPI();
05
06 void setup() {
07
08       tft.begin();
09       tft.setRotation(3);
10
11       tft.fillScreen(TFT_BLACK);
12
13       tft.fillRect(4,4,56,56,TFT_BLUE);
14       tft.drawFastVLine(18,16,44,TFT_BLACK);
15       tft.drawFastVLine(32,4,44,TFT_BLACK);
16       tft.drawFastVLine(46,16,44,TFT_BLACK);
17       tft.fillRect(52,48,4,8,TFT_BLACK);
18
19       tft.setCursor(73, 10);
20       tft.setTextFont(2);
```

```
21        tft.setTextSize(1);
22        tft.setTextColor(TFT_BLUE);
23        tft.println("MicroPython");
24        tft.setCursor(73, 25);
25        tft.println("ST7735");
26        tft.setCursor(73, 40);
27        tft.println("LCD 80x160");
28
29 }
30
31 void loop() {
32
33 }
```

13 : fillRect 함수를 호출하여 사각영역을 파란색으로 채웁니다. 첫 번째, 두 번째 인자는 가로 픽셀, 세로 픽셀의 위치, 세 번째, 네 번째 인자는 픽셀 단위의 가로 크기, 세로 크기, 다섯 번째 인자는 색깔을 나타냅니다.
14~16 : drawFastVLine 함수를 호출하여 수직선을 그립니다. 첫 번째, 두 번째 인자는 가로 픽셀, 세로 픽셀의 위치, 세 번째 인자는 픽셀 단위의 세로 길이, 네 번째 인자는 색깔을 나타냅니다.
17 : fillRect 함수를 호출하여 사각영역을 검정색으로 채웁니다.

2.  버튼을 눌러 프로그램을 실행시킵니다. 다음과 같이 화면에 그림이 표시되는 것을 확인합니다.



픽셀 찍어보기

여기서는 픽셀단위로 색깔을 칠해 봅니다.

1. 다음과 같이 예제를 작성합니다.
2102_3.ino

```
01 #include <SPI.h>
02 #include <TFT_eSPI.h>
03
04 TFT_eSPI tft = TFT_eSPI();
```

```
05
06 void setup() {
07
08       tft.begin();
09       tft.setRotation(3);
10
11       tft.fillScreen(TFT_BLACK);
12
13       for(int y=0;y<80;y++) {
14                for(int x=0;x<160;x++) {
15                         tft.drawPixel(x, y, TFT_GREEN);
16                         delay(1);
17                }
18       }
19
20 }
21
22 void loop() {
23
24 }
```

11 : fillScreen 함수를 호출하여 화면을 검정색으로 설정합니다.
13 : 0에서 80 미만의 정수 y에 대하여 14~17줄을 수행합니다.
14 : 0에서 160 미만의 정수 x에 대하여 15~16줄을 수행합니다.
15 : drawPixel 함수를 호출하여 x, y 위치의 픽셀을 초록색으로 설정합니다.
16 : delay 함수를 호출하여 1밀리 초 기다립니다.

2. 업로드를 수행합니다.



3. 화면에 픽셀 단위로 가로로 차례대로 색깔이 표시되는 것을 확인합니다.



LCD 귀퉁이 점찍어보기

여기서는 LCD의 네 귀퉁이를 확인해 봅니다.

1. 다음과 같이 예제를 작성합니다.

2102_4.ino

```
01 #include <SPI.h>
02 #include <TFT_eSPI.h>
03
04 TFT_eSPI tft = TFT_eSPI();
05
06 void setup() {
07
08       tft.begin();
09       tft.setRotation(3);
10
11       tft.fillScreen(TFT_BLACK);
12
13       for(int y=0;y<10;y++) {
14                for(int x=0;x<10;x++) {
15                         tft.drawPixel(x, y, TFT_RED);
16                }
17       }
18
19       for(int y=70;y<80;y++) {
20                for(int x=0;x<10;x++) {
21                         tft.drawPixel(x, y, TFT_GREEN);
22                }
23       }
24
25       for(int y=70;y<80;y++) {
26                for(int x=150;x<160;x++) {
27                         tft.drawPixel(x, y, TFT_BLUE);
28                }
29       }
30
31       for(int y=0;y<10;y++) {
32                for(int x=150;x<160;x++) {
33                         tft.drawPixel(x, y, TFT_YELLOW);
34                }
35       }
36
37 }
```

```
38
39 void loop() {
40
41 }
```

2. 업로드를 수행합니다.



3. 다음과 같이 화면에 네 귀퉁이에 색깔이 표시되는 것을 확인합니다.



빨간색 위치가 (0, 0), 초록색 위치가 (0, 79), 파란색 위치가 (159, 79), 노란색 위치가 (159, 0)인 것을 확인합니다.

기타 함수 사용해 보기

앞에서 사용하지 않은 몇 가지 주요 함수를 사용해 봅니다.

1. 다음과 같이 예제를 작성합니다.
2102_5.ino

```
01 #include <SPI.h>
02 #include <TFT_eSPI.h>
03
04 TFT_eSPI tft = TFT_eSPI();
05
06 void setup() {
07
08       tft.begin();
09       tft.setRotation(3);
10
11       tft.fillScreen(TFT_BLACK);
12
13       tft.drawLine(0,0,159,79,TFT_RED);
14       tft.drawFastHLine(0,40,159,TFT_BLUE);
15       tft.drawRect(10,50,20,10,TFT_GREEN);
```

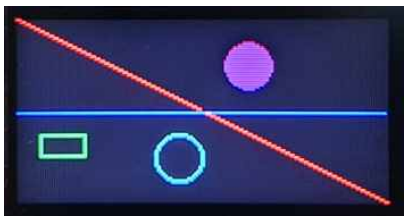
```
16        tft.drawCircle(70,60,10,TFT_CYAN);
17        tft.fillCircle(100,20,10,TFT_PURPLE);
18
19 }
20
21 void loop() {
22
23 }
```

13 : drawLine 함수를 호출하여 선을 그려봅니다.
14 : drawFastHLine 함수를 호출하여 수평선을 그려봅니다.
15 : drawRect 함수를 호출하여 직사각형을 그려봅니다.
16 : drawCircle 함수를 호출하여 원을 그려봅니다.
17 : fillCircle 함수를 호출하여 채워진 원을 그려봅니다.

2. 업로드를 수행합니다.



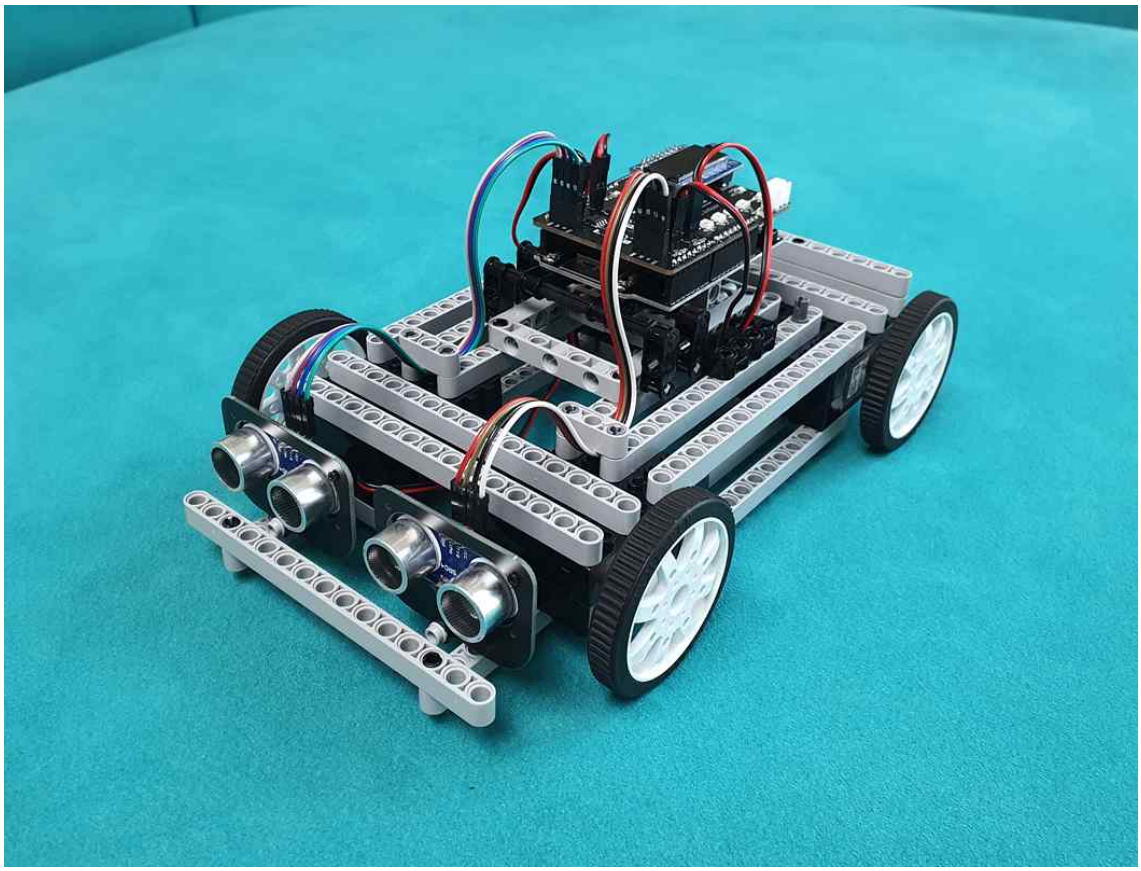
3. 다음과 같이 화면이 표시되는 것을 확인합니다.



Chapter 03

초음파 센서 인공지능 자동차 프로젝트

여기서는 초음파 센서 입력에 따라 방향을 예측하는 인공지능 자동차 프로젝트를 수행합니다. 다음 자동차를 이용하여 인공 신경망을 학습시켜 초음파 센서 입력에 따라 자율주행을 수행하게 합니다.



01 데이터 수집하기

여기서는 학습 데이터를 수집합니다. 먼저 함수 기반의 초음파 센서 자동차 테스트를 수행하고, 다음으로 함수 기반의 초음파 센서 자동차로부터 초음파 센서 데이터와 전진, 좌회전, 우회전의 방향 데이터를 수집합니다. 초음파 센서 데이터는 인공 지능의 입력 데이터가 되며, 방향 데이터는 목표 데이터가 됩니다.

01 초음파 센서 자동차 테스트하기

여기서는 초음파 센서를 이용하여 좌측과 우측에 있는 사물의 거리를 측정하여 회피하는 자동차 루틴을 작성해 봅니다.

1. 제공되는 소스에서 [UltraSonic_RC_CAR_exercise] 예제를 엽니다.

2. 다음과 같이 예제가 열리는 것을 확인합니다.



3. UltraSonic_RC_CAR_exercise를 다음과 같이 작성합니다. 은색 테두리 부분을 완성합니다.

UltraSonic_RC_CAR_exercise

```
71 const int SAFEDISTANCE = 20;
72
73 void setup() {
74   WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0); // brown-out reset disable
75
76   Serial.begin(115200);
77   initMotor();
78   initHCSR04();
79 }
80
81 const int SAMPLE_COUNT = 200;
82
83 void loop() {
84   for(int i=0;i<SAMPLE_COUNT;i++) {
85     int left = 0, right = 0;
86     getDistanceCM(left, right);
87
88     Serial.print("right: ");
89     Serial.print(right);
90     Serial.print(" left: ");
91     Serial.print(left);
92     Serial.println();
93
94     if(left<SAFEDISTANCE||right<SAFEDISTANCE) {
95       if(left>right) {
96         turnLeft();
97       } else {
98         turnRight();
99       }
100      delay(500);
101    } else {
102      goForward();
103      delay(100);
104    }
105  }
106
107  stopMotor();
108  for(;;) ;
109 }
```

71 : SAFEDISTANCE 상수는 초음파 센서 자동차가 방향 전환을 수행할 물체와의 기준 거리입니다. 여기서는 20cm로 초기화하고 있습니다. 15나 25 등으로 변경하여 물체와의 기준 거리를 조절할 수 있습니다.

81 : SAMPLE_COUNT는 초음파 센서 자동차가 거리를 측정할 회수입니다. 여기서는 200회로 초기화하고 있습니다. 100회나 300회 등으로 조절할 수 있습니다.

84 : 85~104줄을 SAMPLE_COUNT 만큼 수행합니다.

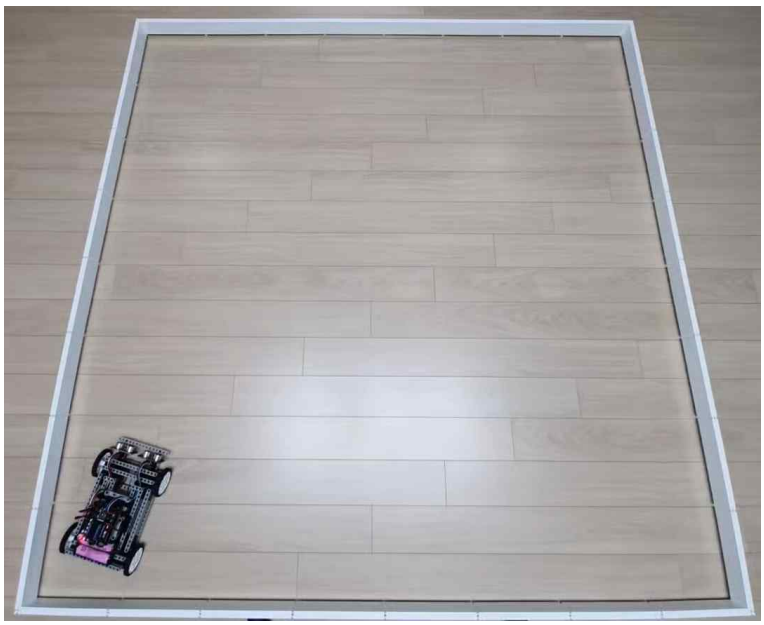
86 : getDistanceCM 함수를 호출하여 거리를 측정한 후, 좌우의 거리를 left, right 변수에

저장합니다.
94 : 왼쪽이나 오른쪽의 거리가 SAFEDISTANCE 이하이면 95~100 줄을 수행합니다.
101 : 그렇지 않으면 102~103줄을 수행합니다.
95 : 왼쪽이 오른쪽보다 멀면
96 : 왼쪽으로 돌고
97 : 그렇지 않으면
98 : 오른쪽으로 돕니다.
100 : 500 밀리초 동안 기다립니다. 이 시간동안 회전을 수행합니다.
102 : 전진을 수행합니다.
103 : 100 밀리초 동안 기다립니다. 이 시간동안 전진을 수행합니다.
107 : stopMotor 함수를 호출하여 모터를 멈춥니다.
108 : 현재 위치를 무한 반복합니다.

2. 업로드를 수행합니다.

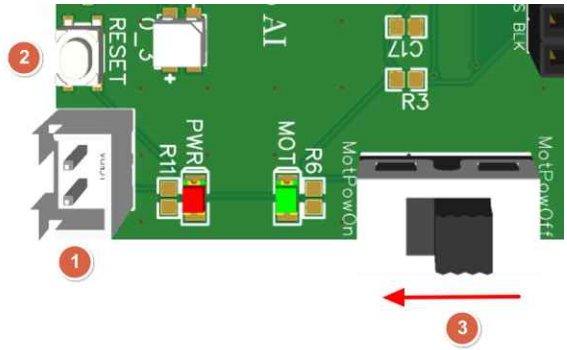


3. 다음과 같이 자동차를 시험장에 놓습니다.



*** 시험장은 종이박스, 페트병, 스티로폼, 적당한 높이의 목재 등을 이용하여 만들 수 있습니다. 사진에서 시험장의 한 면의 길이는 약 1.4m입니다.

4. 다음 순서로 테스트를 수행합니다.



자동차를 바닥에 둔채 ❶ 배터리 전원 단자를 연결을 확인합니다. ❷ 리셋 버튼을 누른 상태로 ❸ DC 모터 전원 스위치를 MotPowOn으로 위치시킵니다. 리셋 버튼을 뗍니다.

*** 소스와 함께 제공되는 car_by_function_ultrasonic.mp4 파일로 테스트 동영상이 제공됩니다.

*** 제공되는 소스에 UltraSonic_RC_CAR는 완성된 예제입니다. 독자 여러분이 작성한 예제가 정상적으로 동작하지 않을 경우 비교해 봅니다.

02 데이터 수집하기

여기서는 초음파 센서 데이터와 방향 데이터를 수집해 봅니다.

1. 제공되는 소스에서 [UltraSonic_RC_CAR_Get_Data_exercise] 예제를 엽니다.

2. 다음과 같이 예제가 열리는 것을 확인합니다.



3. UltraSonic_RC_CAR_Get_Data_exercise를 다음과 같이 작성합니다. 은색 테두리 부분을 완성합니다.

UltraSonic_RC_CAR_Get_Data_exercise

```
71 const int SAFEDISTANCE = 20;
72
73 void setup() {
74   WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0); // brown-out reset disable
75
76   Serial.begin(115200);
77   initMotor();
78   initHCSR04();
79 }
80
81 const int SAMPLE_COUNT = 1000;
82 const int INPUT_NODES = 2;
83 const int OUTPUT_NODES = 3;
84
85 long input[SAMPLE_COUNT][INPUT_NODES] = {0,};
86 long target[SAMPLE_COUNT][OUTPUT_NODES] = {0,};
87
88 void loop() {
89   for(int i=0;i<SAMPLE_COUNT;i++) {
90     int left = 0, right = 0;
91     getDistanceCM(left, right);
92     input[i][0] = left;
93     input[i][1] = right;
94
95     Serial.print(i);
96     Serial.print("right: ");
97     Serial.print(right);
98     Serial.print(" left: ");
99     Serial.print(left);
100    Serial.print(", ");
101
102    if(left<SAFEDISTANCE||right<SAFEDISTANCE) {
103      if(left>right) {
104        Serial.println("turn left");
105        target[i][0] = 1;
106        turnLeft();
107      } else {
108        Serial.println("turn right");
109        target[i][2] = 1;
110        turnRight();
```

81 : SAMPLE_COUNT 상수는 수집할 데이터의 개수입니다.
82 : INPUT_NODES 상수는 수집할 초음파 센서 데이터의 한 단위의 개수입니다.
83 : OUTPUT_NODES 상수는 수집할 방향 데이터의 한 단위의 개수입니다.
85 : input 배열은 수집할 초음파 센서 데이터를 저장할 공간입니다.
86 : target 배열은 수집할 방향 데이터를 저장할 공간입니다.

92 : 왼쪽 초음파 센서 데이터를 저장합니다.
93 : 오른쪽 초음파 센서 데이터를 저장합니다.
105 : 왼쪽 방향 데이터를 저장합니다.
109 : 오른쪽 방향 데이터를 저장합니다.

4. 계속해서 은색 테두리 부분을 완성합니다.

UltraSonic_RC_CAR_Get_Data_exercise

```
111       }
112       delay(500);
113     } else {
114       Serial.println("go forward");
115       target[i][1] = 1;
116       goForward();
117       delay(100);
118     }
119   }
120
121   stopMotor();
122   for(;;) {
123     if(Serial.available()>0) {
124       char userInput = Serial.read();
125       if(userInput == 'p') {
126         // python code
127         printf("# python data\n\n");
128         printf("import numpy as np\n\n");
129         printf("I = np.array([\n");
130         int i;
131         for(i=0;i<SAMPLE_COUNT-1;i++) {
132          printf("\t[[%d,%d]],\n", input[i][0], input[i][1]);
133         }
134         printf("\t[[%d,%d]]\n", input[i][0], input[i][1]);
135         printf("])\n");
136
137         printf("T = np.array([\n");
138         for(i=0;i<SAMPLE_COUNT-1;i++) {
139          printf("\t[[%d,%d,%d]],\n", target[i][0], target[i][1], target[i][2]);
140         }
141         printf("\t[[%d,%d,%d]]\n", target[i][0], target[i][1], target[i][2]);
142         printf("])\n");
143       } else if(userInput == 'c') {
144         // arduino code
145         printf("// c/c++ data\n\n");
146         printf("const double I[][2] = {\n");
147         int i;
148         for(i=0;i<SAMPLE_COUNT-1;i++) {
149          printf("\t{%d,%d},\n", input[i][0], input[i][1]);
150         }
151         printf("\t{%d,%d}\n", input[i][0], input[i][1]);
152         printf("};\n");
153       }
154     }
155   }
156 }
```

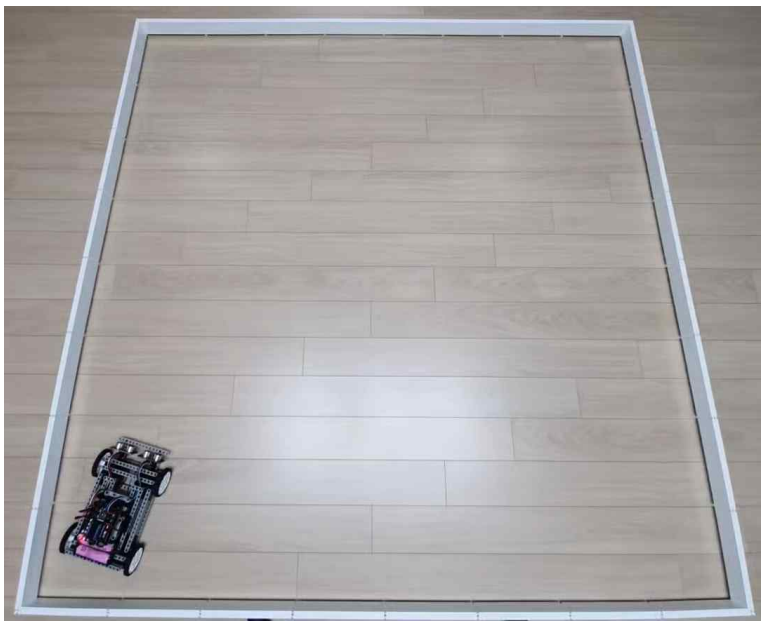
115 : 전진 방향 데이터를 저장합니다.
123 : 시리얼 입력 데이터가 있으면 124~154줄을 수행합니다.
124 : Serial.read 함수를 호출하여 시리얼 입력 데이터를 userInput 변수로 가져옵니다.
125 : 사용자 입력이 p문자이면 126~142줄을 수행합니다. 이 부분은 파이썬 코드를 생성하는 부분으로 입력 데이터와 목표 데이터에 대한 파이썬 코드를 생성합니다. 여기서 생성한 파이썬 코드는 다음 단원에서 Torch를 이용하여 학습을 수행할 때 사용합니다.
129~135 : 입력 데이터에 대한 파이썬 코드를 생성합니다.
137~142 : 목표 데이터에 대한 파이썬 코드를 생성합니다.
143 : 그렇지 않고 사용자 입력이 c문자이면 144~152줄을 수행합니다. 이 부분은 C/C++ 코드를 생성하는 부분으로 입력 데이터에 대한 C/C++ 코드를 생성합니다. 여기서 생성한

C/C++ 코드는 다음 단원에서 아두이노에서 예측 테스트를 수행할 때 사용합니다.
146~152 : 입력 데이터에 대한 C/C++ 코드를 생성합니다.

5. 업로드를 수행합니다.

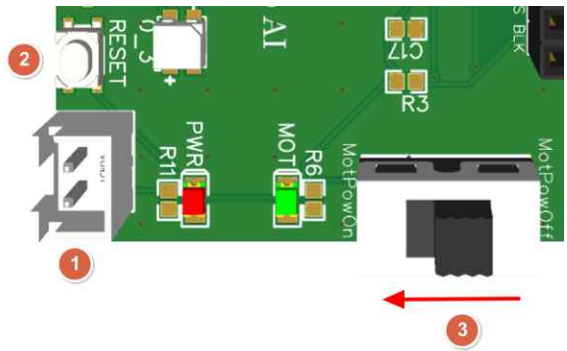


6. 다음과 같이 자동차를 시험장에 놓습니다.



*** 시험장은 종이박스, 페트병, 스티로폼, 적당한 높이의 목재 등을 이용하여 만들 수 있습니다. 사진에서 시험장의 한 면의 길이는 약 1.4m입니다.

7. 다음 순서로 테스트를 수행합니다.



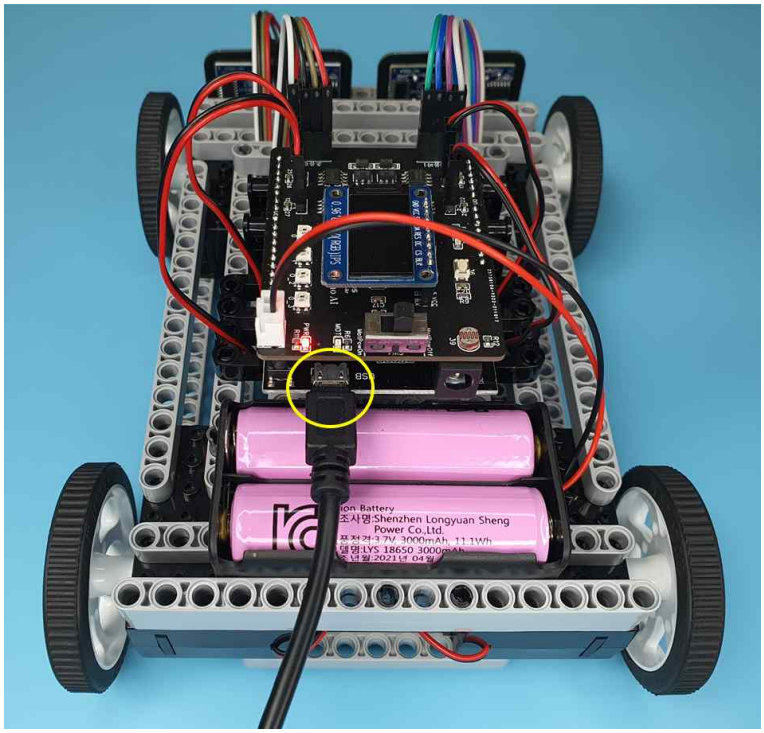
자동차를 바닥에 둔채 ❶ 배터리 전원 단자를 연결을 확인합니다. ❷ 리셋 버튼을 누른 상태로 ❸ DC 모터 전원 스위치를 MotPowOn으로 위치시킵니다. 리셋 버튼을 뗍니다.

*** 소스와 함께 제공되는 data_sampling_ultrasonic.mp4 파일로 테스트 동영상이 제공됩니다.

*** 제공되는 소스에 UltraSonic_RC_CAR_Get_Data는 완성된 예제입니다. 독자 여러분이 작성한 예제가 정상적으로 동작하지 않을 경우 비교해 봅니다.

8. 자동차가 멈출 때까지 기다립니다. 약 2분30초 정도 데이터를 수집한 후, 자동차는 멈춥니다.

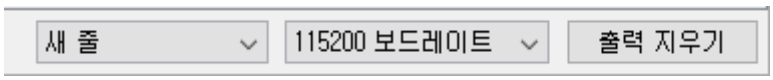
9. 자동차의 USB를 컴퓨터와 연결합니다.



10. [시리얼 모니터] 버튼을 눌러줍니다.



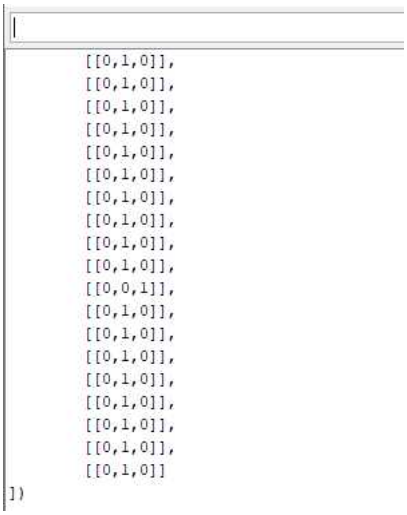
11. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



12. 다음과 같이 시리얼 모니터 입력창에 p를 입력한 후, 엔터키를 누릅니다.



13. 그러면 다음과 같이 수집한 데이터의 파이썬 코드를 출력합니다.

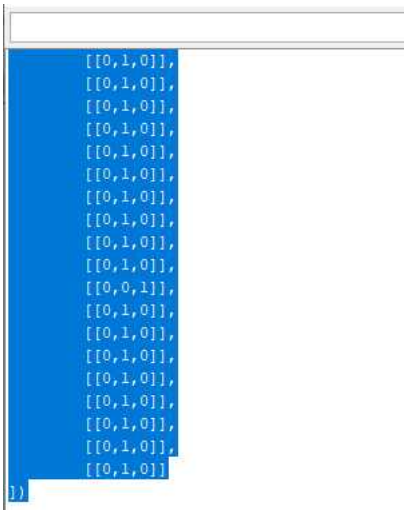


```
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,0,1]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]]
])
```

*** 정상적으로 출력이 되지 않을 경우에는 시리얼 모니터 창을 끈 후, 다시 띄운 후, 12번의 동작을 수행합니다.

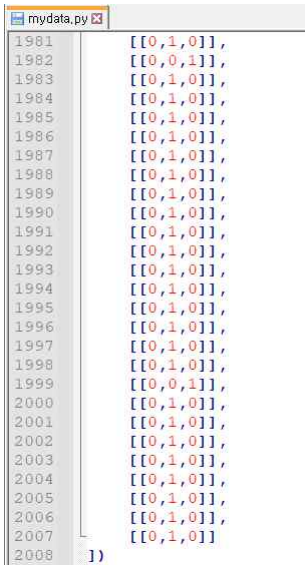
*** 배터리가 방전된 경우에도 정상적으로 출력되지 않을 수 있습니다. 이 경우 배터리를 충전시켜줍니다.

14. 출력된 소스를 전체 선택합니다. 시리얼 모니터 출력창을 마우스 선택한 후, ctrl+a 키를 누르면 전체 선택이 됩니다. 전체 선택이 된 후에는 ctrl+c 키를 눌러 컴퓨터 내부 메모리로 복사합니다.



```
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,0,1]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]],
        [[0,1,0]]
])
```

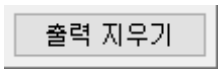
15. 다음과 같이 mydata.py 파일로 적당한 위치에 저장합니다.



*** 이 소스는 바로 다음에 나오는 [02 Torch로 학습시키기] 단원에서 Torch를 이용한 인공지능 학습에 사용됩니다.

*** mydata.py 파일은 소스와 함께 제공되니 참고 바랍니다.

16. C/C++ 소스를 복사하기 위해 시리얼 모니터의 오른쪽 아래에 있는 [출력 지우기] 버튼을 누릅니다.



17. 다음과 같이 시리얼 모니터 입력창에 c를 입력한 후, 엔터키를 누릅니다.

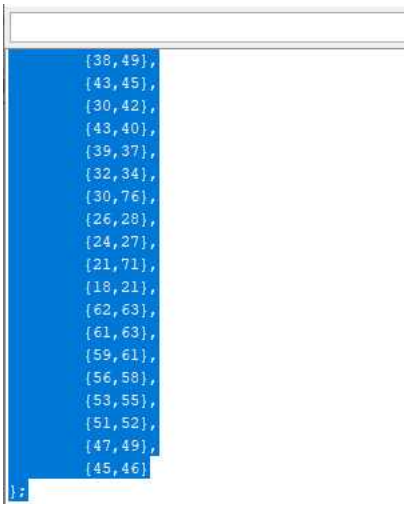


18. 그러면 다음과 같이 수집한 데이터의 C/C++ 코드를 출력합니다.



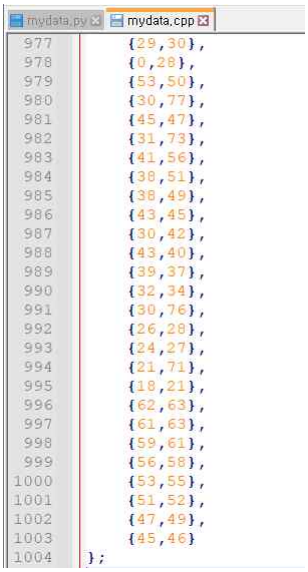
```
        {38,49},
        {43,45},
        {30,42},
        {43,40},
        {39,37},
        {32,34},
        {30,76},
        {26,28},
        {24,27},
        {21,71},
        {18,21},
        {62,63},
        {61,63},
        {59,61},
        {56,58},
        {53,55},
        {51,52},
        {47,49},
        {45,46}
};
```

19. 출력된 소스를 전체 선택합니다. 시리얼 모니터 출력창을 마우스 선택한 후, ctrl+a 키를 누르면 전체 선택이 됩니다. 전체 선택이 된 후에는 ctrl+c 키를 눌러 컴퓨터 내부 메모리로 복사합니다.



```
        {38,49},
        {43,45},
        {30,42},
        {43,40},
        {39,37},
        {32,34},
        {30,76},
        {26,28},
        {24,27},
        {21,71},
        {18,21},
        {62,63},
        {61,63},
        {59,61},
        {56,58},
        {53,55},
        {51,52},
        {47,49},
        {45,46}
};
```

20. 다음과 같이 mydata.py 파일로 적당한 위치에 저장합니다.

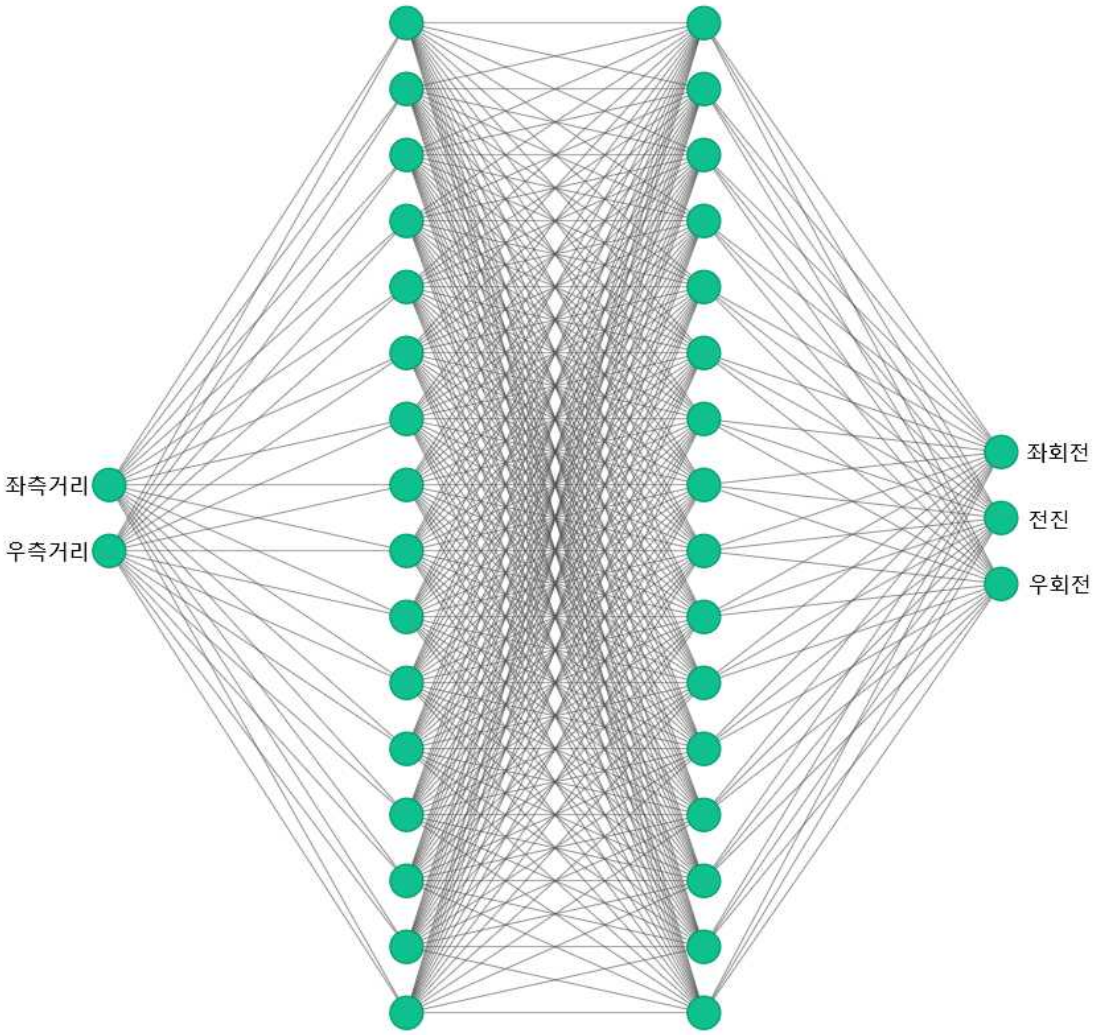


*** 이 소스는 뒤에 나오는 [03 아두이노에서 예측하기] 단원에서 아두이노를 이용한 인공지능 예측 검증에 사용됩니다.

*** mydata.cpp 파일은 소스와 함께 제공되니 참고 바랍니다.

02 Torch로 학습시키기

다음은 Thonny IDE 환경에서 Torch를 이용하여 학습을 수행합니다. 학습된 가중치는 C/C++ 코드로 변환되어 아두이노로 전달되어 예측에 사용됩니다. 사용할 신경망은 다음과 같습니다.



입력 층은 2개의 노드로 구성되며 각각 좌측, 우측에 대한 거리 값을 나타냅니다, 출력 층은 3개의 노드로 구성되며 각각은 이동할 방향을 나타내는 라벨로 좌회전, 전진, 우회전을 나타냅니다. 은닉 층은 2개로 구성되며, 각각의 은닉 층은 16개의 노드로 구성됩니다.

01 수집된 데이터 확인하기

먼저 이전 단원에서 수집한 데이터를 확인해 봅니다.

1. Thonny IDE에서 다음과 같이 예제를 작성합니다.

```
01 import numpy as np
02 import torch
03 import matplotlib.pyplot as plt
04 from mydata import I, T
05
06 x_train = torch.from_numpy(I).float()
07 y_train = torch.from_numpy(T).float()
08
09 x_train = x_train / 250
10
11 print(I.shape)
12 print(T.shape)
13
14 print(I.shape[0], I.shape[1], I.shape[2])
15
16 x_train = x_train.reshape(I.shape[0], I.shape[2])
17 y_train = y_train.reshape(T.shape[0], T.shape[2])
18
19 print(x_train.shape)
20 print(y_train.shape)
```

04 : mydata 모듈로부터 I, T 변수를 불러옵니다.

06 : torch.from_numpy 함수를 호출하여 NumPy 배열 I를 텐서로 변환한 후, float 함수로 32비트 실수형으로 바꿔 x_train 변수를 초기화합니다.

07 : 같은 방식으로 NumPy 배열 T를 32비트 실수형 텐서로 변환하여 y_train 변수를 초기화합니다.

09 : x_train 값을 250으로 나누어 0.0~1.0 사이의 값이 되도록 합니다.

11 : print 함수를 호출하여 I의 모양을 출력합니다.

12 : print 함수를 호출하여 T의 모양을 출력합니다.

14 : print 함수를 호출하여 I.shape의 0, 1, 2 항목 값을 출력합니다.

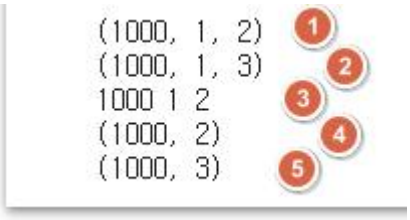
16 : reshape 함수를 호출하여 x_train 텐서의 모양을 2차원으로 변경합니다.

17 : reshape 함수를 호출하여 y_train 텐서의 모양을 2차원으로 변경합니다.

19 : print 함수를 호출하여 x_train의 모양을 출력합니다.

20 : print 함수를 호출하여 y_train의 모양을 출력합니다.

2.  버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.



❶은 I의 모양으로 3차원 배열입니다. 데이터의 개수는 1000개이며 1개의 데이터는 1x2 배열입니다. ❷는 T의 모양으로 3차원 배열입니다. 데이터의 개수는 1000개이며 1개의 데이터는 1x3 배열입니다. ❸은 I.shape의 0, 1, 2 항목 값을 나타냅니다. ❹는 x_train의 모양으로 2차원 배열입니다. ❺는 y_train의 모양으로 2차원 배열입니다.

02 학습 시키고 모델 내보내기

다음은 인공 신경망을 준비하여 학습을 수행한 후, 수행 결과를 저장합니다.

1. Thonny IDE에서 다음과 같이 예제를 수정합니다.

```
001 import numpy as np
002 import torch
003 import torch.nn as nn
004 from torch.utils.data import TensorDataset, DataLoader
005 import matplotlib.pyplot as plt
006 from mydata import I, T
007
008 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
009
010 # ---------------- 데이터 준비 ----------------
011 x_train = torch.from_numpy(I).float()
012 y_train = torch.from_numpy(T).float()
013
014 x_train = x_train / 250
015
016 print(I.shape)
017 print(T.shape)
018
019 print(I.shape[0], I.shape[1], I.shape[2])
020
021 x_train = x_train.reshape(I.shape[0], I.shape[2])
022 y_train = y_train.reshape(T.shape[0], T.shape[2])
023
024 print(x_train.shape)
```

```
025 print(y_train.shape)
026
027 # Keras fit() 기본값: batch_size=32, shuffle=True
028 train_loader = DataLoader(TensorDataset(x_train, y_train),
029                                                 batch_size=32, shuffle=True)
030
031 # ---------------- 모델 정의 ----------------
032 model = nn.Sequential(
033     nn.Linear(I.shape[2], 16),
034     nn.ReLU(),
035     nn.Linear(16, 16),
036     nn.ReLU(),
037     nn.Linear(16, T.shape[2]),
038     nn.Sigmoid(),
039 )
040
041 # Keras Dense 기본 초기화(glorot_uniform, bias=0)와 맞춤
042 def init_keras_style(m):
043     if isinstance(m, nn.Linear):
044             nn.init.xavier_uniform_(m.weight)
045             nn.init.zeros_(m.bias)
046
047 model.apply(init_keras_style)
048 model.to(device)
049
050 # Keras Adam 기본값: lr=0.001, epsilon=1e-7
051 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, eps=1e-7)
052 criterion = nn.MSELoss()
053 mae_fn = nn.L1Loss()
054
055
056 def evaluate(x, y):
057     model.eval()
058     with torch.no_grad():
059             pred = model(x.to(device))
060             y = y.to(device)
061             return criterion(pred, y).item(), mae_fn(pred, y).item()
062
063
064 # ---------------- 학습 ----------------
```

```
065 EPOCHS = 2000
066 history = {"loss": [], "val_loss": [], "mae": [], "val_mae": []}
067
068 for epoch in range(1, EPOCHS + 1):
069     model.train()
070     sum_loss, sum_mae, n = 0.0, 0.0, 0
071     for xb, yb in train_loader:
072             xb, yb = xb.to(device), yb.to(device)
073
074             optimizer.zero_grad()
075             pred = model(xb)
076             loss = criterion(pred, yb)
077             loss.backward()
078             optimizer.step()
079
080             bs = xb.size(0)
081             sum_loss += loss.item() * bs
082             sum_mae += mae_fn(pred, yb).item() * bs
083             n += bs
084
085     # 원본과 동일하게 validation_data=(x_train, y_train)
086     val_loss, val_mae = evaluate(x_train, y_train)
087
088     history["loss"].append(sum_loss / n)
089     history["mae"].append(sum_mae / n)
090     history["val_loss"].append(val_loss)
091     history["val_mae"].append(val_mae)
092
093     if epoch % 100 == 0 or epoch == 1:
094             print(f"Epoch {epoch}/{EPOCHS}"
095                   f" - loss: {sum_loss / n:.4f} - mae: {sum_mae / n:.4f}"
096                   f" - val_loss: {val_loss:.4f} - val_mae: {val_mae:.4f}")
097
098 # ---------------- 학습 곡선 ----------------
099 loss = history["loss"]
100 val_loss = history["val_loss"]
101
102 epochs = range(1, len(loss) + 1)
103
104 plt.plot(epochs, loss, 'g.', label='Training loss')
```

```
105 plt.plot(epochs, val_loss, 'b', label='Validation loss')
106 plt.title('Training and validation loss')
107 plt.xlabel('Epochs')
108 plt.ylabel('Loss')
109 plt.legend()
110 plt.show()
111
112 # ---------------- 평가 / 예측 ----------------
113 np.set_printoptions(formatter={'float_kind': lambda x: "{0:6.4f}".format(x)})
114
115 loss, mae = evaluate(x_train, y_train)
116 print(f"loss: {loss:.4f} - mae: {mae:.4f}")
117
118 model.eval()
119 with torch.no_grad():
120     predictions = model(x_train.to(device)).cpu().numpy()
121
122 print(y_train[976:].numpy())
123 print(predictions[976:])
124
125 # ---------------- 저장 ----------------
126 torch.save(model.state_dict(), "model.pt")
```

003 : torch.nn 모듈을 nn이라는 이름으로 불러옵니다. 신경망 층과 손실 함수가 들어 있습니다.

004 : 데이터를 묶어 주는 TensorDataset 클래스와, 데이터를 배치 단위로 나누어 공급하는 DataLoader 클래스를 불러옵니다.

008 : GPU(CUDA)를 사용할 수 있으면 cuda를, 그렇지 않으면 cpu를 연산 장치로 선택하여 device 변수에 할당합니다.

028~029 : TensorDataset으로 x_train과 y_train을 입력-목표값 쌍으로 묶은 후, DataLoader로 32개씩 배치를 구성합니다. shuffle=True로 설정하여 매 학습 회차마다 데이터 순서를 섞습니다.

032 : nn.Sequential 객체를 생성한 후, model 변수에 할당합니다. 괄호 안에 나열한 층이 순서대로 연결됩니다.

033~034 : 첫 번째 은닉층을 추가합니다. 입력 노드 개수는 I.shape[2], 출력 노드 개수는 16이며, 활성화 함수는 ReLU를 사용합니다.

035~036 : 두 번째 은닉층을 추가합니다. 두 번째 은닉층도 16개의 노드로 구성되며, 활성화 함수는 ReLU를 사용합니다.

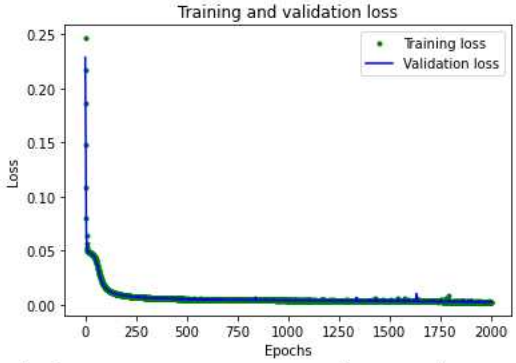
037~038 : 출력층을 추가합니다. 출력층의 노드 개수는 T.shape[2]로 설정하며, 활성화 함수는 Sigmoid를 사용합니다.

042~045 : init_keras_style 함수를 정의합니다. 인자로 받은 층이 nn.Linear이면 가중치를 xavier_uniform 방식으로, 편향을 0으로 초기화합니다.

047 : apply 함수를 호출하여 model의 모든 층에 init_keras_style 함수를 적용합니다.
048 : to 함수를 호출하여 model을 008줄에서 선택한 연산 장치로 옮깁니다.
051 : 최적화 함수로 Adam을 생성한 후, optimizer 변수에 할당합니다. 학습률은 0.001로 설정합니다.
052 : 손실 함수로 평균 제곱 오차(MSE)를 사용합니다.
053 : 예측값과 목표값의 차이(Error)를 절대값으로 변환해 평균화하는 평균 절대 오차(MAE)를 성능 지표로 사용합니다.
056~061 : evaluate 함수를 정의합니다. model을 평가 모드로 바꾸고, 기울기 계산 없이 예측을 수행한 후, 손실값과 MAE 값을 반환합니다.
065 : 학습 회수를 2000으로 설정합니다.
066 : 학습 손실값, 검증 손실값, MAE 값을 저장할 history 딕셔너리를 만듭니다.
068 : 2000회 학습을 반복합니다.
069 : model을 학습 모드로 바꿉니다.
070 : 한 회차의 손실값 합계, MAE 합계, 데이터 개수를 0으로 초기화합니다.
071 : train_loader에서 32개씩 배치를 꺼내 반복합니다.
072 : 배치 데이터를 연산 장치로 옮깁니다.
074 : 이전 배치에서 계산된 기울기를 0으로 초기화합니다.
075 : 배치 입력값으로 예측을 수행합니다.
076 : 예측값과 목표값으로 손실값을 계산합니다.
077 : backward 함수를 호출하여 역전파로 기울기를 계산합니다.
078 : step 함수를 호출하여 계산된 기울기로 가중치를 갱신합니다.
080~083 : 배치 크기를 곱해 손실값과 MAE 값을 누적합니다. 마지막 배치의 크기가 작아도 정확한 평균을 구하기 위해서입니다.
086 : evaluate 함수를 호출하여 검증 손실값과 검증 MAE 값을 계산합니다.
088~091 : 한 회차의 평균 손실값, 평균 MAE 값, 검증 결과를 history에 저장합니다.
093~096 : 첫 회차와 100회차마다 학습 진행 상황을 출력합니다.
099 : 학습 수행 중 손실값을 받습니다.
100 : 검증 용 손실값을 받습니다.
102 : epochs 변수를 선언한 후, 1~2000 범위의 정수값으로 초기화합니다.
104~110 : 학습 수행 손실값과 검증 용 손실값 그래프를 그립니다.
113 : set_printoptions 함수를 호출하여 NumPy의 실수 출력 방법을 변경합니다. 이 예제에서는 소수점 이하 4자리까지 출력합니다.
115~116 : evaluate 함수를 호출하여 손실값과 MAE 값을 계산한 후 출력합니다.
118~120 : model을 평가 모드로 바꾼 후, x_train 값에 대한 예측을 수행합니다. 결과를 CPU로 옮기고 NumPy 배열로 변환하여 predictions 변수로 받습니다.
122 : y_train의 976~999 항목값을 출력합니다.
123 : predictions의 976~999 항목값을 출력합니다.
126 : save 함수를 호출하여 model의 가중치와 편향을 model.pt 파일로 저장합니다.

2.  버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

```
Epoch 1998/2000
32/32 [==============================] - 0s 5ms/step - loss: 0.0025 - mae: 0.0054 - val_loss: 0.0024 - val_mae: 0.0051
Epoch 1999/2000
32/32 [==============================] - 0s 5ms/step - loss: 0.0024 - mae: 0.0051 - val_loss: 0.0024 - val_mae: 0.0050
Epoch 2000/2000
32/32 [==============================] - 0s 5ms/step - loss: 0.0025 - mae: 0.0053 - val_loss: 0.0027 - val_mae: 0.0057
```




```
32/32 [==============================] - 0s 1ms/step - loss: 0.0027 - mae: 0.0057
```

학습이 2000번 수행되는 것을 확인합니다. 학습이 진행됨에 따라 손실값이 0에 수렴하는 것을 확인합니다.

```
[[0 1 0]     [[0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0001 1.0000 0.0000]
 [0 1 0]      [0.0015 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0165 1.0000 0.0000]
 [0 1 0]      [0.0228 1.0000 0.0000]
 [0 1 0]      [0.0000 0.9929 0.2478]
 [0 0 1]      [0.0055 0.0001 0.9980]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]      [0.0000 1.0000 0.0000]
 [0 1 0]]     [0.0000 1.0000 0.0000]]
```

y_train의 976~999 항목값과 predictions의 976~999 항목값을 비교합니다.

03 NumPy로 예측해 보기

다음은 이전 예제에서 학습한 결과를 읽어와 학습된 가중치를 얻어온 후, NumPy를 이용하여 예측을 수행해 봅니다.

1. Thonny IDE에서 다음과 같이 예제를 작성합니다.

```
01 import numpy as np
02 import torch
03 from mydata import I, T
04
05 np.set_printoptions(formatter={'float_kind': lambda x: "{0:6.4f}".format(x)})
06
07 x_train = I
08
09 x_train = x_train / 250
10
11 # 모델 클래스 없이 state_dict만 읽어 가중치를 꺼냄
12  # nn.Sequential 인덱스: 0=Linear, 1=ReLU, 2=Linear, 3=ReLU, 4=Linear,
5=Sigmoid
13 state = torch.load("model.pt", map_location="cpu", weights_only=True)
14
15 # nn.Linear weight는 (out, in) 형태 → Keras와 같은 (in, out)으로 전치
16 W1, b1 = state["0.weight"].numpy().T, state["0.bias"].numpy()
17 W2, b2 = state["2.weight"].numpy().T, state["2.bias"].numpy()
18 W3, b3 = state["4.weight"].numpy().T, state["4.bias"].numpy()
19
20 print(W1.shape, b1.shape)
21 print(W2.shape, b2.shape)
22 print(W3.shape, b3.shape)
23
24 xs = x_train
25 ys = []
26 for x in xs:
27       x = np.array([x])        # x should be array
28       h1 = x @ W1 + b1         # dense layer
29       h1 = np.maximum(0, h1)  # ReLU
30       h2 = h1 @ W2 + b2        # dense layer
31       h2 = np.maximum(0, h2)  # ReLU
32       h3 = h2 @ W3 + b3        # dense layer
33       h3 = 1 / (1 + np.exp(-h3))
```

```
34        ys.append(h3)
35
36 for y in ys[976:]:
37        print(y)
```

03 : mydata 모듈로부터 I, T 변수를 불러옵니다.
05 : set_printoptions 함수를 호출하여 NumPy의 실수 출력 방법을 변경합니다. 이 예제에서는 소수점 이하 4자리까지 출력합니다.
07 : x_train 변수를 선언한 후, I 값으로 초기화합니다.
09 : x_train 값을 250으로 나누어 0.0~1.0 사이의 값이 되도록 합니다.
13 : torch.load 함수를 호출하여 앞에서 저장한 model.pt 파일을 읽어와 state 변수에 할당합니다. state에는 각 층의 가중치와 편향이 이름-값 쌍으로 들어 있습니다. map_location="cpu"는 GPU에서 학습한 경우에도 CPU로 읽어오도록 하고, weights_only=True는 가중치만 안전하게 읽어오도록 합니다.
16 : 입력층과 첫 번째 은닉층 사이의 가중치와 편향을 읽어와 각각 W1, b1에 할당합니다. numpy 함수로 NumPy 배열로 변환하고, PyTorch의 가중치는 (출력, 입력) 모양으로 저장되어 있으므로 .T로 전치하여 (입력, 출력) 모양으로 바꿉니다.
17 : 첫 번째 은닉층과 두 번째 은닉층 사이의 가중치와 편향을 읽어와 각각 W2, b2에 할당합니다.
18 : 두 번째 은닉층과 출력층 사이의 가중치와 편향을 읽어와 각각 W3, b3에 할당합니다.

※ 16~18줄의 0, 2, 4는 nn.Sequential 안에서 Linear 층의 순서 번호입니다. 1, 3, 5번은 ReLU와 Sigmoid로, 가중치가 없기 때문에 state에 들어 있지 않습니다.

20 : print 함수를 호출하여 W1의 모양과 b1의 모양을 출력합니다.
21 : print 함수를 호출하여 W2의 모양과 b2의 모양을 출력합니다.
22 : print 함수를 호출하여 W3의 모양과 b3의 모양을 출력합니다.
24 : xs 변수를 선언한 후, x_train으로 초기화합니다.
25 : ys 변수를 선언한 후, 빈 리스트로 초기화합니다.
26 : xs 배열의 모든 항목에 대해 순서대로 x로 받아 27~34줄을 수행합니다.
27 : x를 한 차원 감싸 배열로 만듭니다.
28~33 : 순전파를 수행합니다. 28, 30, 32줄은 행렬곱과 편향 덧셈으로 Dense 층을 계산하고, 29, 31줄은 ReLU, 33줄은 Sigmoid 활성화 함수를 계산합니다.
34 : append 함수를 호출하여 h3 출력값을 ys에 추가합니다.
36~37 : ys의 976~999 항목값을 출력합니다.

2.  버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

```
(2, 16) (16,)
(16, 16) (16,)
(16, 3) (3,)
```

W1, b1, W2, b2, W3, b3의 모양을 출력한 결과입니다.

```
[[[0.0000 1.0000 0.0000]]] [[0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0001 1.0000 0.0000]]]  [0.0001 1.0000 0.0000]
[[[0.0015 1.0000 0.0000]]]  [0.0015 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0165 1.0000 0.0000]]]  [0.0165 1.0000 0.0000]
[[[0.0228 1.0000 0.0000]]]  [0.0228 1.0000 0.0000]
[[[0.0000 0.9929 0.2478]]]  [0.0000 0.9929 0.2478]
[[[0.0055 0.0001 0.9980]]]  [0.0055 0.0001 0.9980]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]
[[[0.0000 1.0000 0.0000]]]  [0.0000 1.0000 0.0000]]
```

이전 예제의 예측값과 비교합니다. 예측값이 같은 것을 확인합니다. 여기서는 마지막 24가지 결과에 대한 비교를 하고 있습니다.

04 학습된 가중치 내보내기

다음은 학습된 가중치를 C/C++ 언어 형태로 변환하여 아두이노 스케치에서 사용할 수 있도록 합니다.

1. Thonny IDE에서 다음과 같이 예제를 수정합니다.

```
01 import numpy as np
02 import torch
03 from mydata import I, T
04
05 np.set_printoptions(formatter={'float_kind': lambda x: "{0:6.4f}".format(x)})
06
07 x_train = I
08
09 x_train = x_train / 250
10
11 # 모델 클래스 없이 state_dict만 읽어 가중치를 꺼냄
12 # nn.Sequential 인덱스:
13 # 0=Linear, 1=ReLU, 2=Linear, 3=ReLU, 4=Linear, 5=Sigmoid
14 state = torch.load("model.pt", map_location="cpu", weights_only=True)
15
```

```
16 # nn.Linear weight는 (out, in) 형태 → Keras와 같은 (in, out)으로 전치
17 # 전치 덕분에 아래 flatten() 순서도 Keras 버전과 동일해짐
18 W1, b1 = state["0.weight"].numpy().T, state["0.bias"].numpy()
19 W2, b2 = state["2.weight"].numpy().T, state["2.bias"].numpy()
20 W3, b3 = state["4.weight"].numpy().T, state["4.bias"].numpy()
21
22 print(W1.shape, b1.shape)
23 print(W2.shape, b2.shape)
24 print(W3.shape, b3.shape)
25
26 xs = x_train
27 ys = []
28 for x in xs:
29       x = np.array([x])        # x should be array
30       h1 = x @ W1 + b1         # dense layer
31       h1 = np.maximum(0, h1)   # ReLU
32       h2 = h1 @ W2 + b2        # dense layer
33       h2 = np.maximum(0, h2)   # ReLU
34       h3 = h2 @ W3 + b3        # dense layer
35       h3 = 1 / (1 + np.exp(-h3))
36       ys.append(h3)
37
38 for y in ys[976:]:
39       print(y)
40
41 names = ["WH", "BH", "WM", "BM", "WO", "BO"]
42 arrays = [W1, b1, W2, b2, W3, b3]
43
44 for name, array in zip(names, arrays):
45       print("double %s[] = {" % name)
46       print("    ", ", ".join([str(x) for x in array.flatten()]))
47       print("};\n")
```

41 : names 변수를 선언한 후, 가중치와 편향의 이름으로 초기화합니다.

42 : arrays 변수를 선언한 후, 학습된 가중치와 편향으로 초기화합니다.

44 : zip 함수를 이용하여 names와 arrays의 각 항목을 튜플 형태로 묶은 후, 튜플의 각 항목을 name과 array로 받아서 45~47줄을 수행합니다.

45 : 실수형 배열의 선언부(double 이름[] = {)를 출력합니다.

46 : flatten 함수를 호출하여 array를 1차원 배열로 펼친 후, 각 항목을 문자열로 바꿔 리스트로 만듭니다. join 함수를 호출하여 쉼표(,)를 구분자로 하나의 문자열로 묶은 후, 출력합니다.

※ flatten 함수는 2차원 배열을 행 단위로 차례대로 펼칩니다. 18~20줄에서 가중치를 (입력, 출력) 모양으로 전치해 두었기 때문에, 출력되는 C 배열의 순서가 Keras 버전과 같아집니다. 전치하지 않으면 C 코드는 오류 없이 실행되지만 잘못된 결과를 냅니다.

47 : 닫는 중괄호와 쌍반점(};)을 출력하고, 빈 줄을 하나 추가합니다.

2.  버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

```
double WH[] = {
    2.8323684, 2.4012733, -0.10545254, -0.2799292, -0.3383029, 0.072329916, 0.0012191534, 0.07289905, 0.34818685, 2.791, -0.27060542, 2.
};

double BH[] = {
    -0.112590775, -0.2139325, 0.0, 0.0, -0.016333591, -0.058064755, 0.0, -0.04552145, -0.06808182, -0.12100843, 0.42263436, -0.00853241
};

double WM[] = {
    0.24715245, 3.1301131, -0.15842439, -0.3550299, 2.6825402, -3.1907325, -3.2404928, 0.37368813, 0.21138968, 0.25736266, -0.058925744,
};

double BM[] = {
    -0.046499338, -0.74114496, -0.007860072, 0.0, -0.12035783, 0.25053722, 0.97711813, -0.015782986, -0.029896747, -0.07995288, -0.0292
};

double WO[] = {
    0.40645036, 0.30847904, 0.012811568, -1.2748713, 6.952143, -7.7731304, 0.4049355, 0.3011306, 0.14705916, 0.15849286, 0.41754538, 0.
};

double BO[] = {
    1.2746218, -1.4384696, 0.03391649
};
```

가중치와 편향 값이 C/C++ 배열 형태로 변경되었습니다. C/C++ 배열 형태로 변환된 가중치와 편향값은 바로 다음 단원의 아두이노 스케치에서 사용됩니다.

03 아두이노에서 예측하기

여기서는 Torch를 이용하여 학습시킨 후, C/C++ 형태로 변환된 가중치와 편향을 이용하여 아두이노에서 예측을 수행해 봅니다. 그리고 아두이노 인공지능 자동차에 적용하여 봅니다.

01 아두이노에서 예측해 보기

먼저 C/C++ 형태로 변환된 가중치와 편향을 이용하여 아두이노에서 예측을 수행해 봅니다.

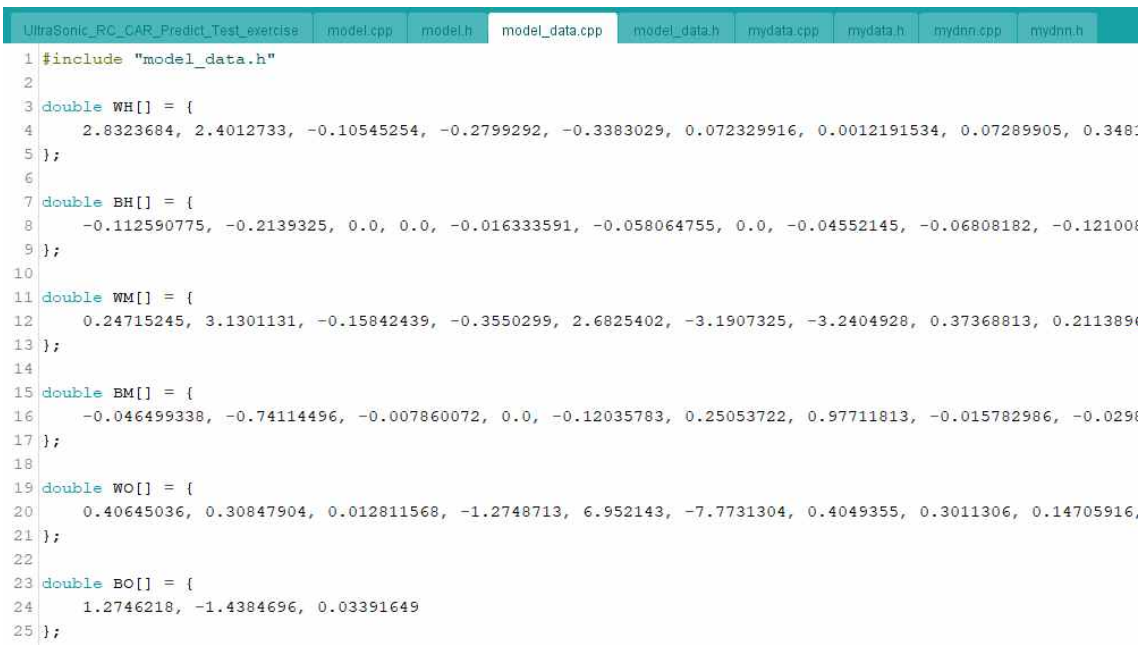
1. 제공되는 소스에서 [UltraSonic_RC_CAR_Predict_Test_exercise] 예제를 엽니다.

2. 다음과 같이 예제가 열리는 것을 확인합니다.



mydnn.cpp, mydnn.h : [아두이노로 코딩하며 배우는 딥러닝] 본문에서 정리한 딥러닝 라이브러리를 좀 더 간략하게 정리한 라이브러리입니다.
mydata.cpp, mydata.h : 앞에서 수집한 데이터를 옮겨놓을 파일입니다.
model_data.cpp, model_data.h : C/C++ 형태로 변환된 가중치와 편향을 옮겨놓을 파일입니다.
model.cpp, model.h : 예측 함수를 작성할 파일입니다.
UltraSonic_RC_CAR_Predict_Test_exercise : 예측을 수행할 파일입니다.

3. model_data.cpp 파일에 이전 단원에서 변환한 C/C++ 형태의 가중치와 편향 변수를 복사해 넣습니다.



1 : model_data.h 파일을 포함해 줍니다.

3~25 : C/C++로 변환된 가중치와 편향 변수입니다.

4. model_data.h 파일의 내용을 확인합니다.

UltraSonic_RC_CAR_Predict_Test_exercise | model.cpp | model.h | model_data.cpp | model_data.h | mydata.cpp | mydata.h | mydnn.cpp | mydnn.h

```
1  #ifndef MODEL_DATA_H
2  #define MODEL_DATA_H
3
4  extern double WH[];
5  extern double BH[];
6  extern double WM[];
7  extern double BM[];
8  extern double WO[];
9  extern double BO[];
10
11 const int NUM_I = 2;
12 const int NUM_H = 16;
13 const int NUM_M = 16;
14 const int NUM_O = 3;
15
16 #endif  // MODEL_DATA_H
```

11 : 입력 층 노드의 개수입니다.
12 : 첫 번째 은닉 층 노드의 개수입니다.
13 : 두 번째 은닉 층 노드의 개수입니다.
14 : 출력 층 노드의 개수입니다.

5. model.cpp의 predict 함수를 다음과 같이 작성합니다. 은색 테두리 부분을 완성합니다.

UltraSonic_RC_CAR_Predict_Test_exercise | model.cpp | model.h | model_data.cpp | model_data.h | mydata.cpp | mydata.h | mydnn.cpp | mydnn.h

```
1  #include "model.h"
2  #include "model_data.h"
3  #include "mydnn.h"
4
5  void predict(double * I, double * P) {
6
7    double H[NUM_H];
8    double M[NUM_M];
9
10   forward_Y(I, WH, BH, H, NUM_I, NUM_H);
11   relu_Y(H, NUM_H);
12
13   forward_Y(H, WM, BM, M, NUM_H, NUM_M);
14   relu_Y(M, NUM_M);
15
16   forward_Y(M, WO, BO, P, NUM_M, NUM_O);
17   sigmoid_Y(P, NUM_O);
18
19 }
```

7 : 첫 번째 은닉층의 값을 저장합니다.
8 : 두 번째 은닉층의 값을 저장합니다.
10~17 : 순전파를 수행합니다. 예측값은 P에 저장됩니다.

6. model.h 파일의 내용을 확인합니다.

UltraSonic_RC_CAR_Predict_Test_exercise | model.cpp | model.h | model_data.cpp | model_data.h | mydata.cpp | mydata.h | mydnn.cpp | mydnn.h

```
#ifndef MODEL_H
#define MODEL_H

void predict(double * I, double * P);

#endif  // MODEL_H
```

7. mydata.cpp 파일에 [01 데이터 수집하기] 단원에서 저장한 mydata.cpp 파일의 내용을 복사해 넣습니다.

UltraSonic_RC_CAR_Predict_Test_exercise | model.cpp | model.h | model_data.cpp | model_data.h | mydata.cpp | mydata.h | mydnn.cpp | mydnn.h

```
// c/c++ data

double I[][2] = {
  {50,112},
  {110,114},
  {48,109},
  {52,106},
  {49,104},
  {48,101},
  {96,98},
  {94,96},
  {90,94},
  {88,92},
  {85,88},
  {49,86},
  {46,82},
  {46,80},
  {75,77},
  {47,75},
  {48,72},
  {47,70},
  {64,68},
  {63,64},
  {59,63},
  {57,60},
  {53,60},
  {51,55},
  {50,53},
```

8. mydata.h 파일의 내용을 확인합니다.

UltraSonic_RC_CAR_Predict_Test_exercise | model.cpp | model.h | model_data.cpp | model_data.h | mydata.cpp | mydata.h | mydnn.cpp | mydnn.h

```
#ifndef MY_DATA_H
#define MY_DATA_H

extern double I[][2];

const int SIZE_I = 1000;

#endif  // MY_DATA_H
```

6 : [01 데이터 수집하기] 단원에서 수집한 데이터의 개수입니다.

9. UltraSonic_RC_CAR_Predict_Test_exercise를 다음과 같이 작성합니다. 은색 테두리 부분을 완성합니다.

UltraSonic_RC_CAR_Predict_Test_exercise | model.cpp | model.h | model_data.cpp | model_data.h | mydata.cpp | mydata.h | mydnn.cpp | mydnn.h

```
1  #include "model.h"
2  #include "mydata.h"
3
4  void setup() {
5
6    Serial.begin(115200);
7
8    double P[3];
9
10   for(int n=0;n<SIZE_I;n++) {
11     I[n][0] /= 250.0;
12     I[n][1] /= 250.0;
13
14     predict(I[n], P);
15     if(n>=976)
16       printf("%.4f, %.4f, %.4f\n", P[0], P[1], P[2]);
17   }
18
19 }
20
21 void loop() {
22
23 }
```

11~12 : 입력 데이터를 250.0으로 나누어 0.0~1.0에 가까운 값으로 변환해 줍니다.
14 : predict 함수를 호출하여 예측을 수행합니다.
15~16 : P 배열의 976~999 항목값을 출력합니다.

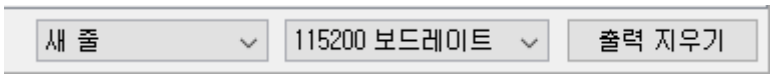
10. 업로드를 수행합니다.



11. [시리얼 모니터] 버튼을 눌러줍니다.



12. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



13. 결과 값을 확인합니다.

```
0.0000, 1.0000, 0.0000 [[0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0001, 1.0000, 0.0000  [0.0001 1.0000 0.0000]
0.0015, 1.0000, 0.0000  [0.0015 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0165, 1.0000, 0.0000  [0.0165 1.0000 0.0000]
0.0228, 1.0000, 0.0000  [0.0228 1.0000 0.0000]
0.0000, 0.9929, 0.2478  [0.0000 0.9929 0.2478]
0.0055, 0.0001, 0.9980  [0.0055 0.0001 0.9980]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]
0.0000, 1.0000, 0.0000  [0.0000 1.0000 0.0000]]
```

이전 예제의 예측값과 비교합니다. 예측값이 같은 것을 확인합니다.

*** 제공되는 소스에 UltraSonic_RC_CAR_Predict_Test는 완성된 예제입니다. 독자 여러분이 작성한 예제가 정상적으로 동작하지 않을 경우 비교해 봅니다.

02 초음파 센서 자동차에 인공지능 적용하기

이제 초음파 센서 자동차에 인공지능을 적용해 봅니다.

1. 제공되는 소스에서 [UltraSonic_RC_CAR_AI_exercise] 예제를 엽니다.

2. 다음과 같이 예제가 열리는 것을 확인합니다.



3. model_data.cpp에 이전 단원에서 변환한 C/C++ 형태의 가중치와 편향 변수를 복사해 넣습니다.

UltraSonic_RC_CAR_AI_exercise | model.cpp | model.h | model_data.cpp | model_data.h | mydnn.cpp | mydnn.h

```
#include "model_data.h"

double WH[] = {
    2.8323684, 2.4012733, -0.10545254, -0.2799292, -0.3383029, 0.072329916, 0.0012191534, 0.07289905, 0.3481
};

double BH[] = {
    -0.112590775, -0.2139325, 0.0, 0.0, -0.016333591, -0.058064755, 0.0, -0.04552145, -0.06808182, -0.12100
};

double WM[] = {
    0.24715245, 3.1301131, -0.15842439, -0.3550299, 2.6825402, -3.1907325, -3.2404928, 0.37368813, 0.2113896
};

double BM[] = {
    -0.046499338, -0.74114496, -0.007860072, 0.0, -0.12035783, 0.25053722, 0.97711813, -0.015782986, -0.0298
};

double WO[] = {
    0.40645036, 0.30847904, 0.012811568, -1.2748713, 6.952143, -7.7731304, 0.4049355, 0.3011306, 0.14705916,
};

double BO[] = {
    1.2746218, -1.4384696, 0.03391649
};
```

4. UltraSonic_RC_CAR_AI_exercise를 다음과 같이 작성합니다. 은색 테두리 부분을 완성합니다.

UltraSonic_RC_CAR_AI_exercise | model.cpp | model.h | model_data.cpp | model_data.h | mydnn.cpp | mydnn.h

```
80 const int SAMPLE_COUNT = 500;
81
82 void loop() {
83   for(int i=0;i<SAMPLE_COUNT;i++) {
84
85     int left = 0, right = 0;
86     getDistanceCM(left, right);
87
88     Serial.print("right: ");
89     Serial.print(right);
90     Serial.print(" left: ");
91     Serial.print(left);
92     Serial.println();
93
94     double I[] = {left/250.0, right/250.0};
95     double P[3] = {};
96
97     predict(I, P);
98
99     printf("%.4f, %.4f, %.4f\n", P[0], P[1], P[2]);
100
101    if(P[0]>P[1] && P[0]>P[2]) {
102      turnLeft();
103      delay(500);
104    } else if(P[2]>P[1]) {
105      turnRight();
106      delay(500);
107    } else {
108      goForward();
109      delay(100);
110    }
111  }
112
113  stopMotor();
114  for(;;) ;
115
116 }
```

94 : I 실수 배열 변수를 선언한 후, left/250.0, right/250.0 값으로 초기화합니다.

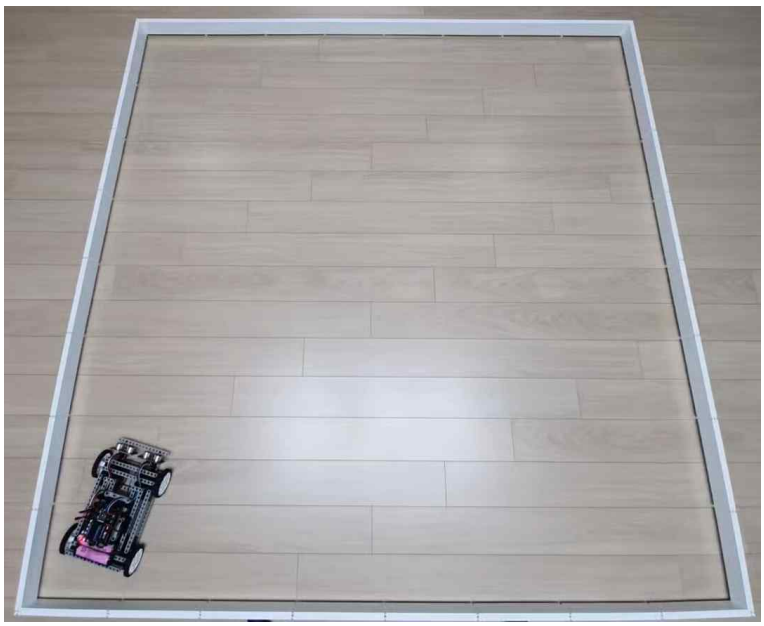
95 : 예측 결과를 저장할 P 실수 배열 변수를 선언합니다.

97 : predict 함수를 호출하여 예측을 수행합니다.

5. 업로드를 수행합니다.

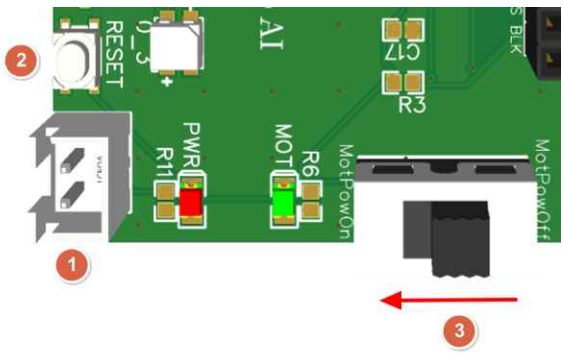


6. 다음과 같이 자동차를 시험장에 놓습니다.



*** 시험장은 종이박스, 페트병, 스티로폼, 적당한 높이의 목재 등을 이용하여 만들 수 있습니다. 사진에서 시험장의 한 면의 길이는 약 1.4m입니다.

7. 다음 순서로 테스트를 수행합니다.



자동차를 바닥에 둔채 ❶ 배터리 전원 단자를 연결을 확인합니다. ❷ 리셋 버튼을 누른 상태로 ❸ DC 모터 전원 스위치를 MotPowOn으로 위치시킵니다. 리셋 버튼을 뗍니다.

*** 소스와 함께 제공되는 car_by_dnn_ultrasonic.mp4 파일로 테스트 동영상이 제공됩니다.

*** 제공되는 소스에 UltraSonic_RC_CAR_AI는 완성된 예제입니다. 독자 여러분이 작성한 예제가 정상적으로 동작하지 않을 경우 비교해 봅니다.